Projektdokumentation

Robotik - Sommersemester 2024
Prof. Dr. Max Fischer

Juni 2024

Inhaltsverzeichnis

Inhaltsverzeichnis 1

1 Projektbeschreibung 4

2 Überblick 4
2.1 Architekturübersicht . . . 4
2.2 Zeitlicher Verlauf . . . 6

3 Projektdurchführung 8
3.1 Organisation . . . 8
3.2 Projektleitende Tätigkeiten . . . 8
3.2.1 Allgemein . . . 8
3.2.2 Top-Down Bottom-Up . . . 9

4 Digitalisierung 11
4.1 Anfertigung der Rohaufnahmen . . . 11
4.2 Perspektivische Korrektur . . . 14
4.3 Segmentierung . . . 15
4.3.1 Segmentierung mit digitaler Bildverarbeitung . . . 15
4.3.2 Segment Anything Model (SAM) . . . 20
4.4 Postprocessing . . . 23
4.4.1 Puzzleteil ohne Hintergrund (RGBA) . . . 23
4.4.2 Kanten Approximieren . . . 23
4.5 ROS-Node . . . 24
4.5.1 Kommunikation . . . 25
4.5.2 Interfaces . . . 25
4.5.3 Architektur . . . 26

5 Solver **27**
5.1 Einleitung . . . 27
5.2 Extractor . . . 29
5.2.1 Motivation . . . 29
5.2.2 Vorgehen . . . 30
5.2.3 Lösung . . . 30
5.3 Matcher . . . 32
5.3.1 Geomatcher . . . 32
5.3.2 Geomatcher RGB . . . 35
5.3.3 ColorMatcher . . . 36
5.3.4 XOR Matcher . . . 39
5.3.5 Evaluierung . . . 42
5.4 Solver Algorithmen . . . 47
5.4.1 Dimensionsbestimmung . . . 47
5.4.2 Tiefensuche . . . 47
5.4.3 Best-First-Algorithmus . . . 51
5.4.4 Visualisierung . . . 53
5.4.5 Simulated Annealing . . . 54
5.4.6 Mock-Puzzle . . . 56
5.4.7 Evaluierung . . . 58
5.5 Postprocessing . . . 61
5.5.1 Linearer Layout Optimierer . . . 61
5.5.2 Non Linearer Layout Optimierer . . . 65
5.5.3 Kombination von lineareren und nichtlineareren Layout Optimierer [Zamani Pozveh] . . . 74
5.6 ROS2 Integration . . . 77
5.6.1 Aufgaben . . . 77
5.6.2 Schnittstellen . . . 77
5.6.3 Node - Ablauf . . . 78
5.6.4 Schnittstellentests . . . 79
5.6.5 Fazit . . . 80

6 Control **80**
6.1 Roboterübersicht . . . 80
6.2 Organisation . . . 80
6.3 Architektur der Control-Node . . . 81
6.4 ROS2 Integration . . . 82
6.4.1 Aufgaben . . . 82
6.4.2 Node - Ablauf . . . 83
6.4.3 Schnittstellentests . . . 84
6.4.4 Fazit . . . 84
6.5 Testing . . . 85
6.5.1 Test Package . . . 85
6.5.2 Unit Tests . . . 86

6.5.3 CI/CD . . . 86
6.5.4 Schnittstellen . . . 87
6.6 Inverse Kinematik . . . 87
6.7 Berechnung des Greifbereichs . . . 90
6.8 Kollisionsprüfung gegenüber der Basis . . . 92
6.9 Robotsbewegung-Wrapper . . . 93
6.10 SLERP - Spherical Linear Interpolation . . . 94
6.11 Interpolation der Bewegungskurve durch den Algorithmus von De Casteljau 95

7 Master Node 98
7.1 Grundidee . . . 98
7.2 User Interface . . . 99
7.3 Watchdog . . . 100

8 Workspace 100

9 Visualizer 100
9.1 Was war das zu lösende Problem? . . . 100
9.2 Was haben wir gemacht? / Wie haben wir die Lösungen umgesetzt / Implementierung . . . 101

10 Ausführen des Programms 104

Literaturverzeichnis 105

Abbildungsverzeichnis 107

1 Projektbeschreibung [Marvin Wolf]

Entwickelt wird ein Roboter, der ein gegebenes Puzzle automatisiert lösen soll. Dieser ist rein virtuell und basiert auf dem Robot Operating System (ROS).
Ein Puzzle der Größe 7x7 wird digitalisiert d.h. jedes einzelne Puzzleteil wird systematisch und möglichst einheitlich abfotografiert und in ein geeignetes Dateiformat umgewandelt. Dies ist ein manueller, einmaliger Prozess, der nicht automatisiert abläuft.
Auf Basis dieser Bilddateien wird nun die korrekte, einzigartige Lösung des Puzzles berechnet. Dabei werden verschiedene Eigenschaften wie die Abbildung auf den Puzzleteilen sowie deren Form miteinbezogen.
Ein KUKA KR6 Roboter soll die Lösung visualisieren. Dieser soll über das Szenegraph-System Coin3D visualisiert und animiert werden. Dementsprechend soll die Darstellung und Bewegung 3-dimensional erfolgen. Die Object-Files, über welche die 7 Komponenten des Roboters in Aussehen und Anordnung definiert werden, lagen bereits vor. Der virtuelle Roboter bewegt jedes Teil einzeln von einem fest definierten Ausgangspunkt an die jeweilige, der Puzzlelösung entsprechende Zielposition.
Die Bewegung soll einer kontinuerlichen und abgerundeten Trajektorie folgen, welche durch die Neuanordnung der Roboterarme realisiert wird. Durch die Nutzung der Denavitt-Hartenberg-Repräsentation für die Anordnung der Koordinatensysteme der einzelnen Arme zueinander können Operationen wie Translation und Rotation über einfache Matrixberechnungen erfolgen. Die Robotics Toolbox von Peter Corke unterstützt bei anderen Aspekten der Roboterbewegung.

2 Überblick [Marvin Wolf]

2.1 Architekturübersicht

Das Team Digitalize macht Aufnahmen der Puzzleteile. Diese werden von der Rohform (RAW-Format) in ein nutzbares Format gewandelt. Auf diesem wird dann die Segmentierung des Puzzleteils vom Hintergrund vorgenommen, d.h. das Teil wird ausgeschnitten. Zusätzlich wird es einheitlich skaliert, so dass die Puzzleteile ihrem Originalmaß entsprechen. Zuletzt wird das segmentierte Puzzleteil auf einen transparenten Hintergrund gelegt sowie ein SVG-File erzeugt, dass den Rand beschreibt. Diese Resultate werden von den Solvern für das Lösen des Puzzles genutzt sowie von den Visualizern für das Anzeigen der Puzzleteile.
Die Gruppe Solve extrahiert im Preprocessing wichtige Merkmale für das Lösen des Puzzles. Dabei werden die Kanten jedes Puzzleteiles mit jeder Kante aller anderen Puzzleteile verglichen und ein Wahrschneinlichkeitsmaß für deren Zusammenpassen gebildet. Die Methoden Edge Matching und XOR Matching vergleicht die Kantenform der Puzzleteile. Das Farbmatching vergleicht den Farbverlauf zweier Kanten. Die entstehenden Wahrscheinlichkeitsmatrizen dienen dem Solver Algorithmus als Eingabe, um das Puzzle zu lösen. Er gibt das gelöste Puzzle in Form einer 7x7 Matrix aus. Das Post-Processing wird benötigt, um die Rotation und Position der Puzzleteile zu berechnen, so dass die Teile ineinander passen und das Puzzle eine lückenlos gefüllte Fläche ergibt.

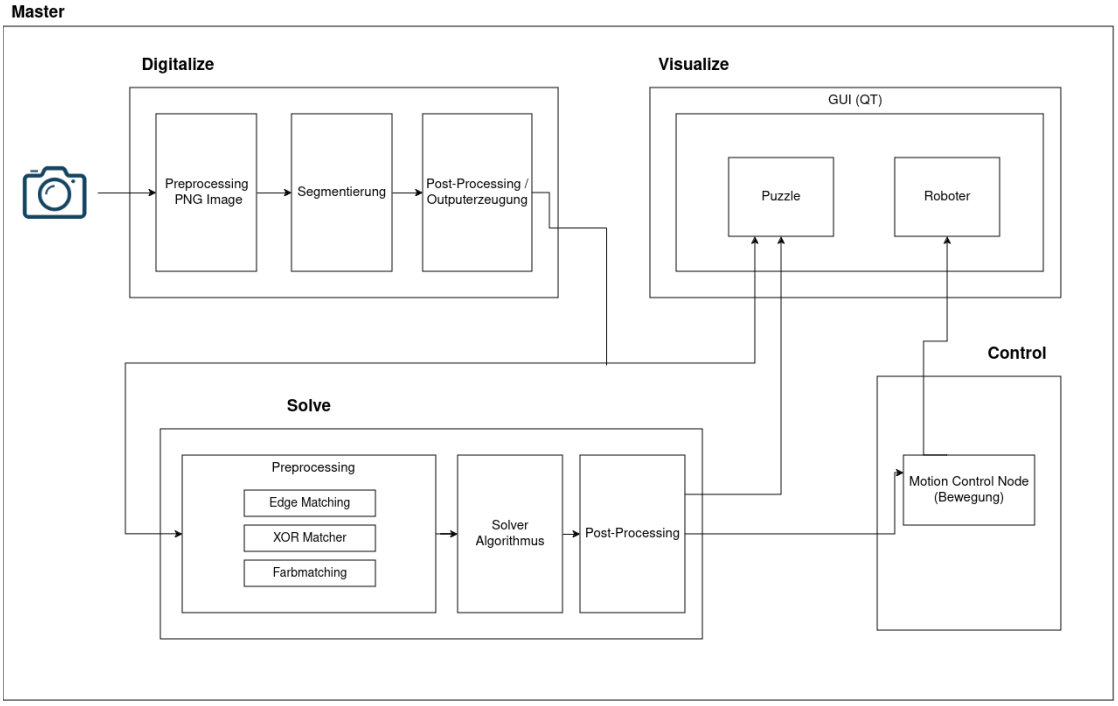


Abbildung 1: Architekturübersicht grob

Diese Informationen nutzt das Team Control, um die Bewegung des Roboterarms von einer fest definierten Startposition zu der jeweiligen Endposition zu berechnen. Diese Bewegung wird in Form einer Abfolge von Winkeln, die die Arme des Roboters zueinander haben und so dessen Anordnung beschreiben, an Visualize weitergegeben.
Die Gruppe Visualize liefert ein Graphical User Interface, in dem der Roboter und das Lösen des Puzzles animiert werden. Die Puzzleteile liegen auf einem Stapel und werden nach und nach vom Roboter gegriffen und an die korrekte Stelle gesetzt. Die Drehung des Puzzleteiles ist bereits zur Initlalisierung gegeben d.h. das Teil muss nur bewegt werden. Die Aufgabe ist gelöst, sobald alle Puzzleteile ausgebreitet sind und ein komplett zusammengesetztes Puzzle ergeben.

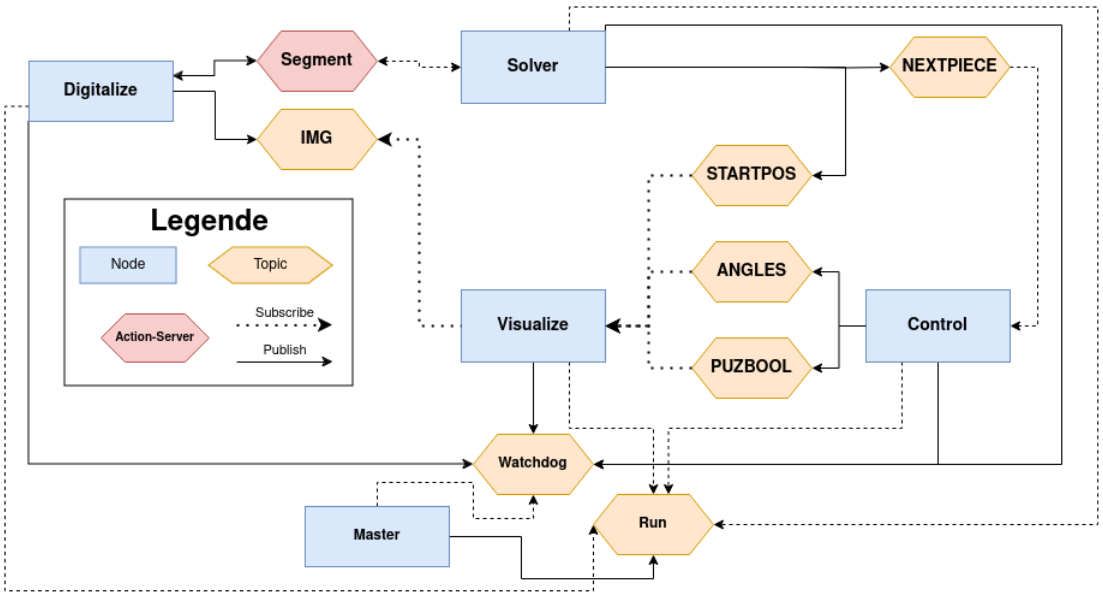


Abbildung 2: Interfaces - In späteren Kapiteln genauer beschrieben

2.2 Zeitlicher Verlauf

Der folgende Verlauf zeigt, wann im Projekt in etwa welche Aufgaben abgeschlossen wurden. Nachbesserungen, Anpassungen und Weiterentwicklungen sind weiterhin möglich.

Wochen 1-4 (21.3.-18.4.)

- Einteilung in Gruppen
- Einarbeitung in die jeweiligen Technologien
- Erstellung einer gemeinsamen Entwicklungsumgebung
- Aufnahmen der Puzzleteile anfertigen
- Einfache Roboterbewegung berechnen

Woche 5 (18.4.-25.4.)

- Entzerren der Aufnahmen
- Extraktion der Puzzleteilkanten

Wochen 6-7 (25.4.-9.5.)

- Visualisierung des Robotermodells
- Kantenvergleich als Wahrscheinlichkeitsmaß

- Erster Ansatz für Solveralgorithmus
- Segmentierung des Puzzleteils
- Berechnung Inverse Kinematik

Wochen 8-9 (9.5.-23.5.)

- Ground Truth Mock zum dynamischen Testen der Solving Algorithmen
- Postprocessing: Ausgaben der Solver Gruppe
- Interpolation Roboterbewegung

Wochen 10-11 (23.5.-6.6.)

- optimierter Solveralgorithmus "Best First + Global Best"
- Simulated Annealing
- Farbmatching der Puzzleteilkanten
- Grundsätzliche Visualisierung dreidimensionales Puzzleteil
- XOR Matcher der Puzzleteilkanten
- Entwicklung verschiedener Schnittstellen / ROS Nodes und Nachrichten

Wochen 12-14 (6.6.-20.6.)

- **Teilintegrationstests:**
  Zusammenfügen von verschiedenen Nodes und Test derer im Verbund
  Szenarien:
  - Control<->Visualize: Bewegungsanimation Roboter
  - Digitalize<->Solve: Visualisierung Puzzle
  - Digitalize<->Solve: Lösen des Puzzles
- Erstellung einer Master Node, die die einzelnen Komponenten zentral ansteuert

Wochen 15-17 (20.6.-.4.7.)

- Integration aller ROS Nodes in ein Gesamtprodukt
- Bugfixes
- Dokumentieren des Projekts

3 Projektdurchführung [Marvin Wolf]

3.1 Organisation

Das Projekt wurde von den Teilnehmern des Kurses weitestgehend eigenverantwortlich durchgeführt. Die anfänglichen Vorgaben - Lastenheft - müssen erfüllt werden. Die Umsetzung ist den Studenten selbst überlassen.
Auf die Ernennung eines Projektleiters folgte die Festlegung von Themengebieten und zugehörigen Aufgaben, die das Endprodukt definieren. Anhand dieser Gebiete wurden die Teams eingeteilt. Die Teams "Digitalize", "Solve", "Control" und "Visualize" entstanden sofort, während sich das Team "Docker" spontan aus der Notwendigkeit einer gemeinsamen Entwicklungsumgebung und vorbildlicher Eigeninititative herausbildete.
Mit den Teams wurden Arbeitspakete in Form von Epics gebildet, die die nächst anstehenden Schritte beschreiben. Diese waren auf eine Laufzeit von zwei bis vier Wochen ausgelegt. Die Terminierung wurde durch die beteiligten Studenten selbst festgelegt. Dadurch stieg das Interresse an einer rechtzeitigen Fertigstellung.
Die Projektmethodik ist auch den Gruppen überlassen. Wie in der Softwareentwicklung üblich folgten die Teams agilen Methoden wie Scrum. Jede Woche gab es zusätzlich ein Meeting, in dem alle Teams persönlich antraten um sich und den Stakeholder auf den aktuellen Stand brachten. So konnte der Auftraggeber nach dem Prinzip "Continuous Feedback" Einfluss nehmen und seine Erfahrung in dem Bereich einbringen. Die persönliche Kommunikation und Zusammenarbeit zwischen den Gruppen steigerte merkbar die Effizienz und war für den Erfolg des Projektes unabdingbar. Interne Kommunikation und Absprachen in den Gruppen über Discord haben allerdings gut funktioniert.

3.2 Projektleitende Tätigkeiten

3.2.1 Allgemein

Der Projektleiter agierte in enger Absprache mit dem Kunden. Er ist verantwortlich, den Fortschritt des Projektes zu überwachen und einzugreifen, falls der Zeitplan in Gefahr gerät. Außerdem sorgt er dafür, dass die Stakeholder in geeigneter Form den Fortschritt des Projektes verfolgen können.
Folgende Methoden wurden u.A. zur Umsetzung angewandt:

- Regelmäßige Besprechungen mit den Teams und Einholen vom aktuellen Stand, Workload und Problemen.
- Aufbereiten und Visualisieren dieser Informationen für Stakeholder.
- Risikoanalyse mit den Gruppen. Beurteilung und Einordnung dieser sowie Darstellung in Form einer Risikomatrix, um anfängliche Risiken und Befürchtungen der Studenten einzuschätzen und einzudämmen.
- Planen des Vorgehens:
  Anfänglich aufteilen der zu implementierenden Komponente in grobe TODOs und

Schätzen des Aufwandes über Zeitleiste. Danach genauere Terminierung über Epics mit Enddatum festlegen.
Es wurden mit den Teams neue Epics geplant, wenn die alten abgeschlossen waren. So konnte die Durchführung des Projekts agil und dynamisch stattfinden. Die aus den Epics resultierende Roadmap (Gitlab-Feature) kann als Gantt-Diagramm betrachtet werden und eignet sich hervorragend als Visualisierung für die Stakeholder.

- Umverteilung der "Mitarbeiter" dynamisch nach Workload und Bedarf.
- Festlegung von Terminen und Planung des Ablaufs.
- Definition von Teilintegrationen d.h. Szenarien, die einzelne Abläufe des Gesamtprojekts darstellen. Dadurch wurden relevante Schnittstellen zwischen mehreren Nodes getestet. Die Komplexität einzelne Komponenten aneindanderzufügen ist deutlich geringer als alle auf einmal zu einem Produkt zu integrieren.
- Festlegen von Standards um gemeinsame Arbeit mit 18 Personen zu ermöglichen. Z.B. bei der Planung der Projektdokumentation- und Präsentation.

3.2.2 Top-Down Bottom-Up

Aufgrund der Unerfahrenheit des Projektleiters ist eine Mischung der Managementtheorien "Top-Down" und "Bottom-Up" praktiziert worden. Die Initiative bei der Gestaltung und Ausführung neuer Projektabschnitte ging vom Projektleiter aus, welcher ausgearbeitete Vorschläge und Ideen initial einbringt. Diese wurden dann von den Studenten, die sich in ihren Teilgebieten besser auskennen und evtl. auch in organisatorischen Bereichen bereits Erfahrung gemacht haben, entweder allgemein akzeptiert oder verbessert und damit optimiert. Manchmal musste trotzdem gegen den Willen der Studenten entschieden werden, um die Komplexität nicht unnötig zu erhöhen oder damit einhergehende Risiken auszuschließen. Durch den Ansatz, "Top-Down" und "Bottom-Up" zu mischen, konnte das Projekt erst gelingen. So wurden Kompetenzen richtig ausgelagert und gleichzeitig Motivation geschaffen, indem jeder Stimme Beachtung geschenkt wurde und so gemeinsam das Projekt gelenkt werden konnte. Die Mischung der Strategien wird auch als "bipolare Strategie" bezeichnet. [1]
Rein der Ansatz "Top-Down" wurde nur beim Konziperen der Projektdokumentation verwendet. Hierbei wurde den Studenten vom Projektleiter vorgestellt, was eine Projektdokumentation ist, welchen Zweck dies erfüllt, wie sie aufgebaut ist und was für Fehler in Zusammenhang mit dieser auftreten können.
Es wurden alle möglichen Bestandteile einer Projektdokumentation aufgelistet:

1. Projektvorfeld (IST-Analyse)
2. Aufgabenstellung (SOLL-Konzept) / Anforderungen
3. Projektziele (SOLL-IST-VERGLEICH)
4. Zeitplanung / Meilensteine

5. Wirtschaftlichkeit
6. Projektverlauf
7. Ausblick / Folgeaktivitäten
8. Pflichtenheft
9. Angebot
10. Preiskalkulation / Nutzwertanalyse
11. Kundeneinweisung
12. Abnahme- und Testprotokoll
13. Rechnung

Nach dem Top-Down Ansatz im Sinne der Modellierung wurde so von oben mit den allgemeinen Punkten angefangen. Der Überbegriff Projektdokumentation wurde in seine Bestandteile heruntergebrochen. Diese werden nun weiter verfeinert. Zuerst werden alle für unser Projekt nicht relevanten Bestandteile entfernt. Z.B. entfällt die IST-Analyse, da bisher kein System bestand und von Null auf entwickelt werden muss. Außerdem handelt es sich um ein einmaliges wissenschaftliches Projekt, welches keinen wirtschaftlichen Zwecke folgt.
Damit entfällt bereits ein Großteil der Inhalte. Folgende bleiben übrig:

1. Aufgabenstellung (SOLL-Konzept) / Anforderungen
2. Zeitplanung / Meilensteine
3. Projektverlauf
4. Kundeneinweisung
5. Abnahme- und Testprotokoll

Nun wird geprüft welche Teile bereits (in einer anderen Form) vorhanden sind. Das SOLL-Konzept ist in Form von Anforderungen bereits im Vorhinein vom Auftraggeber definiert. Die Zeitplanung sowie der grobe Projektverlauf sind bereits als von Gitlab-Wiki Dokumentation oder Epics und der Gitlab Roadmap vorhanden. Diese Informationen wurden vom Projektleiter erfasst, aufbereitet und in die Dokumentation eingepflegt.
Die verbleibenden zwei Komponenten werden in anderer Form verwirklicht. Die Kundeneinweisung findet als Abschlusspräsentation statt während das Abnahme- und Testprotokoll bereits während des Projekts im Zuge von "Continuous Feedback" durchgeführt wurde.
Damit ist das Projekt an sich nach Definition bereits dokumentiert. Sinnvoll ist es aber bei einem Softwareentwicklungsprojekt auch diese zu dokumentieren. Eine Technische Dokumentation zur Übergabe an den Kunden muss verfasst werden, damit dieser das

Projekt weiterentwickeln oder warten kann.
Somit müssen von den Studenten nur noch der individuelle Projektverlauf sowie die technische Dokumentation niedergeschrieben werden.

- Was war das zu lösende Problem?
- Was habe ich gemacht?
- Welche Lösungen / Techniken habe ich angewendet?
- Genauere Beschreibung der Lösung / Implementation

Das genaue Verfassen in Latex wurde vom Projektleiter vorgeschrieben und nur vom Stakeholder beeinflusst, was auch wieder dem Top-Down Ansatz entspricht.

4 Digitalisierung

Zu Beginn steht die Herausforderung, ein physisch vorliegendes Puzzle zu digitalisieren. Eine digitale Version des Puzzles ist einerseits für die Lösungsalgorithmik notwendig, andererseits aber auch zur Visualisierung der Lösung durch einen simulierten Roboter. Diese Aufgabe wurde vom Team Digitalisierung gelöst.

4.1 Anfertigung der Rohaufnahmen [Cedric Holzer]

Vorliegend war ein physisches Puzzle mit 49 Puzzleteilen, von welchem ein möglichst detailgetreues, digitales Abbild erstellt werden sollte. Dieses soll das physische Original hinsichtlich Formen und Farben möglichst fehlerfrei abbilden, d.h. ohne Artefakte wie Schlagschatten an den Kanten, ohne Verzeichnungen und Verzerrungen und mit möglichst hoher Maßstabstreue.

Fotografie Die Puzzleteile wurden einzeln mit einer Digitalkamera vor einem vorbereiteten Hintergrund möglichst gerade von oben fotografiert. Dabei wurde ein Stativ verwendet, um die Kameraposition relativ zum Hintergrund konstant zu halten. Soweit möglich wurden alle Einstellungen der Kamera manuell auf feste Werte gesetzt, um keine künstlichen Abweichungen zwischen den Einzelbildern zu erzeugen. Dazu zählen Weißabgleich, ISO-Wert, Blendenzahl und Belichtungsdauer, aber auch Funktionen wie die automatische Belichtungsoptimierung, die deaktiviert werden müssen. Verwendet wurde eine Canon EOS RP Kamera mit einem Canon RF 35mm F1.8 Macro IS STM Objektiv.

Um Schlagschatten an den Rändern der Puzzleteile zu vermeiden, ist eine gleichmäßige Ausleuchtung elementar. Dazu wurde eine provisorische Ringleuchte verwendet, die aus einem Papierzylinder mit innen aufgeklebten LED-Streifen besteht. Dieser Papierzylinder wurde so aufgestellt, dass Puzzleteil und Objektiv sich im Inneren befinden und so von allen Seiten beleuchtet werden. Der gesamte Aufbau zur Anfertigung der Rohbilder ist in Abbildung 3 zu sehen. Gut zu erkennen sind die LED-Streifen der Ringleuchte und die Schachbretter der Hintergrundgrafik. Am oberen Ende ist die Stativhalterung der Kamera zu sehen.

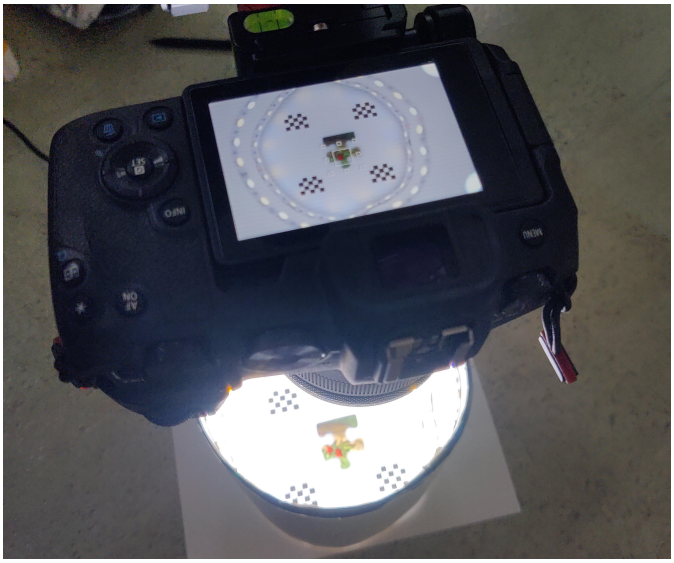


Abbildung 3: Setup zur Fotografie der Puzzleteile mit Stativ, Ringleuchte und Hintergrundgrafik

Hintergrund Die in Abbildung 4 zu sehende Hintergrundgrafik, vor der alle Puzzleteile fotografiert wurden, wurde mittels Inkscape erstellt, einer freien Software zur Erstellung und Bearbeitung von Vektorgrafiken. Die Hintergrundgrafik ist notwendig, um zusätzliche Informationen über die Aufnahmeperspektive maschinenlesbar in den Bildern zu kodieren und damit später die Perspektive korrigieren zu können. Der Hintergrund besteht aus einem weißen Blatt Papier mit vier schwarz-weißen Schachbrettmustern, deren äußere Ecken ein Quadrat von 80 mm Seitenlänge aufspannen. Die Schachbretter bestehen aus 4 Reihen mit je 5 quadratischen Feldern von 4 mm Kantenlänge. Die asymmetrische Form erlaubt die Definition einer eindeutigen Symmetrieachse. Die Fläche, die von den vier Mittelpunkten der Schachbretter aufgespannt wird, ist damit ein 60 mm breites und 64 mm hohes Rechteck mit einem Seitenverhältnis von 15:16.

Entwicklung Die Bilder wurden im Format CR3 aufgenommen, einem RAW-Format. Sie wurden zunächst in ein für die weitere Verarbeitung brauchbareres Format, nämlich JPG, konvertiert. Dafür wurde die Open-Source-Software Darktable eingesetzt. In diesem Schritt wurden die Bilder zugeschnitten, um die sichtbaren Bereiche der Ringleuchte zu entfernen, die nicht von Interesse sind. Zudem wurde die Verzeichnung der Bilder anhand der vom Hersteller bereitgestellten und in Darktable enthaltenen Profile für diese Kamera-Objektiv-Kombination korrigiert. Der Darktable-Stil, eine Art Protokoll der vorgenommenen Schritte, ist zur Reproduzierbarkeit dieses manuellen Vorgehens im digitalizer-Repository zu finden.

Abbildung 5 zeigt ein exemplarisches Foto eines Puzzleteils nach dem Entwicklungsschritt. Das Bild ist so zugeschnitten, dass Teile der Ringleuchte großteils entfernt wurden, die Schachbretter aber noch vollständig und mit etwas Rand sichtbar sind.

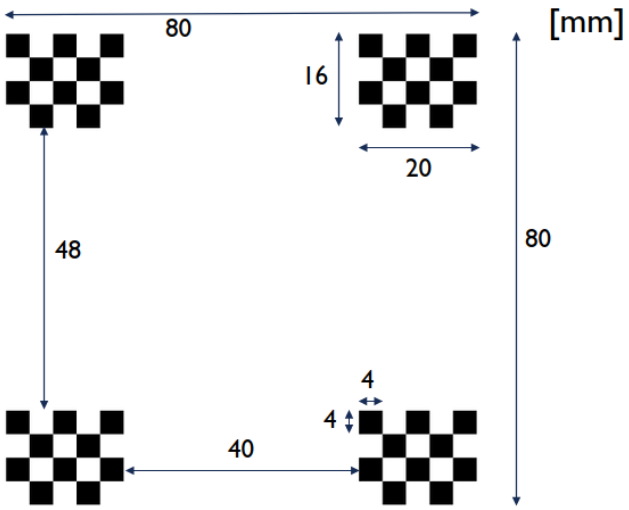


Abbildung 4: Hintergrundgrafik für Aufnahme der Puzzleteile

Abbildung 5: Beispiel eines Bilder nach dem Entwicklungsschritt

4.2 Perspektivische Korrektur [Cedric Holzer]

Im nächsten Schritt wird die Perspektive der Bilder korrigiert. Dadurch werden einerseits alle Bilder auf einen exakt identischen Maßstab skaliert und andererseits werden Verzerrungen beseitigt, die durch eine nicht parallele Ausrichtung von Sensor- und Motivebene entstehen können.

Für die Perspektivkorrektur ist es notwendig, vier Punkte in einem Bild zu erkennen, deren Lage im abgebildeten Raum bekannt ist. Für diesen Zweck bestehen die Schachbrettmuster im Hintergrundbild. Deren innere Kreuzungspunkte, d.h. Punkte, an denen sich vier benachbarte Felder des Schachbretts treffen, können mittels der Funktion `cv2.findChessboardCorners` erkannt werden, die in der Bildverarbeitungsbibliothek OpenCV enthalten ist. Sie findet ein Schachbrett definierter Größe in einem Bild. Damit dieser Schritt korrekt funktioniert, werden die Bilder zunächst in einem Vorbereitungsschritt leicht aufgehellt und stark kontrastverstärkt, sodass die Schachbretter fast nur aus reinen Schwarz/Weiß Pixeln bestehen.

Da ein Bild vier Schachbretter beinhaltet, OpenCV aber nur eines pro Bild detektiert, wird das Bild zunächst in vier Viertel geteilt, in welchen separat nach dem jeweils enthaltenen Schachbrett gesucht wird. Die gefundenen Koordinaten der Schachbretter müssen anschließend wieder in das Koordinatensystem des Ursprungsbildes zurückgerechnet werden.

Die Koordinaten der Kreuzungen jedes Schachbrettes werden gemittelt, sodass sich vier Mittelpunkte pro Bild ergeben. Da auf den perspektivisch noch nicht korrigierten Bildern gemittelt wird, verzerrt dieser Schritt die vier Mittelpunkte unterschiedlich. Sollte sich die Perspektive der Bilder stark unterscheiden, dürfte es vorzuziehen sein, nur den äußersten Punkt jedes Schachbretts zu verwenden ohne zu mitteln. Dieses Vorgehen wäre verzerrungsfrei, dafür aber anfälliger für ungenau erkannte Kreuzungskoordinaten. Durch die Verwendung eines Stativs ist die Perspektive in unserem Fall fast konstant und die Sensorebene annähernd parallel zur Motivebene. Damit ist nur eine leichte Verzerrung zu erwarten, die für alle Bilder annähernd identisch ist.

Die realen Abstände der ermittelten vier Mittelpunkte sind bekannt. Ein Beispiel für diesen Schritt ist in Abbildung 6 zu sehen. Die vier Mittelpunkte der Schachbretter spannen im unkorrigierten Bild ein Parallelogramm auf, das nun in ein 1500px x 1600px Rechteck transformiert wird. Dabei werden die ermittelten Mittelpunkte in die Ecken des Bildes verschoben. Das abgebildete Rechteck entspricht einer realen Fläche von 60mm x 64mm, was einen Maßstab von $1mm = 25px$ beziehungsweise $1px = 40\mu m$ ergibt.

Implementierung Der Darktable-Stil zur Entwicklung der Rohbilder sowie die SVG-Datei des Hintergrunds liegen im digitalizer-Repository im `/resources` Ordner. Zur perspektivischen Korrektur aller Bilder in einem Ordner kann im selben Repository das Python-Skript `src/rectification/rectification.py` verwendet werden.

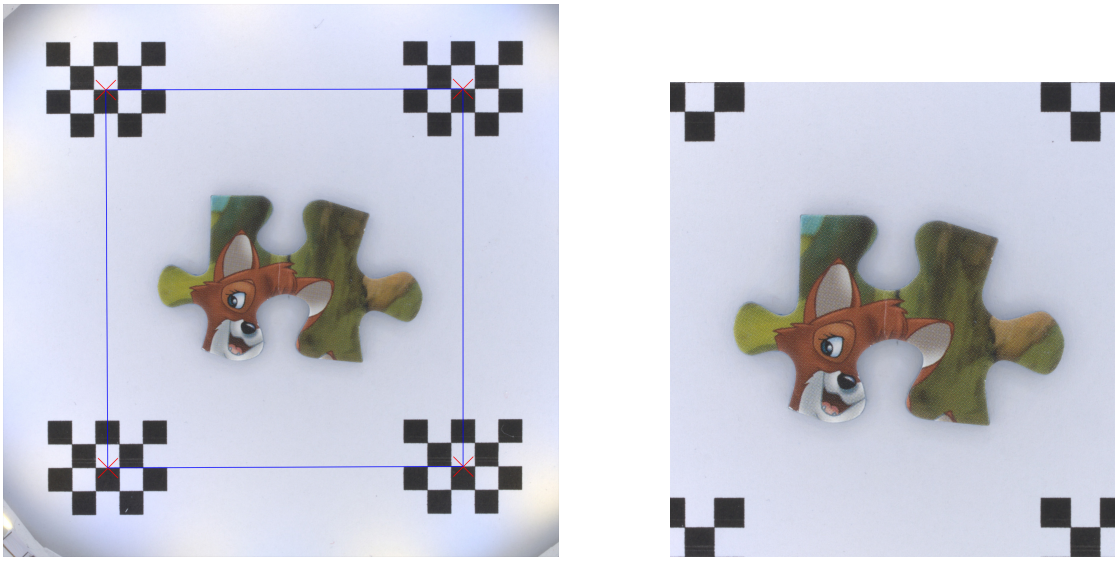


Abbildung 6: Erkannte Abbildungsfläche im unkorrigierten Bild (links) und Resultat der Perspektivkorrektur (rechts)

4.3 Segmentierung

Durch das Pre-Processing werden also aus den fotografierten Puzzleteilen, linear entzerrte und einheitlich zugeschnittene JPG Bilder generiert, wie in Abbildung 7 dargestellt. Diese dienen als Eingabe für den nächsten Schritt, die *Segmentierung*.

Bei der Segmentierung werden zusammenhängende Strukturen in einem Bild erkannt und die entsprechenden Pixel dem Vordergrund bzw. dem Hintergrund zugeordnet. Dabei entsteht verbunden binäre Bildregionen, welchen im Weiteren die Form eines Objektes bzw. in diesem Fall des Puzzleteils beschreiben [2]. Anhand der Form kann der Solver später zum Beispiel Kanten- und Color Matching vornehmen.

Aufgaben ist nun, diese Puzzleteile zu erkennen und an den Kanten auszuschneiden bzw. den Hintergrund zu entfernen. Dafür wurden zwei verschiedene Ansätze exploriert, wobei einer ausschließlich auf digitale Bildverarbeitungsmethoden zurückgreift, während die andere eine Künstliche-Intelligenz (KI) beinhaltet. Beide Ansätze benutzen dabei die gleichen Bilder als Basis. Diese werden im Folgenden beschrieben.

4.3.1 Segmentierung mit digitaler Bildverarbeitung **[Kipp]**

Die digitale Bildverarbeitung bietet einen großen Werkzeugkasten an bekannten und bewährten Algorithmen für Bildmanipulation und Transformation. Die nachfolgend dargestellte Segmentierungsmethode stützt sich ausschließlich auf diese Werkzeuge und kommt dabei ganz ohne künstliche Intelligenz aus. Das hat den Vorteil, dass diese selbst auf schwacher Hardware zielführend genutzt werden kann. Die genutzten Algorithmen etwa zur Konvertierung der Farbräume, der Faltung von Bildern mit einem Kernel oder das Finden von Kanten sind schon lang bekannte und daher gut optimiert, dokumentier-

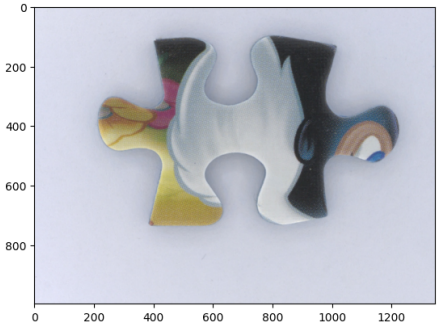


Abbildung 7: Linear entzerrtes und zugeschnittenes Puzzleteil

te und in diversen Quellen hinreichend nachvollziehbar erklärt. Da es sich um konkrete Anleitungen handeln, wie die Pixel in einem Bild verarbeitet werden müssen, ist kein vorheriges Trainieren bzw. Lernen eines Models notwendig, die Abarbeitung der statischen Instruktionen ist ausreichend. Daher ist der Output deterministisch, da stets bei gleicher Eingabe, die gleiche Ausgabe zu erwarten ist.

Segmentierungsansätze In der Bildverarbeitung gibt es nicht die *einen* Methode, um Objekte von einem Hintergrund zu segmentieren. Die Wahl des Vorgehens hängt von der Aufgabe ab und wie gut eine Methode auf dem gegebenen Problem performt. Grob lassen sich Segmentierungsansätze in zwei Kategorien unterteilen. Diese lassen sich selbst wiederum unterteilen [3].

Diese Klasse an *Similarity-Based* Verfahren segmentiert Regionen basierend auf der Ähnlichkeit seiner Pixel. Darunter fallen alle *Thresholding* und *Region Growing Techniques* wie der *Watershed* Algorithmus.

Thresholding Methoden teilen die Pixel in einem Bild nach ihrem Helligkeitswert anhand eines Schwellwertes in Vorder- und Hintergrund ein. Sie funktionieren gut, wenn das Objekt heller ist als der Hintergrund.

Für das Puzzleprojekt wurde in einer frühen Phase ein adaptiver Threshold für die Segmentierung verwendet. Bei dem adaptiven Threshold wird kein globaler Schwellwert, welcher sehr anfällig für Schatten ist, sondern mehrere regionale Schwellwerte verwendet, welche robuster gegen Lichteinfluss sind. Die Eingabe Bilder sind gut, aber nicht perfekt ausgeleuchtet, so zeigt das Ergebnis ein grundsätzlich richtig segmentiertes Puzzleteil, jedoch mit noch zu vielen Schatten an den Rändern wie in Abbildung 11.

Das Watersehd Verfahren interpretiert die Pixel und ihren Gradienten der Helligkeit in einem monochromen Bild als Höhenkarte. Dabei stellen lokale Minima Täler dar, welche mit Wasser geflutet werden. Der Wasserspiegel steigt und trifft auf andere Täler bzw. Wasseransammlungen, welche dann vereint werden. Um Regionen separat voneinander zu halten, werden Dämme als Grenzen eingesetzt. Diese Dämme werden mittels Dilatation ermittelt. Je höher der Gradient eines Pixels, desto eher handelt es sich um einen Damm.

Auch diese Methode wurde für das Puzzleprojekt evaluiert und hat aber mit den gewählten Parametern und der gewählten Umsetzung keine Puzzleteile segmentiert, sondern

die Objekte auf den Puzzleteilen unterteilt, wie in Abbildung 11 zu sehen. Die Unterteilung war also zu fein und hätte gröber sein müssen. Es ist anzumerken, dass das keine allgemein gültige Aussauge ist und das Problem hätte mit mehr Recherche und anderen Parametern mit Sicherheit behoben werden können, was eine bessere Segmentierung erlaubt hätte. In Anbetracht der kurzen Projektdauer und der Tatsache, dass andere Projektkomponenten auf den segmentierten Bildern aufbauen, ist ein anderes, einfacheres, Segmentierungsverfahren gewählt worden.

Diese Klasse an *Discontinuity-Based* Verfahren basiert auf der Annahme, dass Regionen und Segmente in einem Bild durch abrupte Änderungen der Helligkeitswerte bestimmt werden können. Diese werden meist auf *monochromen*, also einfarbigen Bildern, angewendet. Die Kantenerkennung spielt dabei eine entscheidende Rolle, da Kanten als signifikante Übergänge in der Helligkeit definiert werden können. Methoden zur Kantenerkennung betrachten den Gradienten, also die Ableitung der Helligkeitsänderung in x und y Richtung. Die dadurch gewonnen Stärke und Richtung der Kante hilft bei der Identifizierung von relevanten Kanten und Konturen des Puzzleteils. Solche Techniken nutzt zum Beispiel der **Canny Edge Detector**, welcher im weiteren Verlauf beschrieben, angewendet und evaluiert wird.

Segmentierung mit Canny Edge Detector Um schnell Prototypen entwickeln und testen zu können und weil alle vorher genannten Methoden als Bibliotheken verfügbar sind, ist Python als Programmiersprache für die Umsetzung der Puzzleteilsegmentierung gewählt worden. Für die eigentliche Bildverarbeitung werden die Packages `openCV` [4] und `Pillow` [5] verwendet.

Da die Image Processing Segmentation Klasse (`ip_segmentation.py`) später neben einer KI Segmentation Klasse (`sam_segmentation.py`) in einen ROS2 [6] Node Eingebungen werden soll, wurde die Segmentierungsfunktion gekapselt und stellt eine einzelne, public Funktion `segment(input_image_path: str) -> (list, list)` bereit, welche nur den Pfad zu den Eingabebildern benötigt und nach Abarbeitung eine Liste an segmentierten Bildern und eine Liste an Kanten zurückgibt. Die `segment(...)` Funktion liest die 49 JPG Bilder ein und wendet die nachfolgend beschriebenen Verarbeitungsschritte zur Ausschneidung des Hintergrunds an.

Zu Beginn wird das Bild als Numpy-Array mithilfe der `Image.open()` Funktion von Pillow als `RGBA` geöffnet. Der vorher nicht vorhandene vierte Alpha-Kanal wird mit voller Deckkraft hinzugefügt. Das Numpy Array hat die Shape `(998, 1347, 4)`, das heißt ein Bild wird durch vier Matrizen, drei Farbkanäle und ein Alphakanal, mit den Dimensionen 988×1347 dargestellt.

Die auf Kantenerkennung basierende Segmentierung funktioniert am besten auf einfarbigen Bildern. Für die Farbraumkonvertierung von RGB zu Graustufen, wird die openCV Funktion `cvtColor()` verwendet. Intern wird dabei für jedes Pixel aus den drei Farbkanälen gewichtet eine Helligkeit ermittelt. Die Gewichtung der Farben ist orientiert an der Farbwahrnehmung des menschlichen Auges. Die zurückgegebene Matrix enthält die gleichen Dimensionen aber nur noch einen Kanal, siehe Abbildung 8.

Als nächstes wird der Canny Edge Detector auf das Graustufenbild angewendet. Auch

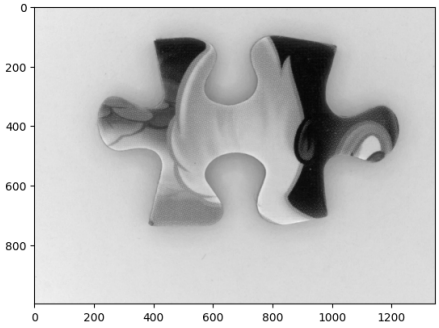


Abbildung 8: Graustufen des Puzzelteils

dafür stellt openCV eine Funktion `canny()` bereit. Der Canny Detector beseht aus mehreren Schritten und hebt Kanten, also abrupte Helligkeitsänderungen hervor. Zuerst wird mit einem Gausfilter Rauschen auf dem Eingabebild unterdrückt, indem jedes Pixel durch eine Gaußsche Verteilung gewichtet gemittelt wird. Das reduzierte Rauschens hilt, falsche Kanten zu unterdrücken. Danach wird ein Sobel Filter horizontal und vertikal über das Bild geschoben um die erste Ableitung zu berechnen. Die Gradientrichtung und -Stärke können dann aus den Pixeln der Ausgabebildern berechnet werden. Im dritten Schritt werden nicht-Maximalen Werte unterdrückt. Hierzu werden entlang der Gradientenrichtung nach lokalen Maxima gesucht, welche behalten werden. Das führt dazu, dass breite Kanten auf einen Pixel Breite verjüngt werden. Zum Schluss wird ein Hystersis Thresholding, also ein doppelter Schwellwert, angewendet um schwache Kanten und Rauschen rauszufiltern. Eine Kante unterhalb des unteren Schwellwerts oder zwischen dem oberen und unteren Schwellwert werden nicht weiter berücksichtigt. Nur starke Kanten, auch wenn sie nicht durchgehend über dem oberen Schwellwert sind, werden behalten [7]. Die min und max Schwellwerte müssen der Canny-Funktion übergeben werden. Durch experimentelle Annäherung konnten die Werte 20 und 110 ermittelt werden. Die Ausgabe des Cannys ist also eine Binärbild, wie in Abbildung 9, welches ausschließlich starke Kanten beinhaltet. Zu beachten ist, dass sowohl die Konturen des Puzzleteils als auch jegliche Kanten des Motivs darstellen, enthalten sind. Die tatsächliche Kantenextraktion des Puzzelteils erfolgt in einem späteren Schritt.

Da der Canny Edge Detector nur starke Kanten in Form eines Binärbildes übrig lässt, kann es sein das die Kante, welche die Kontur des Puzzelteils darstellt, um wenige Pixel unterbrochen ist. Würde jetzt eine Kantenextraktion durchgeführt werden, könnte die gesuchte Kontur nicht gefunden werden. Aus diesem Grund wird das Binärbild vorher noch Weichgezeichnet. Bei dem Blur Filter in diesem Fall handelt sich um einen einfachen Durchschnitt, das heißt jeder Pixel nimmt den gemittelt Wert seiner unmittelbaren Umgebung an. Dadurch wird erreicht, dass im Historgram wieder Graustufen zwischen 0 und 255 zu sehen sind und kleiner Lücken in den Kanten geschlossen werden.

Die `findContours()` Funktion findet zusammenhängende Konturen in Bildern. Konturen können als Grenzen einer Form mit selben Helligkeitswert bezeichnet werden. Intern wird eine Form des *Border-Follow-Algorithm* verwendet. Dieser wandelt zuerst das Bild

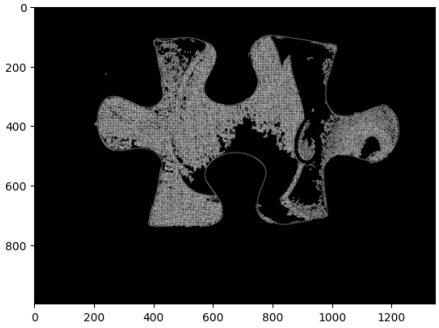


Abbildung 9: Verbesserte Kanten mit Canny Edge Detector

wieder mit einem Threshold wieder in ein Binärbild um, um dannach zeilenweise Pixel für Pixel nach eine vordergrund Pixel zu suchen. Wenn eins gefunden wurde, wird dieses als Startpunkt markiert und die Nachbarschaft nach ebenfalls vordergrund Pixeln durchsucht. Dieser Prozess wird solang wiederholt, bis der Startpunkt wieder erreicht ist. Die Ausgabe ist eine Liste an geschlossenen Kanten. Die gesuchte Kontur vom Puzzelteil ist die Kontur welche den größten Flächeninhalt einspannt. Also wird mit einer weiteren OpenCV Funktion `contourArea()` nach der größten Fläche gesucht. Mit der Gaußschen Trapezformel lässt sich der Flächeninhalt von beliebigen Polygonen bestimmen. Da nur die eine Kontur für das Projekt relevant ist, werden die anderen gefunden Konturen nicht weiter beachtet. Die Aufgetragen Kontur auf das Originalbild ist in Abbildung 10 zu sehen.

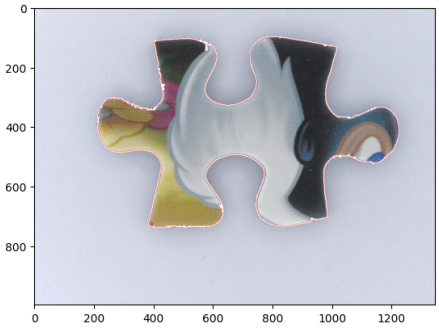


Abbildung 10: Nicht geglätette Kontur

Ein finaler Median-Blur glättet die noch teilweise unsauberen Kanten, wie in Abbildung 11 zu sehen.

Fazit Zusammenfassend kann festgestellt werden, dass die aufgeführte Segmentierungsmethode basierend auf den Canny Edge Detector für jedes 49 Puzzleteile zuverlässig funktioniert. Bei genauer Betrachtung fällt allerdings auf, dass immernoch einge Fransen vorhanden sind und Schattige stellen falsch Segmentiert werden. Im Vergleich mit der

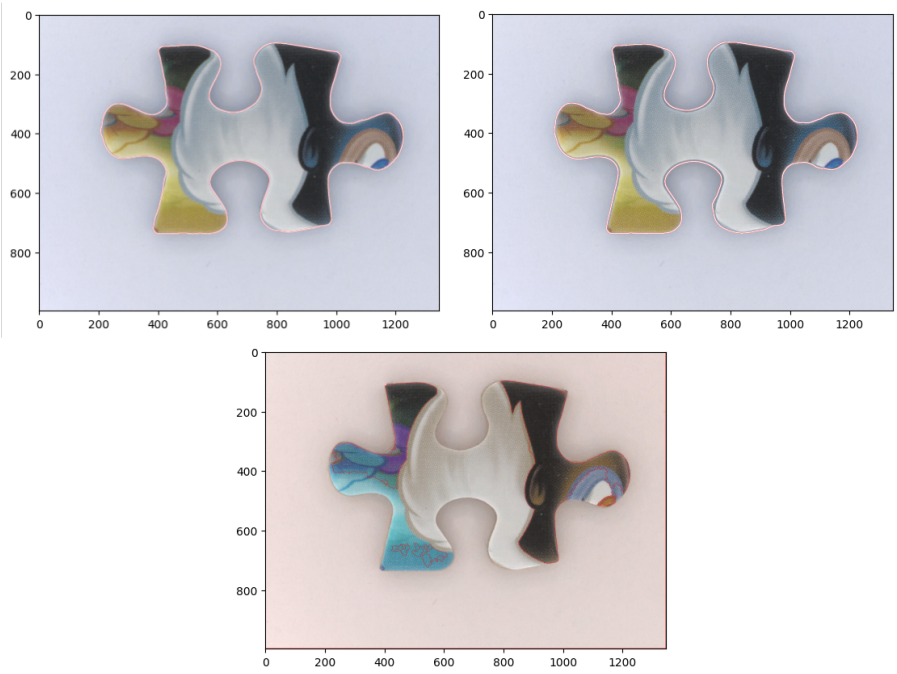


Abbildung 11: Segmentierung - Canny (links), Threshold (rechts), Watershed (unten)

Threshold Methode werden dennoch deutlich mehr Schatten herausgefiltert. Die Watershed Methode hat durch die falsche Parameterwahl auch schlechter Ergebnisse geliefert. Für eine bessere Segmentierung sollte das Watershed Verfahren nocheinmal beleuchtet werden.

Trotz der Störungen ist es dem Solver aber möglich, das Puzzle mit den ausgabebildern zu lösen.

Noch bessere Ergebnisse erziehlt der nachfolgende Segmentierungsansatz mit KI.

4.3.2 Segment Anything Model (SAM) [Then]

Segment Anything Model (SAM) ist die KI-Alternative, die zum Segmentieren von Puzzleteilen verwendet wird. SAM ist ein neues KI-Modell von Meta AI, das jedes beliebige Objekt in jedem Bild äusschneiden"kann, wenn es die gewünschte Eingabe erhält. SAM ist ein Zero-Shot Segmentierungssystem, das sich ohne zusätzliches Training auf unbekannte Objekte und Bilder generalisieren lässt. Mit über 1 Milliarde Masken auf 11 Mio. lizenzierten und die Privatsphäre respektierenden Bildern ist SAM so konzipiert und trainiert, dass es prompt auf neue Bildverteilungen und Aufgaben übertragen werden kann [8]. Zero-Shot ist ein maschinelles Lernszenario, bei dem ein KI-Modell darauf trainiert wird, Objekte zu erkennen und zu kategorisieren, ohne sie zuvor gesehen zu haben, bzw. ohne dass das Modell zuvor auf die Objekte trainiert wurde [9]. SAM wurde als KI-Modell wegen seiner erstaunlichen Zero-Shot-Fähigkeiten ausgewählt, da das Modell auf einem riesigen Datensatz trainiert wurde. Daraus ergibt sich die Frage, ob SAM-Vorhersagen

ein akzeptables Segmentierungsergebnis liefern können, ohne dass das Modell für die Segmentierung von Puzzleteilen trainiert werden muss.

Um die Frage zu beantworten, muss ein angepasster Prozess eingerichtet werden, der den gewünschten Output liefert. Es muss ein Automatisierungsprozess eingerichtet werden. Es gibt zwei Methoden zur Integration von SAM in den Automatisierungsprozess: die lokale Installation der erforderlichen Pakete oder die Durchführung über Cloud-Computing-Dienste wie Google Colab. Für dieses Projekt wurde die letztere Methode gewählt und ein Jupyter-Notebook entwickelt, um SAM in die Lösung zu integrieren. Colab nutzt eine Jupyter-Notebook-Umgebung, die auch `Python` als Kernel verwendet. SAM kann nun heruntergeladen und abgeleitet werden.

Abbildung 12: Bounding Box Region

SAM bietet eine Reihe von Segmentierungsfunktionen, aber die in diesem Projekt verwendete ist der `SamPredictor`. Dies ermöglicht die Verwendung der Funktion `predict`, die vor der Inferenz bestimmte Eingabeparameter erfordert. Zu Beginn muss SAM mit dem trainierten Modell initialisiert werden. Die entscheidenden Parameter sind dann das Eingabeformat zur Aufforderung einer Auswahlregion und die Option einer Multimask-Ausgabe. Im Rahmen der Segmentierung wurde ein Eingabeformat verwendet, das einer Bounding Box folgt, welche einem NumPy-Array entspricht. Dieses Format wurde eingesetzt, um die Region des Bildes zu bestimmen, in der die Segmentierung durchgeführt werden soll. Dadurch wurde das Modell angewiesen, den in diesem Bereich erzielten Ergebnissen mehr Gewicht zu verleihen. Alternativ hätten auch die Pixelpunkte auf dem Bild verwendet werden können, um die gewünschte Segmentierung zu markieren. Diese Vorgehensweise wurde jedoch nicht gewählt, da das Modell dazu tendiert, die feineren Merkmale auf den Puzzleteilen zu segmentieren, wie beispielsweise die auf dem Puzzleteil vorhandenen Bilder.

Die Option MMultimask outputïst optional und standardmäßig aktiviert. Mit diesem Parameter kann SAM drei Masken ausgeben, zusammen mit der Einschätzung der Qualität dieser Masken durch das Modell. Diese Einstellung ist für mehrdeutige Aufforderungen gedacht, was hier nicht der Fall war. Es hat sich jedoch als vorteilhaft erwiesen, sie aktiviert zu lassen, da sie dem Modell mehr Spielraum (drei Masken) für genauere Vorhersagen lässt. Sobald der Inferenzprozess abgeschlossen ist, werden die segmentierten Masken zusammen mit ihren jeweiligen Punktzahlen angezeigt. Abbildung 13 ver-

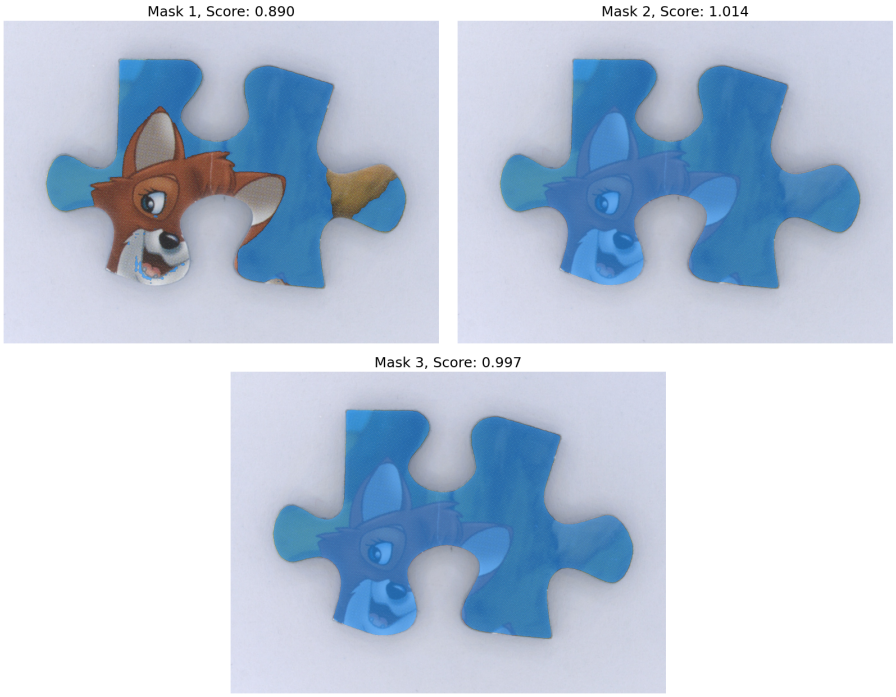


Abbildung 13: Ergebnisse der Multimask-Output-Bewertung in der Reihenfolge von Maske 1 (links), Maske 2 (rechts) und Maske 3 (unten)

anschaulicht die Ergebnisse, die durch einen einzigen Aufruf des `predictor()` Funktion erzielt werden. Die erhaltenen Segmentierungsmasken ermöglichen nun die Extraktion der optimalen Maske sowie den Beginn der Post-Processing-Phase.

In der Zusammenfassung lässt sich festhalten, dass SAM im Vergleich zur digitalen Bildverarbeitung ein qualitativ besseres Segmentierungsergebnis liefert. Es ist jedoch zu beachten, dass dieser Ansatz mit einem erheblichen Rechenaufwand verbunden ist. Dies wurde bei der anschließenden Portierung dieser Methode auf ROS noch deutlicher. Das Checkpoint-Modell war nicht für den mobilen Einsatz optimiert, was zu einer längeren Download-Zeit führte. Als nächste Herausforderung stellte sich das Laden des Modells in einen Docker-Container dar, um die notwendigen Inferenzen durchzuführen. Dies führte zum Auftreten von Speicherfehlern, die aufgrund der zeitlichen Beschränkungen des Projekts nicht behoben werden konnten. Ein alternativer Ansatz bestand darin, das Modell durch einen kleineren Modell-Checkpoint zu ersetzen. Die Qualität der Segmentierungsmaske, die durch diesen kleineren Checkpoint erzielt würde, konnte jedoch nicht evaluiert werden und wird daher lediglich als potenzielle Alternative oder als ein möglicher Ausblick auf die Zukunft zur Optimierung dieser Methode bezeichnet.

4.4 Postprocessing [Then]

Nachdem mit den oben genannten Methoden eine Segmentierungsmaske des gewünschten Objekts erstellt wurde, müssen die Ergebnisse nachbearbeitet werden, bevor sie den anderen Teams zur Verfügung gestellt werden können. Team Solver benötigt ein Bild des Puzzlestücks ohne Hintergrund (transparent), während Team Visualiser die Konturen der Puzzlestücke in einem reduzierten Format benötigt. In diesem Abschnitt wird beschrieben, wie ein Bild des Puzzlestücks mit transparentem Hintergrund (RGBA) und eine Maske der Konturen eines Puzzlestücks mit approximierten Kanten erstellt wird.

4.4.1 Puzzleteil ohne Hintergrund (RGBA)

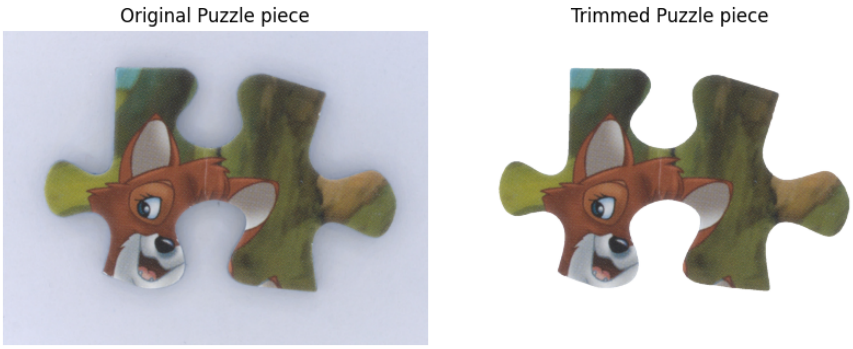


Abbildung 14: Vor (links) und nach (rechts) dem Zuschneiden mit der Segmentierungsmaske

Die Segmentierungsmaske liefert die Pixelpositionen in Form einer binären Maske des zu segmentierenden Bereichs. Um den Hintergrund zu entfernen, muss alles außer dem Puzzlestück identifiziert und entfernt werden. Zu diesem Zweck wird das Bild zunächst mit Hilfe der Bibliothek `openCV` in vier Kanäle (RGBA) umgewandelt. Dann wird unter Verwendung der Pixelposition der erhaltenen Maske eine einfache Bedingung festgelegt, um den Alphakanal jeder Position außer der der Maske auf Null zu ändern. Das Ergebnis ist ein Bild, das nur aus dem Puzzleteil besteht. Das neu erhaltene Bild kann nun im PNG-Format gespeichert werden, das üblicherweise zum Speichern von Bildern mit vier Kanälen verwendet wird.

4.4.2 Kanten Approximieren

In ähnlicher Weise können die Konturen anhand der Position der Maske ermittelt werden. Die Konturen können mit der Funktion `find_contours()` der Bibliothek `scikit-image` [10] gemessen werden, die die Maske als Eingabeparameter benötigt. Sie gibt die Kontur in Form von Pixelpositionen aus, die gegen x- und y-Koordinaten ausgetauscht werden müssen, damit sie in das Scalable Vector Graphics (SVG)-Format konvertiert werden können. Schließlich können die Pixelpositionen der Kontur in die SVG-Zeichnung eingefügt

werden, so dass eine Datei entsteht, die nur die Konturen des Puzzlestücks enthält. Bei der Überprüfung der Ausgabe wurde jedoch ein wichtiges Problem festgestellt: Die Konturen waren zu detailliert. Wie bereits erwähnt, wurden die Pixelpositionen der Konturen direkt in die SVG-Datei eingefügt, was zu einer sehr detaillierten Kontur des Puzzlestücks führte. Die Einbeziehung so vieler Informationen würde den Rechenaufwand für die anschließende Visualisierung des Puzzleteils erheblich erhöhen, was nicht wünschenswert ist.

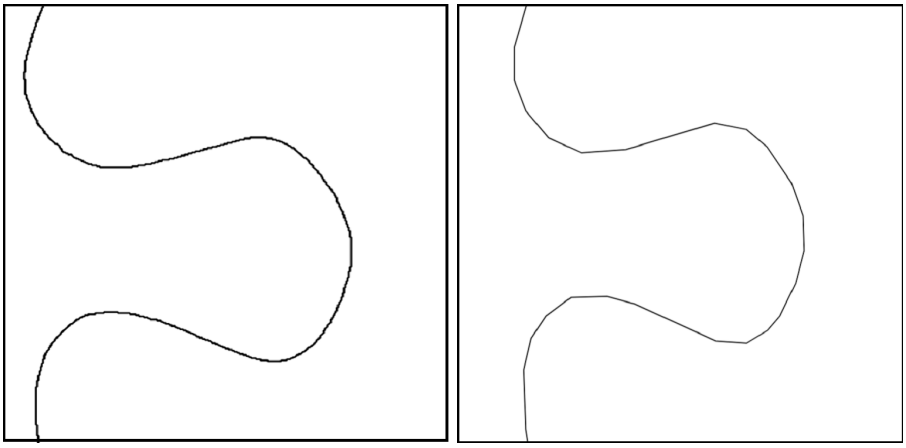


Abbildung 15: Vergleich von vor (links) und nach (rechts) der Approximierung

Um dieses Problem zu lösen, müssen die Kanten einer Form der Merkmalsreduktion unterzogen werden. Der Ramer-Douglas-Peucker-Algorithmus (RDP) [11] wurde eingesetzt, um die Anzahl der Punkte in einer durch eine Reihe von Punkten approximierten Kurve zu reduzieren. Abbildung 15 zeigt die Ergebnisse bei der Verringerung der Anzahl der Punkte, die erforderlich sind, um den Großteil der Krümmung an den Kanten der Konturen zu erhalten. Mit den neu erhaltenen approximierten Kanten kann diese schließlich in der endgültigen SVG-Datei gespeichert werden.

4.5 ROS-Node **[Kipp]**

Das Projekt schreibt vor, dass die Teilkomponenten Digitalizer, Solver, Controller und Visulizer über das Roboter Operation System, kurz ROS, kommunizieren. Das hat unter anderem den Vorteil, dass der ROS-Node, welcher die Segmentierung mit KI durchführt, auf einer sehr leistungsstarken Hardware laufen kann, während andere Nodes auf sparsamer Hardware oder verteilt laufen können. Durch ROS ist also das Übertragungsprotokoll definiert, die Interfaces zwischen den Nodes müssen unter enger Abstimmung selbst definiert und eingehalten werden. Der Digitalize-Node hat eine Schnittstelle zu dem Solver- und dem Visualisierungs-Node. Die Solver fragen den Digitalizer nach segmentierten Puzzleteile, um das Puzzle zu lösen und die Visualizer wollen die segmentierten Puzzleteile anzeigen, sobald es vom Roboter bewegt wird.

4.5.1 Kommunikation

Es muss sichergestellt werden, dass der Solver alle Puzzleteile empfängt oder zumindest überprüfen kann, ob Bilder vollständig sind. ROS2 beinhaltet *Action Server*, welche von einem Client mit einem Request bzw. Goal aufgerufen werden können. Der Action Server entscheidet dann, ob das Goal akzeptiert wird und sendet ein Acknowledgement an den Client. Daraufhin kann der Action Server das Goal ausführen und während dessen Feedback an den Client schicken. Sobald das Goal erreicht wurde, wird eine Response verschickt. Dieses Actionprotokoll kann gut auf den Segmentierungsvorgang übertragen werden. So sendet der Solver eine `digitalize/segment` Action an den Digitializer-Node mit den Request, Puzzleteile zu segmentieren. Der Digitalizer kann daraufhin mit der Segmentierung beginnen und sobald Puzzelteile fertig sind, diese als Feedback an den Solver verschicken. Ist die Übertragung beendet, wird eine Response versendet mit der der Solver überprüfen kann, ob alle Bilder korrekt empfangen wurden und daraufhin das Puzzle lösen. In Sequenzdiagramm von Abbildung 16 ist die Action oben zu sehen.

Die Visualizer bekommen von Control Bewegungskommandos und dazugehörige Puzzelteil IDs. Sie brauchen also ebenfalls alle Puzzelteile, können sich jedoch nicht auf die Feedback-Topic der Segmentierungs-Action subscriben. Für das Just-In-Time Mapping von neuen IDs zu Puzzleteilen wäre ein REST-API Endpunkt ähnlicher Get Request über einen ROS-Service denkbar gewesen, der Einfachheit halber wurde aber eine Topic `digitalize/imgs` eingerichtet, auf welcher nach erfolgreicher Segmentierung alle Puzzleteile publiziert werden. Das hat den Vorteil, dass der Visualize-Node nur auf die Topic subscriben muss und sofort alle Bilder im Speicher hat. Ein entsprechendes Publish-Verhalten ist im Sequenzdiagramm 16 zu sehen.

4.5.2 Interfaces

Für den Solver ist es wichtig, das Puzzleteil als PNG zu bekommen, sowie eine ID als Integer. ROS stellt mit Message `sensor_msgs/Image` bereits eine Möglichkeit bereit, wie Bilder verschickt werden könne. Das Segmentierungs-Goal muss nicht weiter definiert werden, da klar ist, was zu tun ist. Ob alle Puzzleteile empfangen worden sind, kann überprüft werden, indem die Response die Anzahl der versendeten Bilder beinhaltet. Demgemäß ergibt sich folgende Schnittstelle von Digitalize zu Solver für die Segment-Action:

```
                    # Request
---
int32 png_total_count # Response
---
sensor_msgs/Image png # Feedback
int32 id # Feedback
```

Die Kommunikation zwischen Visualize und Digitalize basiert auf einfachen Topics, die jederzeit versendet werden können, eine einfach ROS Message ist also ausreichend. Zusätzlich zu dem Puzzle PNG und der ID, benötigen der Visualize-Node noch die Konturen des Puzzleteils, um dieses Triangulieren zu können. Die Konturen werden als Liste

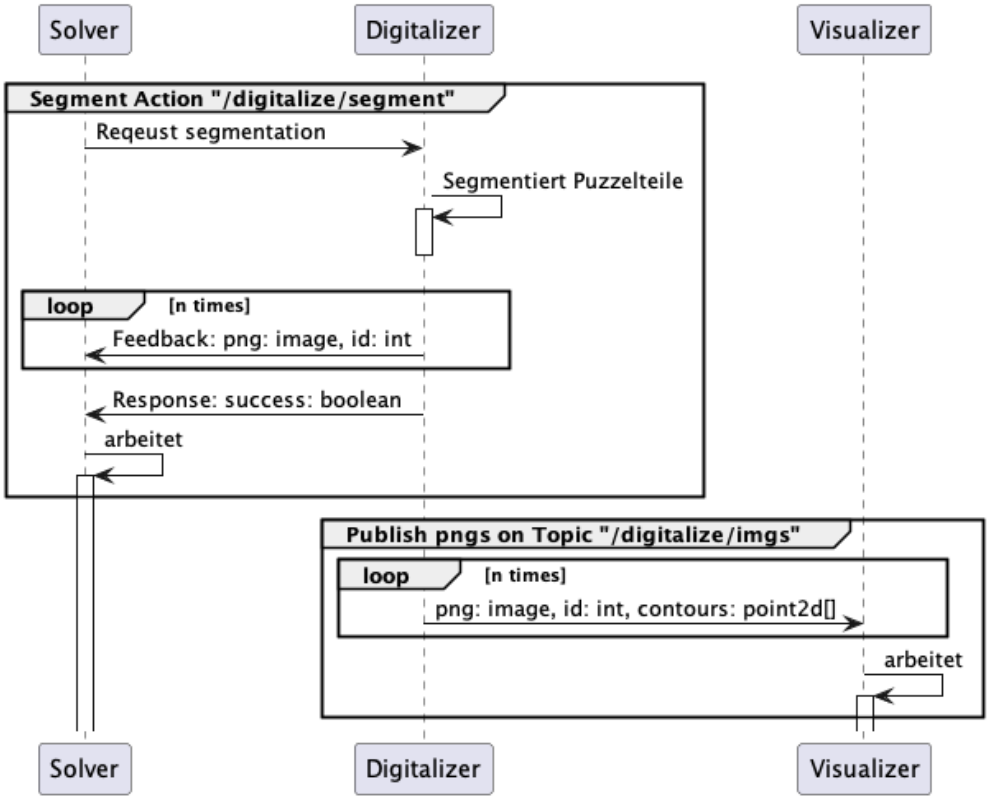


Abbildung 16: Sequenzdiagramm Kommunikation

von (x, y) Koordinaten versendet. Daraus ergibt sich folgende Schnittstelle von Digitalize zu Visualize für den Image-Publisher:

```
int32 id
sensor_msgs/Image png
Point2d[] points
```

Wobei `Point2d` wie folgt definiert ist:

```
int32 x
int32 y
```

4.5.3 Architektur

Der Digitalize ROS2 Node ist mit hilfe der `rclpy`, ein ROS Package, ebenfalls in Python geschrieben. Die Architektur kann dem Diagramm 17 entnommen werden. Der `manager` wird ist der Entry-Point in den node, wird also mit `ros2 run pgk_digitalize manager` gestartet. Dieser wiederum registriert den Segment Action Server und den Image Publisher. Segmentiert wird entweder mit der mit reiner Bildverarbeitung oder mit KI.

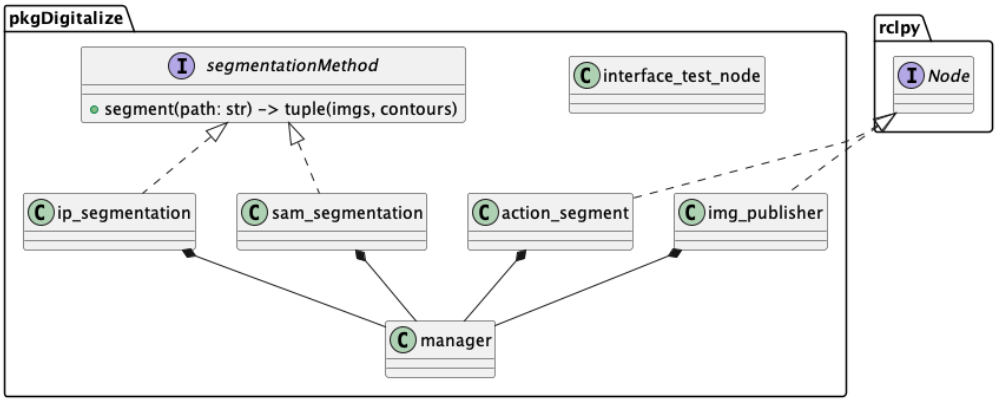


Abbildung 17: Digitalize ROS Node Architektur

5 Solver

5.1 Einleitung [Jonas Leitner]

Als Solver standen wir vor dem Auftrag, ein in Form von Bildern digitalisiertes Puzzle so zu lösen, dass es anschließend vom Roboter richtig zusammengesetzt werden kann. Dabei sahen wir uns mit einer Reihe von Problemen konfrontiert. Jedes Bild repräsentiert genau ein Puzzleteil. Zum einen muss das Bild so verarbeitet werden, dass die jeweiligen Kanten (Edges) der Puzzleteile extrahiert werden können. In einem weiteren Schritt ist es notwendig, ein Vergleichsmaß zu erhalten, mit dem zwei Kanten auf Ähnlichkeit verglichen werden können. Diese zwei Schritte haben wir auch als Preprocessing definiert. Mit diesen Informationen kann dann ein Algorithmus entwickelt werden, der die richtige Anordnung der Puzzleteile in einer Matrix löst. Dabei ist zu beachten, dass die Puzzleteile in Schritten von 90 Grad gedreht werden können. So weiß man nach dem Lösungsalgorithmus die genaue Anordnung der Puzzleteile und welche Kante zu welcher gehört.

Das Puzzle ist damit aber noch nicht zusammengesetzt. Hierfür ist ein Postprocessing-Schritt notwendig, in dem die genaue Position berechnet wird, damit sich die Teile korrekt überlappen. Eine Übersicht des Prozesses ist auch in Abbildung 18 visualisiert.

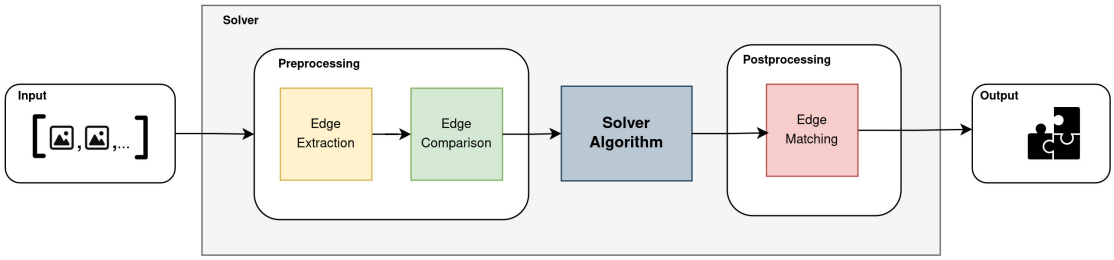


Abbildung 18: Übersicht des Solver Prozesses

Klassenstruktur Es wurde eine Klassenstruktur konzipiert um die Handhabung zu erleichtern und Abstraktion der einzelnen Problematiken zu erreichen. Ein Überblick erleichtert im Folgenden die Verständlichkeit der einzelnen Abschnitte.

Den Kern bilden die Klassen *Puzzle*, *PuzzlePiece* und *Edge*. Ein *PuzzlePiece* setzt sich stets aus exakt vier Edges zusammen. Jede *Edge* ist durch eine eindeutige ID, einen Pfad, welcher die Kante beschreibt, sowie zwei Ecken, an denen die Kante anschließt, charakterisiert. Zusätzlich wird jeder *Edge* ein Type zugewiesen. Eine Kante kann entweder als „FLAT", „MALE" oder „FEMALE" definiert werden.

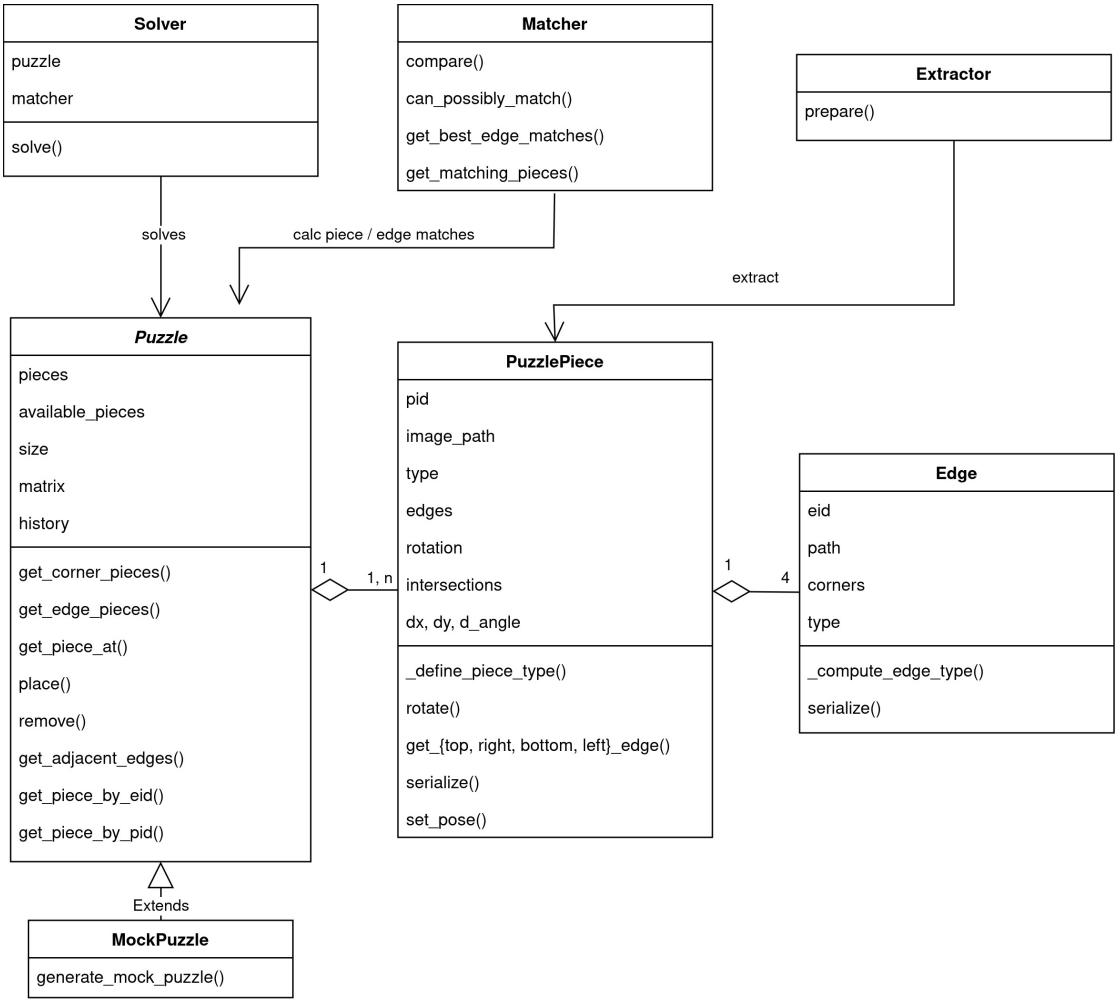


Abbildung 19: Vereinfachtes Klassendiagram

Das *PuzzlePiece* verfügt ebenfalls über eine ID sowie einen Type. Der Type wird über die Types der dazugehörigen Edges bestimmt. Sind zwei der Kanten „FLAT", dann handelt es sich um eine „CORNER". Ist eine der Kanten „FLAT", handelt es sich um ein „EDGE" Piece. Ansonsten ist der Type „DEFAULT". Jedes *PuzzlePiece* verfügt über einen Pfad zum dazugehörigen Bild des Puzzleteils. Das Attribut „edges" beinhaltet eine

Liste mit vier Kanten, welche im Uhrzeigersinn angeordnet sind. Dabei stellt die erste Kante die obere dar. Ein Puzzleteil kann um 90, 180 und 270 Grad rotiert werden. Dabei rotiert die Position der Edges natürlich auch mit. Zusätzlich verfügt das *PuzzlePiece* über Intersections, welche die Schnittpunkte der Hough Transformation halten. Diese werden im Postprocessing benötigt, um die exakte x-, y-Position und genaue Gradverdrehung des Puzzleteils zu berechnen.

Der sogenannte *Extractor* hat die Aufgabe, eine Liste von Eingabebildern in eine Liste von definierten Puzzleteilen zu verwandeln. Der genaue Ablauf wird in Unterabschnitt 5.2 beschrieben. Da der Extraktionsschritt eine gewisse Zeit in Anspruch nehmen kann, verfügen *PuzzlePiece* und *Edge* über eine *serialize*()-Funktion, mithilfe derer die Liste an Puzzleteilen in Form eines JSON gespeichert werden kann. Somit ist es nicht erforderlich, den Extrahierungsschritt bei jedem Durchlauf auszuführen.

Die Liste der Puzzleteile kann anschließend als *Puzzle* erweitert werden. Das *Puzzle* speichert eine Liste aller Puzzleteile im *pieces*-Attribut. Mithilfe dieser Liste kann auch die Größe des Puzzles berechnet werden. Das Puzzle hält eine Matrix, in der die einzelnen Puzzleteile platziert werden. Platzierte Puzzleteile werden aus der *available_pieces*-Liste entfernt. Das Puzzle hat zudem einige Hilfsfunktionen. Unter anderem gibt es Funktionen die es ermöglichen alle Corner-Pieces oder Edge-Pieces zu erhalten. Ein *Piece* kann mit *place*() an einer bestimmten Stelle innerhalb der Matrix platziert oder mit *remove*() auch wieder entfernt werden. Des Weiteren besteht die Möglichkeit, mit *get_adjacent_edges*() alle anliegenden Edges für eine spezifische Position innerhalb der Matrix zu erhalten.

Der *Matcher* ermöglicht die Erstellung einer Match-Matrix, welche sämtliche Kanten eines Puzzles beinhaltet. Beim Erstellen werden alle MALE und FEMALE Kanten miteinander verglichen. Der Matcher ist eine Basisklasse. Es existieren unterschiedliche Implementierungen des Matchers, wobei sich die *Matcher* in der *compare*()-Funktion hinsichtlich der Berechnung des Ähnlichkeitswertes von zwei Edges unterscheiden. In Unterabschnitt 5.3 werden die einzelnen Matcher ausführlich betrachtet. Zudem beinhaltet der Base-Matcher Hilfunktionen um beispielsweise für eine bestimmte *Edge* eine sortierte Liste der am Besten dazu passenden *Edges* zu erhalten.

Die Aufgabe des *Solver* ist es dann, ein bestimmtes Puzzle mit einem ausgewählten Matcher zu lösen. Auch hier gibt es verschiedene Implementierungen des Solvers, die in Unterabschnitt 5.4 näher beschrieben sind.

Das *MockPuzzle* bietet die Möglichkeit, ein Puzzle zu mocken, um die implementierten Solver besser testen zu können. Dies wird in Unterunterabschnitt 5.4.6 näher erläutert.

5.2 Extractor **[Simon Hampp]**

5.2.1 Motivation

Zu lösen war das Extrahieren der Kanten aus den PNG der Puzzleteile. Die Puzzleteile sind alle einzeln abfotografiert, schon segmentiert und optisch korrigiert.

Die Annahme ist also, dass der Alphakanal Vorder- und Hintergrund voneinander trennt und kein zusätzliches Preprocessing des PNG mehr nötig ist.

Um das Puzzle zu lösen, braucht man jedoch mehr als nur die optische Darstellung. Das

Ziel ist es, anhand des Bildes alle vier Kanten zu extrahieren, und für jede Kante einen eindeutigen Pfad zu extrahieren, sowie den Typ der Kante. Wir unterscheiden zwischen $female, male, flat$ Kanten.

5.2.2 Vorgehen

Zuerst wird die gesamte Kante des Puzzleteils extrahiert und dann die Kante an den vier Ecken des Puzzles geteilt. Im letzten Schritt wird der Verlauf der jeweiligen Kante analysiert, um herauszufinden, um welchen Typ es sich handelt.

Der erste Schritt ist trivial, da der Alpha-Kanal des PNG schon die Kante repräsentiert. Die erste Pixelreihe, die nicht mehr transparent ist, ist die Kante des Puzzles.

Der nächste Schritt, die Ecken zu finden, ist das schwierige. Hierfür habe ich recherchiert, welche bestehenden Methoden existieren und insbesondere welche Methoden OpenCV bereitstellt, um es in Python zu implementieren. Ich habe mehrere Methoden hierfür ausprobiert, wie bspw. Hough Transformation, goodFeaturesToTrack und Harris Corner Detection. Hierbei gab es jedoch das Problem, dass die Methoden einzeln entweder nicht die richtigen vier Ecken zuverlässig gefunden haben oder zu unpräzise waren. Vor allem mit der Hough Transformation konnten nicht alle Ecken finden, da diese Methode oft nur lokal in spezifischen Regionen der Kante Geraden angelegt hat. Nur goodFeaturesToTrack() von OpenCV konnte zuverlässig die vier Ecken der Puzzleteile finden, jedoch waren diese zu unpräzise. Die gefundenen Ecken liegen nicht direkt auf der Kante, sondern etwas näher im Inneren, wie auch in Abbildung 22 zu erkennen.

5.2.3 Lösung

Kanten extrahieren Um Ecken zuverlässig und präzise zu erkennen, wurde eine Kombination aus goodFeaturesToTrack und Hough Transformation gewählt. Mit goodFeaturesToTrack werden die Regionen der vier Ecken gefunden, und in diesen Regionen die Hough Transformation lokal angewendet. Bei der Hough Transformation ist zusätzlich noch zu beachten, dass eine Fallbacklogik eingebaut ist; sollten die beiden besten Geraden nicht plausibel sein, werden immer mehr Gerade gesucht, bis eine Kombination aus zwei plausiblen Geraden gefunden wird. Plausible Geraden sind definiert als zwei Geraden, die einen Winkel aufspannen, der größer als 45 Grad und kleiner als 135 Grad. Mit dieser Einschränkung findet der Extractor für alle 49 Puzzleteile in unserem Beispiel Puzzle die richtigen Ecken.

In Algorithmus 1 ist nochmal der schematische Ablauf der Kanten-Extraktion abgebildet.

Wie die Extrahierung in der Praxis aussieht, ist an Abbildung 20, Abbildung 21, Abbildung 22 zu erkennen. In Abbildung 20 ist der Input der Hough Transformation abgebildet, im Algorithmus 1 entspricht dies der $corner_region$. In Abbildung 21 sieht man die Hough Geraden ($line_1, line_2$) geplottet auf dem Kantenbild ($edge$) des Puzzles. Abbildung 22, ist dasselbe wie Abbildung 21 nur an die untere linke Ecke rangezoomt. So kann man den Schnittpunkt ($intersection$) und die Ecke, die goodFeaturesToTrack erkannt hat ($corner$), besser erkennen. Der grüne Punkt im Bild ist die endgültige Ecke,

Algorithm 1 Kantenextraktion aus einem PNG-Bild

Eingabe: PNG-Bild image
Ausgabe: Liste von Kanten edges
edge ← GetEdgeUsingAlphaChannell(png)
corners ← GoodFeaturesToTrackOpenCV(edge)
final_corners ← []
for corner ∈ corners **do**
    corner_region ← GetCornerRegion(edge, corner)
    $line_1$ and $line_2$ ← GenerateLinesWithHoughTransformation(corner_region)
    intersection ← CalculateIntersection($line_1$, $line_2$)
    nearest_point ← GetNearestPointOnEdge(edge, intersection)
    Füge nearest_point zu final_corners hinzu
end for
edges ← SplitEdgeAtCorners(final_corners, edge)
return edges

die den nächsten Punkt (*nearest_point*) auf der Kante zu dem Schnittpunkt darstellt, gemessen anhand der Euklidischen Distanz.

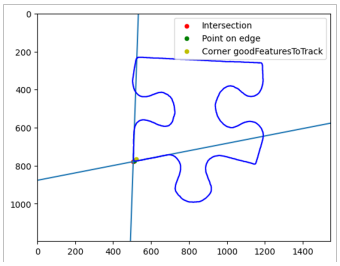


Abbildung 20: Input für Hough Transformation

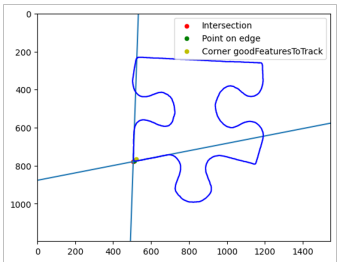


Abbildung 21: Output Hough Transformation

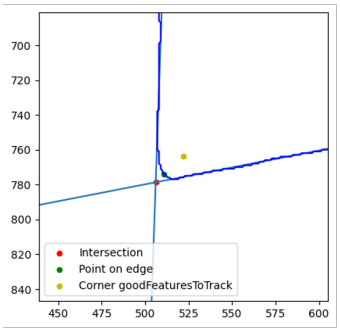


Abbildung 22: Zoom an Ecke

Kantentyp Anhand der gefundenen Kanten und Ecken muss im letzten Schritt noch der Typ ($male, female, flat$) der Kante erfasst werden. Im Code hat die Kantenklasse einen Pfad, der einem numpy Array entspricht, der die 2D-Koordinaten der Kante speichert. Der Kantentyp muss anhand dieses Pfads und der zwei Ecken, die die Kante aufspannt, erkannt werden. Hierfür wurde ein CCW Test implementiert. Bei der Implementierung der Kanten-Extraktion wurden alle Kanten und deren Ecken im Uhrzeigersinn (CW) definiert und so auch in den Datentypen abgespeichert. Diese Orientierung kann nun genutzt werden. Ein Vektor wird aufgespannt von der ersten bis zur nächsten Ecke. Dann ein weiterer Vektor von der ersten Ecke bis zu dem Punkt auf dem Kantenpfad, der am weitesten vom Vektor der Ecken entfernt ist. Anschließend wird geprüft, ob der Vektor

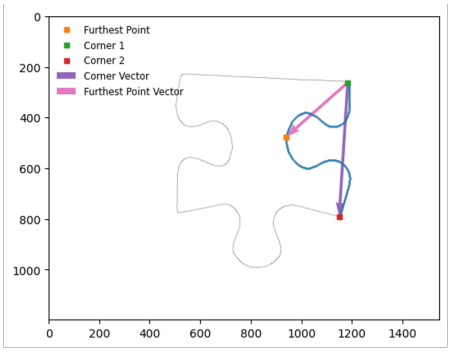


Abbildung 23: Kantentyp: $female$

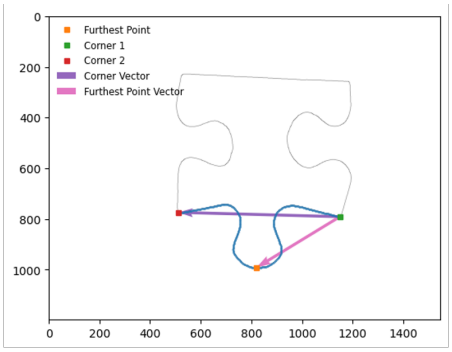


Abbildung 24: Kantentyp: $male$

zum weitesten Punkt links oder rechts vom Ecken-Vektor ist. Ist er links, ist es eine Kante vom Typ $male$. Ist der Vektor recht, ist die Kante vom Typ $female$. Zuletzt wird noch überprüft, wie weit der Punkt vom Vektor entfernt ist; ist die Entfernung sehr klein, wird der Typ auf $flat$ gesetzt. Den Threshold, ab wann eine Kante $flat$ ist, ist empirisch ermittelt worden. Übliche Werte der maximalen Distanz eines Kantenpunkts zum Ecken Vektor für nicht $flat$ sind in der Größenordnung 10^2, wohingegen Werte für $flat$ in der Größenordnung 10^0 liegt. Deshalb lassen sich diese Typen sehr gut voneinander unterscheiden.

5.3 Matcher

5.3.1 Geomatcher **[Cedric Holzer]**

Wir wollen anhand der Formen der Kanten beurteilen, wie gut zwei konkret rotierte Puzzleteile zusammenpassen. Dazu werden die beiden zu vergleichenden Kanten zunächst transformiert, um sie hinsichtlich Position und Orientierung vergleichbar zu machen. Die Größe darf nicht verändert werden, da die Bilder maßstabsgetreu sind. Jede Kante ist durch eine zusammenhängende Punktwolke variabler Größe gegeben. Jede Kante verfügt über zwei Eckpunkte, die beim Einlesen der Kanten generiert werden. Diese werden aus Sicht des Puzzleteil-Zentrums, also nach außen blickend, als linke und rechte Ecke bezeichnet.

Vorverarbeitung Aus Effizienzgründen werden alle Kanten vorverarbeitet. Pro Kante werden zwei Punktserien berechnet, die für alle Kanten im gleichen Koordinatensystem liegen. Es sind zwei Punktserien notwendig, da die zu vergleichenden Puzzleteile nebeneinander platziert werden müssen. Die transformierten Punktserien "liegen" auf der x-Achse, d.h. ein Puzzleteil liegt darunter und eines darüber. Jedes Puzzleteil wird für beide Lagen vorbereitet.

Zunächst werden die Punkte der Kante rotiert, sodass ein Eckpunkt im Ursprung liegt. Für die unter der x-Achse liegende Position wird hier der linke Eckpunkt gewählt, für die darüber liegende Position der rechte. Anschließend werden die Punktwolken um den

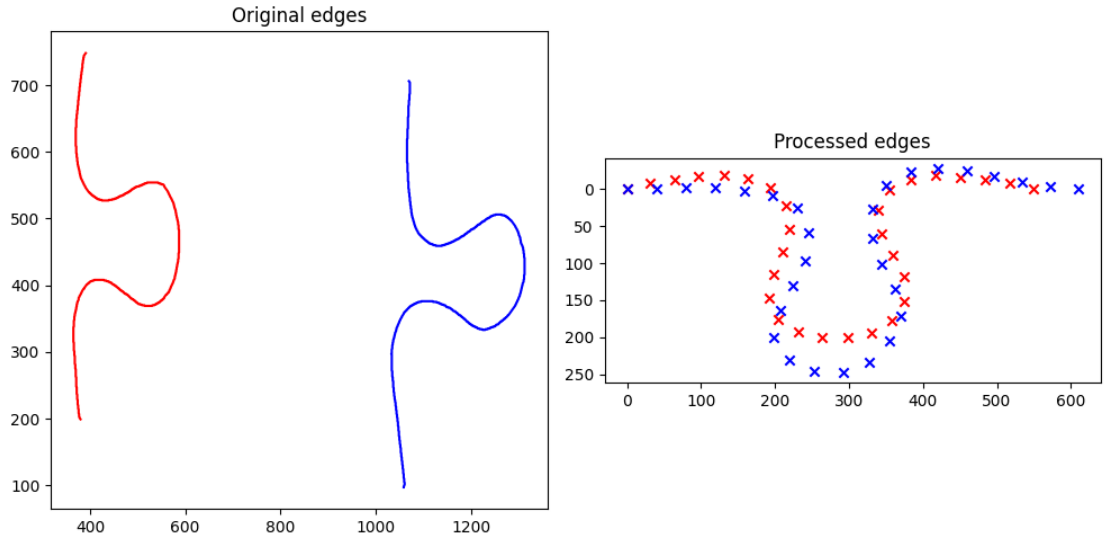


Abbildung 25: GeoMatcher Kanten-Vorverarbeitung: Ursprüngliche Kanten im lokalen Bildkoordinatensystem (links) und transformierte Punktwolken (rechts, N=30), die paarweise verglichen werden können.

Ursprung rotiert, sodass der verbleibende Eckpunkt auf der positiven x-Achse zum Liegen kommt.

Anschließend werden die Punktwolken, die unterschiedlich viele Punkte beinhalten können, durch eine konstante Anzahl von Punkten approximiert. Dazu wird zunächst die kumulative Kurvenlänge der Kante als dritte Dimension neben (x, y) eingeführt. Nun werden zwei Interpolationsfunktionen berechnet, die von der kumulativen Kantenlänge auf x und y abbilden. Abbildungen $x \to y$ oder $y \to x$ sind nicht eindeutig, da die Kante eine nicht-funktionelle Kurve ist. Auf der kumulativen Kurvenlänge können n gleichmäßig verteilte Punkte berechnet und über die vorbereiteten Interpolationsfunktionen auf (x, y) Koordinaten auf der Kurve abgebildet werden. Diese Punkte haben jetzt voneinander einen konstanten Abstand, sind also gleichmäßig über die Kurve verteilt. Abbildung 25 zeigt zwei unterschiedliche Kanten, die jeweils in eigenen lokalen Koordinatensystemen ihres Puzzleteils liegen. Die vorverarbeiteten Versionen liegen einheitlich orientiert auf der positiven x-Achse und wurden durch 30 gleichverteilte Punkte approximiert.

Vergleich Um zwei Kanten zu vergleichen, werden für beide Kanten die vorverarbeiteten Punktwolken abgefragt. Abbildung 26 veranschaulicht die beiden unterschiedlichen Versionen einer Kante, die durch die Vorverarbeitung entstanden sind. Beim Vergleich ist zu beachten, dass für eine der Kanten die unter und für die andere Kante die über der x-Achse liegende Version gewählt werden muss, damit die beiden Teile zusammenpassen können. Nun können die beiden Punktwolken gegenläufig iteriert und die einzelnen Punkte paarweise verglichen werden. Dadurch lässt sich der Vergleich zweier Kanten auf n Vergleiche zweier Punkte abbilden. Dieses Framework wird später auch für den

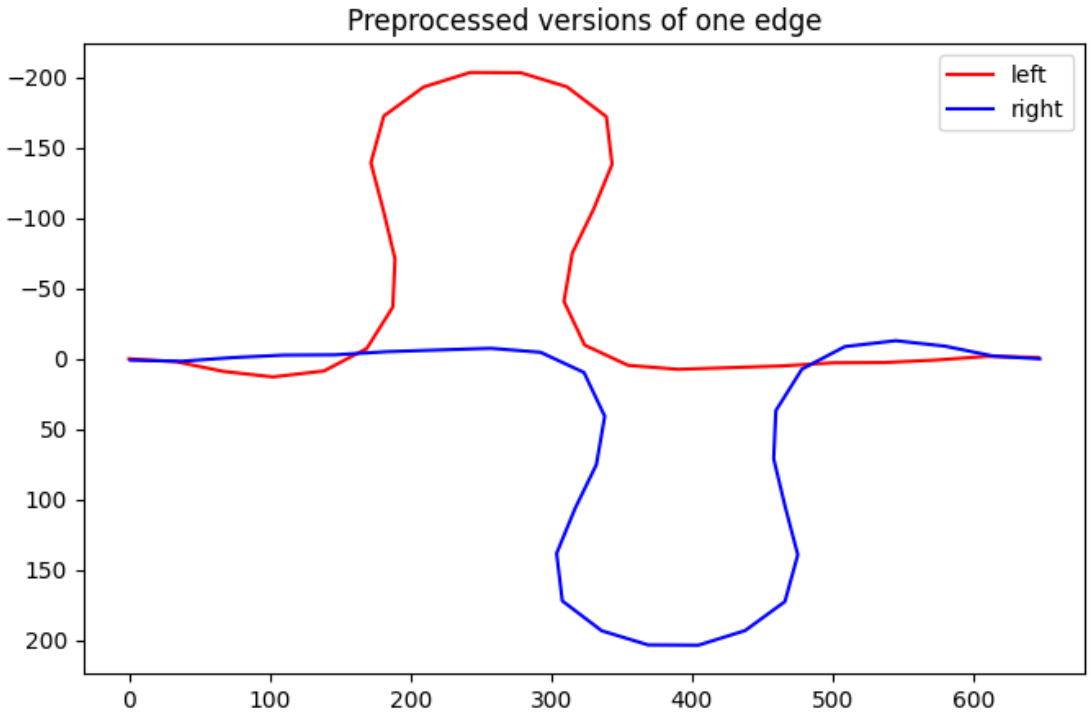


Abbildung 26: Beide vorverarbeiteten Punktwolken (interpoliert, N=30) einer einzelnen Kante im GeoMatcher

GeoMatcherRGB genutzt.

Sind die beiden Kanten nicht gleich lang, hängt ihre relative Ausrichtung zueinander und damit der resultierende Match-Score davon ab, wie die Puzzleteile den beiden Positionen (über und unter der x-Achse) zugewiesen werden. Das ist unerwünscht, da diese Zuweisung nicht sinnvoll deterministisch vorgenommen werden kann. Die Lösung besteht darin, beide möglichen Ausrichtungen zu berechnen und die resultierenden Scores zu mitteln.

Im Fall des GeoMatchers werden die einzelnen Punkte anhand ihres euklidischen Abstandes verglichen. Die Abstände aller Punktpaare werden für die Kante aufsummiert.

Globale Skalierung der Scores Die Fehlerwerte der einzelnen Kanten liegen in $[0, \infty[$, wobei 0 einen perfekten Match darstellt. Gewünscht ist aber eine Darstellung durch Scores in $[0, 1]$, wobei 1 einen perfekten Match darstellt. Daher berechnet der GeoMatcher zu Beginn alle möglichen Matches zwischen zwei Kanten und skaliert diese mittels $x' = 1 - (x \ / \ max_x)$, wobei max_x der höchste im Datensatz vorkommende Match-Score und x der unskalierte Match-Score ist.

Implementierung Der GeoMatcher ist im Package `algorithm.src.matcher.geo` im solver-Repository implementiert. Er nutzt intern ein eigenes Datenmodell, um eine Ent-

kopplung von dem volatilen solver-weiten Datenmodell zu erreichen und dieses frei von Matcher-spezifischen Funktionen zu halten.

5.3.2 Geomatcher RGB [Simon Hampp]

Motivation In den Farben der Puzzlekanten steckt viel Information, was uns beim Lösen des Puzzles helfen kann. Da unser zu lösendes Puzzle viele Kanten hat, die von der Form her ähnlich aussehen, ist der Farbvergleich wichtig, um diese voneinander zu differenzieren. Die Schwierigkeiten bestand darin, eine effiziente Methode zu finden, die trotzdem alle Farbwerte (RGB) miteinander vergleicht.

Vorgehen Die Basis für die Methode zum Vergleichen der Farbe ist der schon implementierte *GeoMatcher* Unterunterabschnitt 5.3.1. Wir hatten die Idee, diesen Matcher zu erweitern, um statt (x, y) Koordinaten der Pfade, die Farbwerte (r, g, b) der Kante zu vergleichen. Der Vorteil hierbei ist, dass die Struktur und viele benötigten Funktionen schon implementiert waren. Lediglich die Methode, um eine Kante zu laden, um sie mit einer anderen zu vergleichen, musste neu implementiert werden. Hier wird dann statt einer Liste von Koordinaten, eine Liste von Farbwerten zurückgegeben.

Die Listen $\mathbf{p}$ mit den Farbvektoren $\mathbf{p_i}$ zweier Kanten m und n werden dann wie folgt verglichen.

Seien

$$\mathbf{p}_i^{\mathrm{m}} = \begin{pmatrix} r_i^{\mathrm{m}} \\ g_i^{\mathrm{m}} \\ b_i^{\mathrm{m}} \end{pmatrix} \quad \text{und} \quad \mathbf{p}_i^{\mathrm{n}} = \begin{pmatrix} r_i^{\mathrm{n}} \\ g_i^{\mathrm{n}} \\ b_i^{\mathrm{n}} \end{pmatrix}$$

$$\text{error} = \frac{1}{n} \sum_{i=1}^{n} d(\mathbf{p}_i^m, \mathbf{p}_i^n)$$

wobei

$$n = \min(\text{len}(\mathbf{p}^{\mathrm{m}}), \text{len}(\mathbf{p}^{\mathrm{n}}))$$

$$d() \mathrel{\widehat{=}} \text{euklidische Distanz}$$

Lösung Um die Farbwerte entlang der Kante zu ermitteln, wird diese schrittweise selektiert. Für jeden Pixel a der Kante, für den ein Farbwert generiert werden soll, wird ein Vektor $\mathbf{ab}$ von a zu einem weiteren Punkt b auf der Kante berechnet. Anschließend wird der Normalenvektor $\mathbf{n}_{ab}$ berechnet, der orthogonal auf der Kante bei a steht. Der Farbwert wird dann an der Koordinate $a + \mathbf{n}_{ab}$ ermittelt. Dieses Vorgehen kann man beispielhaft in Abbildung 28 sehen.

Die Methode Normalenvektoren zu generieren für die Kante wurde schon für den ColorMatcher Unterunterabschnitt 5.3.3 benutzt, weshalb auch hier Code wiederverwendet werden konnte.

Bei dem GeoMatcherRGB lassen sich einige Hyperparameter einstellen. Wie viele Farbwerte aus der Kante genommen werden, und wie lang der Normalenvektor ist, und somit

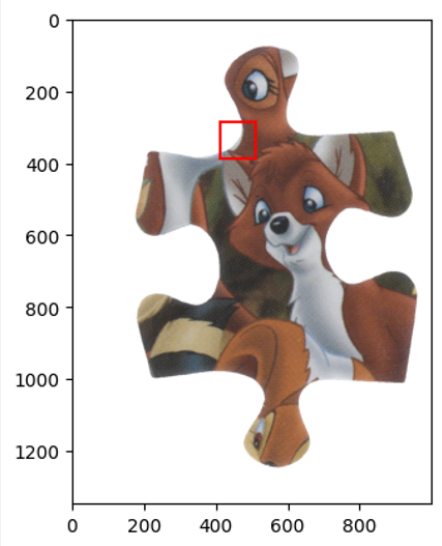


Abbildung 27: Beispiel Puzzleteil

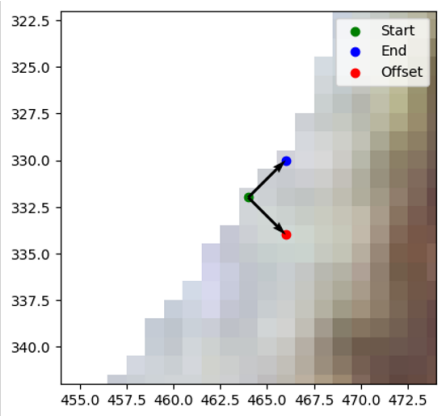


Abbildung 28: Farbextrahierung mit Normalenvektor

wie groß der Abstand zur Kante ist. Diese wirken sich auf die Effektivität des Matchers aus. Um gute Werte zu finden, wurde die Matcher Evaluation Unterunterabschnitt 5.3.5 verwendet. Dabei hat sich herausgestellt, dass es gut ist, viele Farbwerte zu nehmen und nahe (2 Pixel Abstand) an der Kante zu sein. Zu nah an der Kante ist jedoch auch nicht gut, da teilweise Schatten oder Rand an den Kanten ist, der sonst die Auswertung negativ beeinflusst. In der fertigen Lösung, wird für jeden Punkt auf der Kante der Farbwert berechnet, womit alle Kanten des 49-Teile-Puzzles innerhalb von 3.4 Sekunden verglichen werden können, ohne die Bilder der Puzzleteile zu skalieren.

5.3.3 ColorMatcher **[Maximilian Weigle]**

Das Farbmatching ist ein Algorithmus, welcher dafür genutzt wird die Ähnlichkeit der Kanten zweier Puzzleteile zu bestimmen. Anders als bei den geometrischen Verfahren werden hierbei die Farbwerte der Pixel an den Kanten für den Vergleich verwendet.

Die Grundannahme für das Verfahren ist, dass, wenn die Farbwerte entlang zweier Kanten möglichst ähnlich zueinander sind, davon ausgegangen werden kann, dass diese Kanten zusammengehören.

Beschreibung des Verfahrens Zunächst muss mithilfe der bereits extrahierten Kanten-Koordinaten, ein Bereich definiert werden, dessen Pixel für den Vergleich verwendet werden. Da sich die Farbwerte entlang einer Kante stark voneinander unterscheiden können, muss diese in mehrere Segmente oder PPatchesäufgeteilt werden, welche dann einzeln miteinander verglichen werden. Die Größe und Anzahl dieser Patches ist konfigurierbar, standardmäßig aber auf 50 gleich große Patches eingestellt, welche jeweils 5 Pixel ins Bild hineinragen (siehe Abbildung 29). Dies entspricht einer ungefähren Größe der Patches

von 5x5 Pixeln, was auch von Zhi et al. empfohlen wird [12].

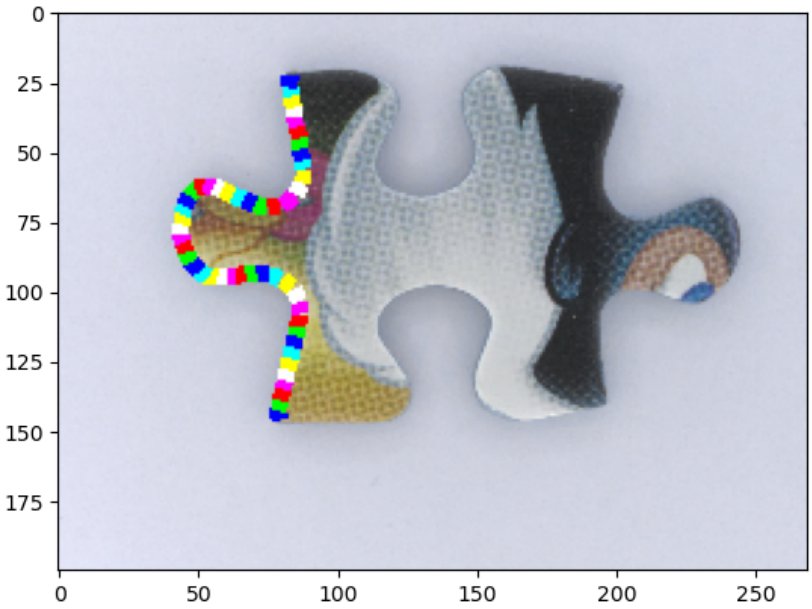


Abbildung 29: Farbliche Kennzeichnung der Patches entlang einer Kante

Anschließend wird für jeden dieser Patches ein Histogramm über die Farbwerte bestimmt. Dafür werden, wie in Abbildung 30 zu sehen, die Hue- und Saturation-Komponenten der HSV Darstellung verwendet, da diese die meisten Informationen über einen gleichmäßigen Übergang zwischen der linken und rechten Kante beinhalten. Alternativ kann das Verfahren auch nur in einer Dimension mit den Graustufen-Werten der Bilder verwendet werden, was eine schlechtere Genauigkeit bei besserer Performance mit sich bringt.

Die Verwendung von Histogrammen hat gegenüber anderen Verfahren, wie beispielsweise einem durchschnittlichen Farbwert über alle Pixel, den Vorteil, dass nur tatsächlich existierende Farbwerte in den Vergleich mit aufgenommen werden. Bei zwei sehr kontrastreichen farbigen Flächen innerhalb eines Patch, würde sonst ein Durchschnitt berechnet, welcher im Patch selbst überhaupt nicht vorkommt.

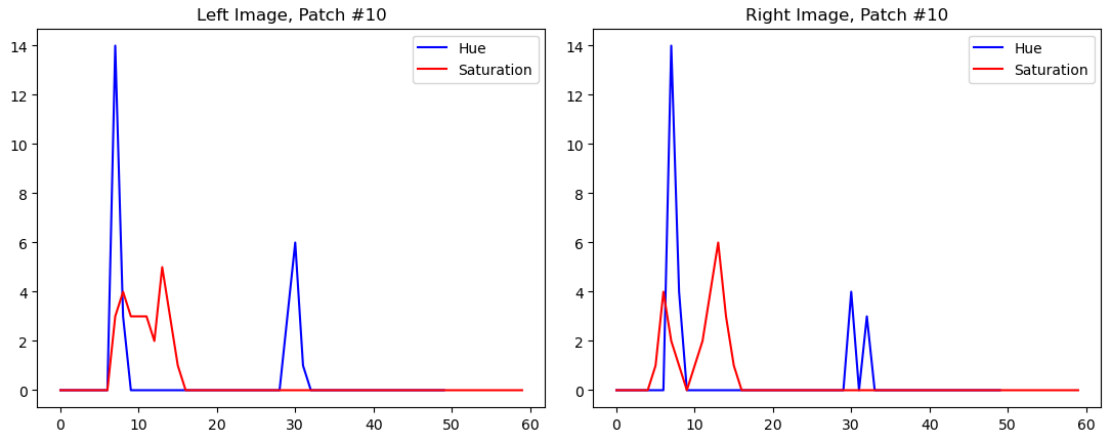


Abbildung 30: Histogramme zweier übereinstimmender Patches

Im nächsten Schritt werden dann Patch für Patch das Histogramm der rechten mit dem der linken Kante auf Korrelation verglichen. Das Ergebnis hiervon ist ein Wert zwischen 0 und 1, welcher die Ähnlichkeit dieser beiden Patches beschreibt (siehe Abbildung 31).

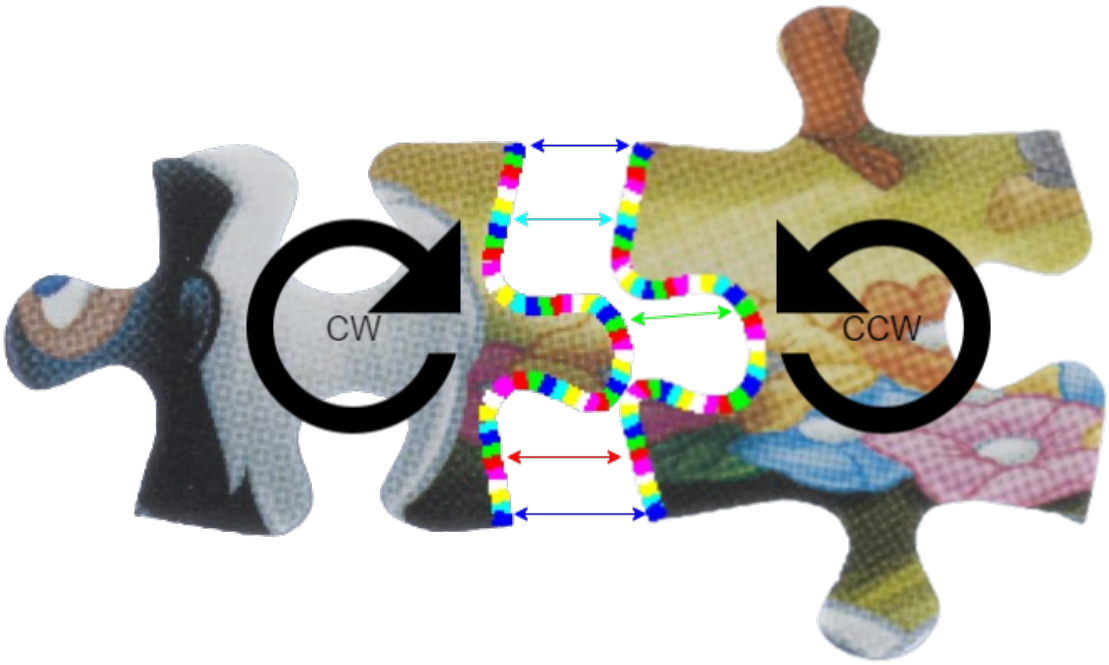


Abbildung 31: Vergleich der jeweils korrespondierenden Patches

Wird nun ein Mittelwert über die Korrelationswerte aller Patches bestimmt, dann gibt dieser die Ähnlichkeit der beiden Katen zueinander an. Dieser Wert kann dann an der entsprechenden Stelle in die Match-Matrix eingetragen werden, welche vom Solver Algorithmus verwendet wird um das Puzzle zu lösen.

Größe der Patches Die Wahl der Patch-Größe ist sehr wichtig für die Qualität der Ergebnisse. Sie sollen einerseits möglichst klein sein, da dann eine Übereinstimmung der Farbwerte lokal möglichst genau verortet werden kann, die Genauigkeit des Matching also steigt. Andererseits besteht bei zu kleinen Patches die Gefahr, dass Aureißer ein zu hohes Gewicht bekommen. Beispielsweise sind die Fotos oftmals nicht perfekt abgeschnitten, sodass noch ein Streifen grauer Pixel oder ein Schatten am Rand zu sehen ist, der das Ergebnis verfälschen kann. Auch ist es möglich, dass die Ränder der gegenüberliegenden Patches nicht perfekt aufeinander passen, sodass Pixel die eigentlich ins nächste Patch gehören, in den Vergleich mit einfließen. Die Wahl von ungefähr 5 x 5 Pixel großen Patches hat sich bei der gewählten Auflösung der Bilder als guter Kompromiss herausgestellt.

Probleme und Lösungen Beim Vergleich der Patches entlang zweier Kanten muss darauf geachtet werden, dass nicht an den entgegengesetzten Enden der aneinander gelegten Kanten begonnen wird. Hierbei ist die Eigenschaft hilfreich, dass die Eck- und Kanten-Koordinaten in ihrer Reihenfolge im PuzzlePiece Objekt stets im Uhrzeigersinn (Clockwise Ordering/CW) abgespeichert werden. Somit müssen an einer der Kanten die Patches in umgekehrter Reihenfolge (damit Counter-Clockwise) iteriert werden, damit die aneinander liegenden Patches miteinander verglichen werden (siehe Abbildung 31).

Durch die Clockwise-Anordnung wird garantiert, dass eine nach rechts zeigende Normale immer ins Puzzleteil "hineinßeigt. Dies konnte genutzt werden, um bei der Bestimmung der Patches vom Rand ausgehend den Bereich 5 Pixel weit ins Bild zu erweitern.

Die Größe der Bilder von ca. 1000x800 Pixeln war für die Berechnung und den Vergleich vieler Histogramme zu groß und hat die Performance extrem verschlechtert (Berechnung einer Match-Matrix in ca. 30 Minuten). Das Skalieren der Bilder auf 20% der Größe hat dabei Abhilfe geschaffen, die Berechnungszeit auf nur noch unter 10 Sekunden zu bringen, ohne dabei die Qualität der Matches merklich zu verschlechtern.

Auch die Berechnung der Histogramme musste optimiert werden, um in einer vertretbaren Zeit berechenbar zu sein. Hierfür wurde zunächst die Anzahl der verwendeten Kanäle von 3 (RGB) auf 2 (Hue und Saturation der HSV-Darstellung) reduziert. Außerdem ist es nicht notwendig für alle 256 möglichen Werte eines Farbkanals Bins im Histogramm bereitzustellen. Hier konnten mit 50, bzw. 60 Bins die Übereinstimmung genügend genau festgestellt werden.

5.3.4 XOR Matcher **[Markus Ertl]**

Im folgenden soll die Funktionsweise als auch Einbindung ins Projket des XOR-Matchers genauer betrachtet werden.

Grundidee des XOR matchers

Der XOR-Edge-Matcher basiert auf der Überlagerung der Binärmasken zweier Puzzelkanten 32. Ziel ist es, durch Rotation und Verschiebung der Kanten eine minimale Überlappung und den geringsten Leerraum zu erreichen. Diese Methode berücksichtigt ausschließlich geometrische Aspekte des Puzzels, ohne Farben oder Muster der Puzzelteile

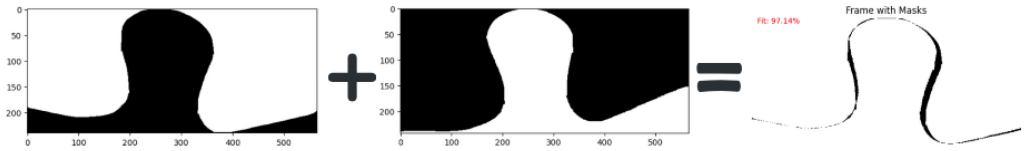


Abbildung 32: Grundidee des XOR Matchers

zu berücksichtigen.

Vorbereitung der Daten

Die Datenvorbereitung umfasst die Umwandlung einer Kantenrepräsentation in eine Binärmaske. Dafür sind mehrere Zwischenschritte notwendig:

1. Normalisierung Die Ausrichtung der Kanten variiert. Sie können horizontal oder vertikal mit unterschiedlichen Abständen zum Rand angeordnet sein. Um verwendbare Binärmasken zu generieren, müssen diese Abweichungen normalisiert werden. Dies beinhaltet die Rotation der Kanten um 90°, falls nötig, und die anschließende Verschiebung an den oberen linken Bildrand. Die Notwendigkeit einer Rotation wird durch den Vergleich der Varianz entlang der x- und y-Achse bestimmt:

$$\mathrm{Var}(x) < \mathrm{Var}(y) \implies \text{vertikale Kante}$$
$$\mathrm{Var}(x) > \mathrm{Var}(y) \implies \text{horizontale Kante}$$

Die Rotation wird durch eine Achsenvertauschung gefolgt von einer Spiegelung entlang der x-Achse durchgeführt:

$$B = S_x A^T$$

wobei

$$S_x = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

die Spiegelungsmatrix entlang der x-Achse ist und A^T die transponierte Matrix von A darstellt. Die Matrix A repräsentiert dabei das Eingangsbild, wobei die Kante mit dem Wert 1 dargestellt wird, und der Rest mit dem Wert 0.

2. Binärmasken Die normalisierten Kanten werden anschließend gefüllt. Nach verschiedenen Experimenten mit Techniken wie "fillPoly" von OpenCV und "floodFill" von scikit-image, wurde ein eigener Füllalgorithmus entwickelt. Die Binärmasken werden von oben nach unten gefüllt, wobei die Kante genau ein Pixel breit sein muss. Ein x-zu-y-Mapping

identifiziert den letzten zusammenhängenden weißen Pixel jeder Zeile als Kantenpixel. Der Füllvorgang wechselt die Farbe beim Erreichen einer Kante zwischen Weiß (1) und Schwarz (0). Der Algorithmus wird in Abbildung 2 detaillierter erklärt.

Algorithm 2 Maskenfüllungsalgorithmus

Input: BildMaske, KantenMapping
Output: GefüllteMaske
Initialisiere GefüllteMaske mit den Abmessungen von BildMaske
Setze alle Pixel von GefüllteMaske auf 0 (Schwarz)
for Jede Spalte y von oben nach unten **do**
    $farbe \leftarrow 1$ ▷ Startfarbe Weiß
    for Jedes Pixel x in Spalte y **do**
        if x ist in KantenMapping für y **then**
            $farbe \leftarrow 1 - farbe$ ▷ Wechselt zwischen 0 und 1
        end if
        Setze Pixel (x, y) in GefüllteMaske auf $farbe$
    end for
end for
return GefüllteMaske

Bei der Auswertung der Ergebnisse wurde festgestellt, dass der Algorithmus Schwierigkeiten bei der Verarbeitung der Scheitelpunkte der Konturen aufweist. Um diese Bildfehler zu korrigieren, wird folgender Ansatz verfolgt: Wenn ein Pixel und seine beiden horizontal benachbarten Pixel unterschiedliche Werte aufweisen, wird das betrachtete Pixel an die Farbe seiner Nachbarn angeglichen.

Der aktuelle Verarbeitungsschritt berücksichtigt nicht, ob es sich bei den Kanten um männliche oder weibliche Typen handelt. Es ist jedoch notwendig, die Binärmaske entsprechend dem im Edge-Objekt definierten Kantentyp anzupassen. Dies geschieht durch die Auswertung der Randpixel: Überwiegen weiße Pixel, handelt es sich vermutlich um eine weibliche Kante; ein Überschuss an schwarzen Pixeln deutet auf eine männliche Kante hin. Stimmt der erkannte Kantentyp nicht mit dem gespeicherten Typ überein, wird das Bild invertiert – weiße Pixel werden schwarz und umgekehrt.

Während des Optimierungsschritts werden die Masken korrekt positioniert und gegebenenfalls leicht rotiert. Hierbei kann ein kleiner schwarzer Rand entstehen, der jedoch nicht als Fehler betrachtet werden sollte. Um mögliche visuelle Störungen zu minimieren, wird zusätzlich ein Padding am Rand der Maske hinzugefügt, um die Darstellung zu verbessern.

Optimierung

Nun geht es um die eigentliche XOR-Optimierung. Dieser Schritt besteht im Grunde aus folgenden sich wiederholenden Schritten:

Masken in Frame platzieren Als erstes werden die beiden Binärmasken in einem Frame platziert. Dabei wird die Binärmaske der weiblichen Kante am oberen Rand platziert, und die Maske der männlichen Kante wird so platziert, dass sie bereits etwas mit der weiblichen Binärmaske überlappt. Diese Überlappung wird mit XOR berechnet.

Fit berechnen Um nun den Fit zu berechnen werden alle weißen und schwarzen Pixel gezählt. Daraus wird der Score mit dieser Formel berechnet:

$$1 - \frac{|AnzahlSchwarzerPixel|}{|AnzahlWeierPixel|}$$

Verschieben und Rotieren Zur Verbesserung des Übereinstimmungsscores wird die Position und Orientierung der Masken durch geringfügiges Verschieben und Rotieren feinjustiert. Dieser Prozess wird so lange fortgesetzt, bis keine weitere Verbesserung des Scores möglich ist, was auf das Erreichen eines lokalen Minimums hinweist. In idealen Fällen entspricht dieses lokale Minimum einem globalen Minimum, allerdings ist die Überprüfung dieser Annahme zeitintensiv und nicht unmittelbar realisierbar.

Einbindung in Solver

Der XOR-Optimierer ist, wie auch die anderen Matcher, über das Matcher-Interface in das Projekt integriert.

Bewertung des Matchers

Die Auswertung des Beispiel-Puzzles zeigte, dass der XOR-Optimierer nur teilweise korrekte Ergebnisse liefert. Die isolierte Betrachtung einer Kante erweist sich nicht als ausreichend einzigartig, um verlässlich vorherzusagen, ob zwei Puzzleteile zusammengehören. Eine mögliche Verbesserung der Vorhersagegenauigkeit könnte durch die Kombination des XOR-Matchers mit einem Farb-Matcher erreicht werden.

5.3.5 Evaluierung **[Cedric Holzer]**

Um die Leistung unserer Matcher vergleichen sowie optimale Hyperparameter finden zu können, ist eine objektive Beurteilung ihrer Güte notwendig. Dazu werden im Folgenden verschiedene Metriken untersucht.

Dieses Kapitel nutzt eine händische gefundene Lösung des Puzzles als Referenz. Für jede Kante ist also der korrekte Partner bekannt.

Binärer Prediktor Die Matcher können als binäre Klassifikatoren interpretiert werden. Dafür werden zunächst alle Kanten miteinander verglichen. Für jede Kante wird dann diejenige Partner-Kante ausgewählt, die den besten Matcher-Score erreichte. Dadurch wird für jede Kante so ein Partner als “passend” prädiziert, während alle anderen Kantenpaare als “unpassend” prädiziert werden.

Metrik	Berechnung	Interpretation
Accuracy	$\frac{tp+tn}{tp+tn+fp+fn}$	Anteil aller korrekt prädizierten Kantenpaare
Precision	$\frac{tp}{tp+fp}$	Wahrscheinlichkeit, mit der ein als passend prädiziertes Kantenpaar tatsächlich passend ist.
Recall	$\frac{tp}{tp+fn}$	Wahrscheinlichkeit, mit der ein tatsächlich passendes Kantenpaar als solches prädiziert wird.
F1-Score	$\frac{2*tp}{2*tp+fp+fn}$	Kombination aus Precision und Recall
Specificity	$\frac{tn}{tn+fp}$	Wahrscheinlichkeit, mit der ein unpassendes Kantenpaar als solches prädiziert wird.

Tabelle 1: Gängige Metriken zur Beurteilung der Leistung von binären Klassifikatoren

Mit dieser Interpretation lassen sich anschließend gängige Metriken für binäre Klassifikatoren anwenden. Dazu zählt man zunächst die Anzahl der richtigerweise als passend (tp) und unpassend (tn) und fälschlicherweise als passend (fp) bzw. unpassend (fn) prädizierten Kantenpaaren, stellt also eine Konfusionsmatrix auf. Darauf aufbauend berechnet man die in Tabelle 1 beschriebenen Metriken.

Die Metriken Precision, Recall und F1-Score sind dabei redundant, da das vorliegende Problem eine besondere Form hat: Pro Kante wird genau ein Kantenpaar als passend prädiziert. Wird ein unpassendes Kantenpaar fälschlicherweise als passend prädiziert, so kann das tatsächlich passende Kantenpaar nicht auch noch als passend prädiziert werden. Die Anzahl der falsch klassifizierten Kantenpaare ist also für beide Label gleich, es gilt $fp = fn$.

Trennschärfe Ein zusätzlicher Ansatz zur Beurteilung der Aussagekraft eines Matchers ist die Trennschärfe, die anders als die Betrachtung der binären Klassifikatoren auch die exakten Match-Scores berücksichtigt. Dazu wurde für jede Kante die Differenz aus dem Matcher-Score des passenden Kantenpaares und dem höchsten Matcher-Score eines unpassenden Kantenpaares gebildet. Diese Metrik wird im nachfolgenden als absolute Trennschärfe (selectiveness) bezeichnet. Sie nimmt Werte in $[-1, 1]$ an, wobei ein Wert von -1 bedeuten würde, dass passende Kantenpaare immer einen schlechten Score von 0 erhalten, während mindestens ein unpassendes Kantenpaar den Wert 1 erhält. Ein Wert von 0 wäre mit einem konstanten Matcher-Score trivial erreichbar und ein Wert von 1 wäre perfekt: Passende Kantenpaare hätten einen Score von 1, während alle unpassenden Kantenpaare einen Score von 0 erhalten.

Die relative Trennschärfe wird gebildet, indem der absolute Trennschärfe-Wert durch den Score des besten unpassenden Kantenpaares geteilt wird.

Kombinationen von Matchern Mehrere Matcher können kombiniert werden, indem sie unabhängig voneinander Match-Scores für ein Kantenpaar errechnen und das Produkt der einzelnen Scores als Gesamtscore gewertet wird. Das Bilden des Produktes übergewichtet niedrige Scores dahingehend, dass ein einziger sehr niedriger Einzelscore ausreicht, um den Gesamtscore stark zu senken, selbst wenn viele andere Einzelscores sehr hoch sind.

Metrik	Geo^4	XOR^4	$Geo^2 \cdot XOR^2$
Accuracy	99.83%	97.71%	99.55%
Precision / Recall	92.86%	5.95%	81.55%
Specificity	99.91%	98.84%	99.77%
∅ Trennschärfe (absolut)	0.139	-0.109	0.057
∅ Trennschärfe (relativ)	27.81%	-17.67%	12.24%
∅ Matcher-Score	0.085	0.139	0.102

Tabelle 2: Matcher Evaluation für formbasierte Matcher

Dieses Vorgehen basiert auf der Annahme, dass korrekte Kantenpaare von jedem Matcher einen recht hohen Score erhalten. Inkorrekte Kantenpaare erhalten aber tendenziell von mindestens einem Matcher einen schlechteren Score und werden so stärker aussortiert, selbst wenn die anderen Einzelmatcher sie nicht von dem korrekten Kantenpaar hätten unterscheiden können.

Kombiniert man einen einzigen Matcher mit sich selbst, entspricht dies einer Potenzierung des Scores. Dabei sinken niedrige Scores schneller als hohe, die Trennschärfe steigt automatisch. So hat beispielsweise der GeoMatcher eine absolute Trennschärfe von 0.049. Kombiniert man ihn jedoch mit sich selbst, erreicht er auf einmal einen Wert von 0.087 bei identischen Werten für die Metriken der binären Klassifikation. Der Matcher scheint sich also zu verbessern, ohne dass zusätzliche Informationen oder Fähigkeiten eingeflossen sind. Um daher die nachfolgenden Ergebnisse vergleichbar zu machen, wird stets eine Kombination aus vier Matchern evaluiert. Sind weniger Matcher vorhanden, werden diese mehrfach verwendet. Die Potenzen über den Matchernamen zeigen die Gewichtung der Matcher an.

Ergebnisse

Formbasierte Matcher Tabelle 2 zeigt die Evaluationsresultate für die formbasierten Matcher, nämlich den Geo- und XOR-Matcher. Beide sind nicht in der Lage, alle Puzzleteile korrekt zu prädizieren. Der GeoMatcher klassifiziert zumindest den überwiegenden Teil der Kantenpaare korrekt und erreicht so eine Precision von 92.86% bei relativ niedriger, aber positiver Trennschärfe. Der XOR-Matcher scheitert beim Großteil der Kanten daran, dem passenden Kantenpaar den jeweils höchsten Score zu verleihen. Dementsprechend ist auch die Trennschärfe hier negativ. Eine Kombination der beiden Matcher liefert keinen Mehrwert und erreicht das Ergebnis des GeoMatchers nicht.

Farbbasierte Matcher Wie in Tabelle 3 erkennbar schneiden die beiden farbbasierten Matcher GeoMatcherRGB und ColorMatcher deutlich besser als die formbasierten ab. Der GeoMatcherRGB kann alleine bereits alle Kantenpaare korrekt prädizieren und erreicht perfekte Accuracy-, Precision- und Specificity-Werte. Zeitgleich zeigt er eine bessere Trennschärfe als der GeoMatcher.

Metrik	RGB^4	$Color^4$	$RGB^2 \cdot Color^2$
Accuracy	100.00%	99.94%	100.00%
Precision / Recall	100.00%	97.62%	100.00%
Specificity	100.00%	99.97%	100.00%
∅ Trennschärfe (absolut)	0.177	0.017	0.063
∅ Trennschärfe (relativ)	71.25%	360.46%	209.28%
∅ Matcher-Score	0.042	0.000	0.004

Tabelle 3: Matcher Evaluation für farbbasierte Matcher

Metrik	$Geo^2 \cdot RGB^2$	$Geo^2 \cdot Color^2$	$Geo^2 \cdot RGB \cdot Color$
Accuracy	100.00%	100.00%	100.00%
Precision / Recall	100.00%	100.00%	100.00%
Specificity	100.00%	100.00%	100.00%
∅ Trennschärfe (absolut)	0.259	0.086	0.165
∅ Trennschärfe (relativ)	85.10%	241.86%	173.83%
∅ Matcher-Score	0.053	0.005	0.016

Tabelle 4: Matcher Evaluation für farb-/formbasierte Matcher kombiniert

Der ColorMatcher prädiziert einige wenige Kantenpaare inkorrekt, schlägt die formbasierten Matcher aber dennoch. Er verfügt über eine sehr niedrige absolute Trennschärfe. Generell erzeugt er sehr niedrige Scores, die durch die Potenzierung stark verringert werden. Dies schlägt sich auch im Durchschnitt der Scores nieder und erzeugt so eine sehr hohe relative Trennschärfe.

In Kombination können beide Matcher alle Kantenpaare richtig prädizieren. Die hohe Genauigkeit des GeoMatcherRGBs wird beibehalten, aber der ColorMatcher sorgt auch hier für deutlich niedrigere Scores und damit eine hohe relative Trennschärfe. Ein realer Mehrwert dieser Kombination über den GeoMatcherRGB ist schwer zu beurteilen.

Kombination von farb- und formbasierten Matchern Eine Kombination von form- und farbbasierten Matchern ist vielversprechend, weil so Kanten ähnliche Formen und Farben haben müssten, um fälschlicherweise hohe Scores zu erhalten. So sollte sich in vielen Fällen, in denen ein einzelner Matcher mehrere Kandidaten liefert, das passende Kantenpaar erkennen lassen. Die Ergebnisse der Evaluation sind in Tabelle 4 dargestellt.

In Kombination mit dem GeoMatcher können beide farbbasierten Matcher alle Kantenpaare richtig prädizieren. Die Trennschärfe der Kombination mit dem ColorMatcher übersteigt trotz eines höheren durchschnittlichen Scores die Werte, die die Kombination der beiden farbbasierten Matcher erreichen konnte. Abbildung 33 zeigt die Verteilung der Matcher-Scores für die Kombination aus GeoMatcher und GeoMatcherRGB für jeweils passende und unpassende Kantenpaare. Dabei zeigt sich eine deutliche Separierung in fast linear trennbare Punktwolken.

In Kombination mit dem XOR-Matcher (nicht dargestellt) ist weder der ColorMatcher

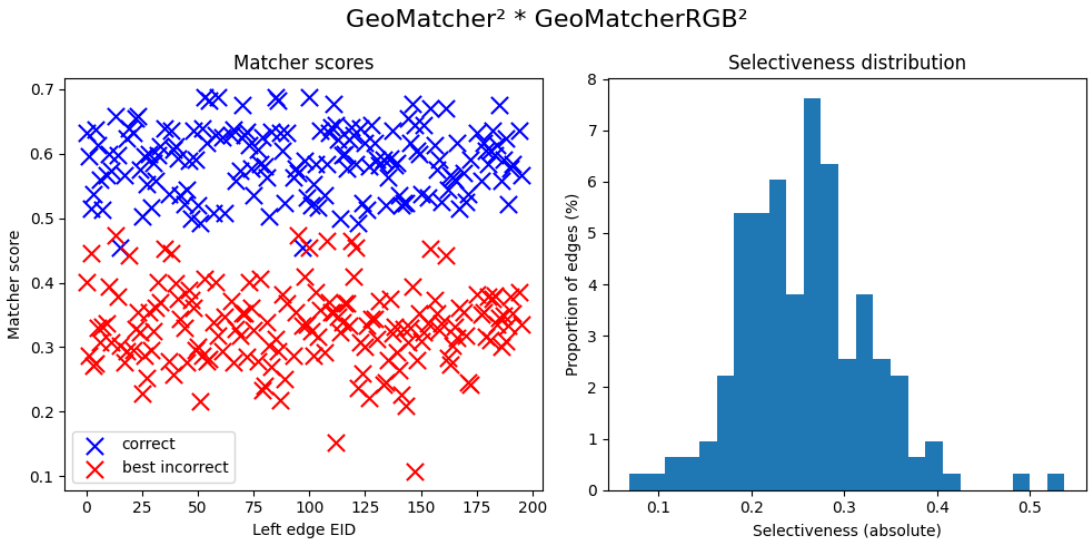


Abbildung 33: Matcher-Scores und Verteilung der Trennschärfe für Kombination aus GeoMatcher und GeoMatcherRGB

noch der GeoMatcherRGB in der Lage, alle Kantenpaare korrekt zu klassifizieren.

Schlussfolgerungen Der GeoMatcherRGB ist der leistungsstärkste Einzelmatcher, der im Rahmen des Projekts entwickelt wurde. Der GeoMatcher ist eine sinnvolle Kombination, um neben der Farbe auch die Form von Kanten als zweite Komponente zu berücksichtigen. Der ColorMatcher wirkt sich vor allem stark senkend auf die Durchschnittswerte der Scores aus und lässt so die relative Trennschärfe ansteigen. Ob er darüber hinaus Vorteile bietet, lässt sich mit dem vorhandenen Datensatz nicht sagen. Abbildung 34 zeigt die Matcher-Scores und die Verteilung der absoluten Trennschärfe für eine Kombination aus GeoMatcher, GeoMatcherRGB und ColorMatcher. Es zeigt sich eine ähnlich starke Ausbildung nahezu linear separierbarer Punktwolken wie in Abbildung 33, wobei die Scores durch den ColorMatcher insgesamt in einen niedrigeren Wertebereich gedrückt werden. Der XorMatcher ist den anderen Matchern deutlich unterlegen. Insgesamt empfiehlt sich damit die Anwendung eine Kombination aus GeoMatcher und GeoMatcherRGB, die gegebenenfalls mit dem ColorMatcher ergänzt werden kann.

Implementierung Die Evaluation der Matcher ist im solver-Repository im package `src.evaluation.matchers` implementiert. Dort findet sich auch ein entsprechendes Jupyter-Notebook, in welchem die Evaluation im Detail nachvollzogen werden kann.

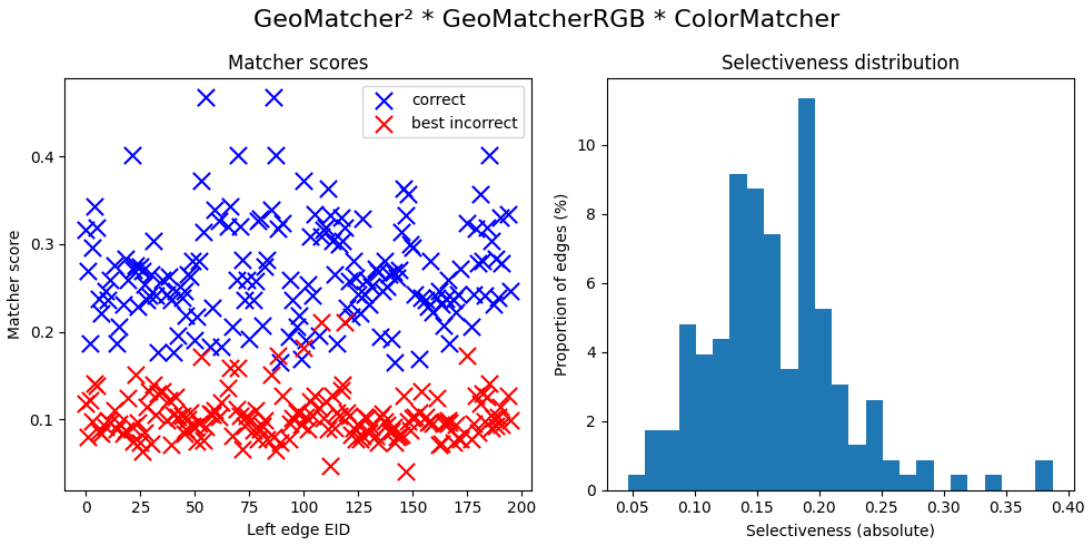


Abbildung 34: Matcher-Scores und Verteilung der Trennschärfe für Kombination aus GeoMatcher, GeoMatcherRGB und ColorMatcher

5.4 Solver Algorithmen

5.4.1 Dimensionsbestimmung [Luis Schubert]

Da zunächst die Dimensionen des Puzzles nicht bekannt, diese jedoch zum Lösen der Puzzles hilfreich sind, können sie wie folgt bestimmt werden. Um die Anzahl der Zeilen r und Spalten c aus der Anzahl der Puzzleteile p und Kanten k zu berechnen dient folgendes Gleichungssystem:

$$\begin{aligned} g_1 :=& p = r \cdot c \\ g_2 :=& k = 2(c + r) - 4 \quad . \end{aligned}$$

Auflösen nach r und c ergibt

$$\begin{aligned} c =& \frac{2 + \frac{1}{2}k + \sqrt{(4 + 2k + \frac{1}{4}k^2 - 4p)}}{2} \text{ und} \\ r =& \frac{p}{c} \quad . \end{aligned}$$

5.4.2 Tiefensuche [Jonas Leitner]

Nachdem durch den Extractor 5.2 und den Matcher 5.3 die Grundlagen für die Repräsentation von Puzzleteilen sowie für die Erhebung von Ähnlichkeitswerten zwischen einzelnen Kanten geschaffen wurden, ist es nun möglich, einen darauf aufbauenden Algorithmus zu entwickeln, der das Puzzle lösen kann.

Ein erster primitiver Gedanke war, die Matrix Position für Position mittels einer Tiefensuche (Depth Search) zu durchlaufen und jeweils das Puzzleteil auszuwählen, das am

besten zum vorherigen passt. Dies ist jedoch schwierig, solange man die genaue Größe des Puzzles nicht kennt. Aus diesem Grund war eine weitere Idee, wie wir Menschen es auch oft tun, zuerst den gesamten Rahmen des Puzzles zu konstruieren.

Frame Builder Konzentriert man sich zunächst nur auf ein Teilproblem, ist dieses leichter zu lösen. Für den Puzzle-Rahmen müssen nur noch CORNER- und EDGE-Pieces betrachtet werden. Auch Validierungen, ob ein Puzzleteil mit einer bestimmten Rotation an eine Stelle passt, sind wesentlich einfacher zu handhaben.

Aus diesem Grund wurde der *FrameBuilder* entwickelt, dessen Aufgabe darin besteht, den kompletten Rahmen des Puzzles aufzubauen. Der Algorithmus versucht, den Rahmen im Uhrzeigersinn aufzubauen, bis keine CORNER- oder EDGE-Pieces mehr vorhanden sind und somit der Rahmen geschlossen ist. Der Ablaufplan des gesamten Algorithmus ist in Abbildung 35 dargestellt.

Zur Initiierung wird eines der vier Eckpuzzleteile in die obere linke Ecke positioniert. Im Anschluss beginnt der rekursive Teil des Algorithmus. In jedem Schritt wird das zuvor platzierte Puzzleteil mit seiner Position sowie einer sogenannten Build Direction übergeben. Der Algorithmus beginnt mit der Prüfung der Abbruchbedingung, bei der festgestellt wird, ob noch EDGE- oder CORNER-Puzzleteile zum Platzieren vorhanden sind. Im Anschluss erfolgt die Berechnung der nächsten Position. Die nachfolgende Position ist dabei abhängig vom vorherigen Puzzleteil sowie der sogenannten Build Direction.

- Im Falle eines EDGE-Puzzleteils wird die Build Direction beibehalten. Wird also nach rechts gebaut, dann ist die neue Position $x = x + 1, y = y$.
- Wohingegen, wenn das vorherige Puzzleteil eine Ecke war, dann wird die Build Direction gewechselt. Handelt es sich um die zweite Ecke, dann ändert sich die Build Direction von rechts nach unten. Bei der dritten von unten zu links und so weiter.

In einem nächsten Schritt wird für die anliegende *Edge*, eine Liste an Matching Edges vom *Matcher* erstellt. Diese ist sortiert, wobei die am besten passende Kante zuerst kommt. In einer Schleife, die mit dem besten Match beginnt, wird nun das zur Edge dazugehörige Puzzleteil geladen und in die korrekte Position rotiert. Danach wird mit dem zugrundeliegenden Wissen validiert, ob das Puzzleteil auch an die Stelle passt.

Durch die Build Direction weis man automatisch an welcher Stelle die FLAT-Edge platziert sein muss. Diese muss stets nach außen zeigen. Zudem weis man durch die Berechnung der Puzzlegröße (siehe Unterunterabschnitt 5.4.1), an welchen Positionen ein CORNER-Piece kommen muss. Schlägt die Validierung fehl, wird der nächstbeste Match probiert. Andernfalls wird das PuzzlePiece platziert und die Funktion erneut rekursiv aufgerufen. Wurden alle möglichen Matches ausprobiert, jedoch konnte keine erfolgreiche Piece-Platzierung gefunden werden, wird eine Rekursionsstufe höher gegangen und das zuvor platzierte Puzzleteil entfernt. Anschließend wird auf der Ebene das nächstbeste Puzzleteil probiert. Dieser Prozess wird solange wiederholt, bis entweder erfolgreich alle EDGE- und CORNER-Pieces platziert wurden und somit eine Lösung gefunden wurde

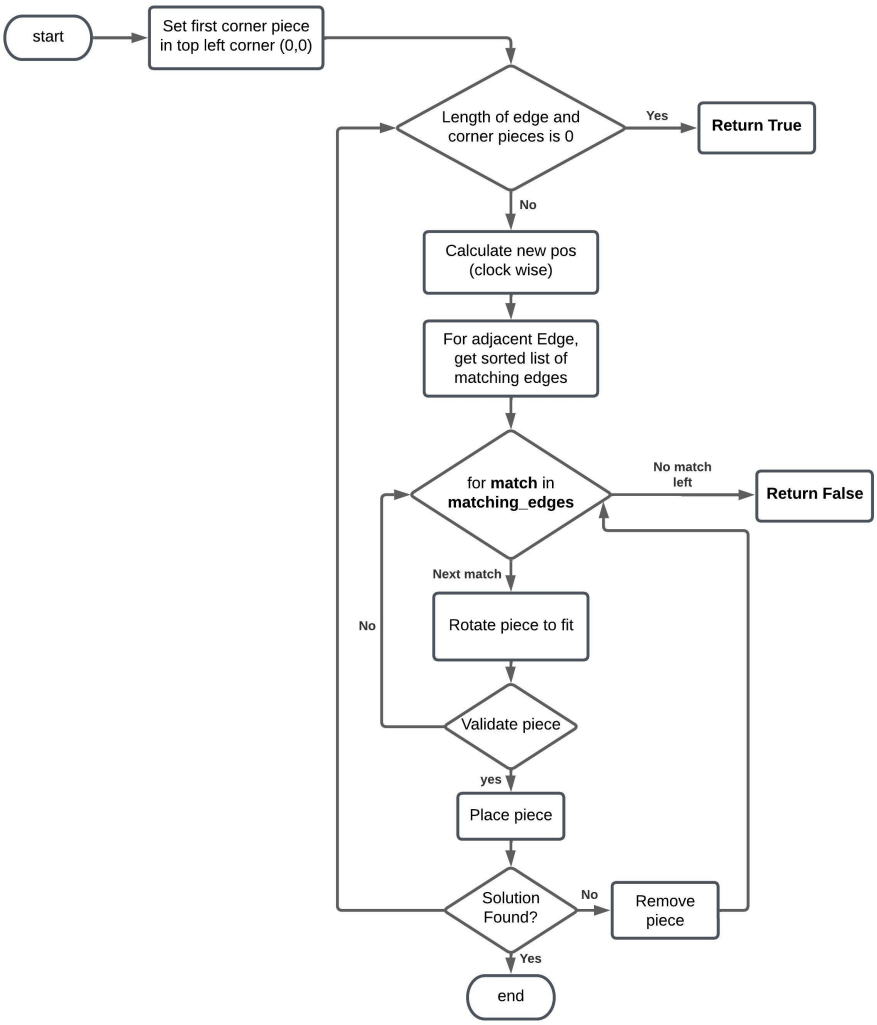


Abbildung 35: Frame Builder Algorithmus

oder bis alle möglichen Optionen ausprobiert wurden und keine Lösung gefunden werden konnte.

Lösen des Puzzles-Inneren Im nächsten Schritt kann das Innere des Puzzles gelöst werden. Da der Puzzlerahmen die Dimension des Puzzles vorgibt, kann nun Reihe für Reihe durchgegangen werden. Der Algorithmus startet an Position $(1, 1)$. Wie im $FrameBuilder$ wird der Algorithmus wieder Schritt für Schritt rekursiv aufgerufen. Zu Beginn wird die Abbruchbedingung geprüft, indem festgestellt wird, ob das Puzzle noch $available_pieces$ zum Setzen hat.

Algorithm 3 Depth-Search-Algorithmus

```
1: Eingabe: Puzzle mit gelöstem Rahmen
2: Ausgabe: gelöstes Puzzle
3: start_pos ← (1,1)
4: NEXT_PIECE(start_pos)
5:                                  ▷ Rekursive Funktion für jedes weitere Puzzleteil
6: function NEXT_PIECE(pos)
7:     available_pieces ← puzzle.available_pieces
8:     if len(available_pieces) == 0 then                          ▷ Abbruchbedingung
9:         return True
10:    end if
11:    adjacent_edges ← puzzle.get_adjacent_edges(pos)
12:    matching_pieces ← matcher.get_matching_pieces(adjacent_edges)
13:    for matching_piece ∈ matching_pieces do
14:        Rotiere matching_piece
15:        Platziere matching_piece an pos
16:        new_pos ← get_next_position(pos)    ▷ Berechnet Position Reihe für Reihe
17:        if NEXT_PIECE(new_pos) then
18:            return True
19:        else
20:            Entferne matching_piece an pos
21:        end if
22:    end for
23: end function
```

Ein Vorteil gegenüber der vorherigen Situation im *FrameBuilder* ist zudem, dass sich an einer Position nun immer mindestens zwei anliegende Kanten befinden. Über die *Puzzle*-Klasse kann für eine bestimmte Position die Menge aller *adjacent_edges* abgefragt werden. Mithilfe dieser kann über die *Matcher*-Funktion *get_matching_pieces*() die Menge aller passenden Puzzleteile berechnet werden, die wie folgt funktioniert:

Für jede Edge in den angrenzenden n Edges wird eine Liste von Matches erzeugt. Aus dieser wird jeweils aus der EdgeID die PieceID zurückgerechnet: $PieceID = EdgeID \div 4$. Über alle n Listen wird dann die Schnittmenge gebildet. Dies ist notwendig, da die passenden Edges auch alle zum selben Puzzleteil gehören müssen. In einem weiteren Schritt werden dann die übrig gebliebenen Pieces für die Position in allen vier Rotierungsvarianten getestet:

1. Dabei wird zunächst festgestellt, ob die passenden Edges auch an den passenden Positionen liegen. Im Falle, dass die *adjacent_edges* beispielsweise oben und links sind, müssen die passenden Edges am Piece auch in der selben Reihenfolge nebeneinander liegen.

2. Des Weiteren kann ein Piece auch in mehreren Rotierungen an die Position passen.

3. Für jede mögliche Position wird zudem der passende Match Score durch Aufsummieren der einzelnen Edge Scores berechnet.

Die Liste der Matching Pieces wird nach Score absteigend sortiert.

In einer Schleife wird nun wieder mit dem am besten passenden Match begonnen. Das Puzzleteil wird in die entsprechende Position rotiert und platziert. Im Anschluss erfolgt die Berechnung der nächsten Position, welche von links nach rechts Reihe für Reihe durchgegangen wird. Im nächsten Schritt wird eine Rekursionsstufe weitergegangen. Falls auf einer Ebene wiederum alle Matches ausprobiert wurden oder keine passenden Matches gefunden wurden, wird in der Ebene höher das letzte Puzzleteil entfernt. Dieser Prozess wird wiederholt, bis eine Lösung gefunden wurde oder alle Optionen ausprobiert wurden.

5.4.3 Best-First-Algorithmus [Jonas Leitner]

Ist die Qualität der Vergleichswerte für die Edges gut, so lässt sich bereits eine hohe Effektivität des Depth-Search-Algorithmus feststellen. Wählt der Algorithmus jedoch gleich zu Beginn ein „falsches" Puzzleteil aus, sucht er sehr tief nach der richtigen Lösung, die er nie finden kann. Er müsste im Grunde den gesamten Suchbaum aufwärts gehen, um zu einem anderen Zweig zu gelangen. Dieser Prozess kann sehr ineffizient sein und viel Zeit in Anspruch nehmen.

Aus diesem Grund wurde ein weiterer Lösungsansatz entwickelt, der eine breitere Suche nach dem bestmöglichen Match ermöglicht. Anstatt nur für eine bestimmte Position den besten Match zu ermitteln, können alle offen liegenden Edges berücksichtigt werden. Genau diesen Ansatz verfolgt der Best-First-Algorithmus (siehe Algorithmus 4).

Algorithm 4 Best-First-Algorithmus

```
1: Eingabe: Puzzle mit gelöstem Rahmen
2: Ausgabe: gelöstes Puzzle
3: exposed_edges ← Initialisiere mit puzzle        ▷ Alle inneren Kanten des Rahmens
4: NEXT_PIECE(exposed_edges)
5:                                  ▷ Rekursive Funktion für jedes weitere Puzzleteil
6: function NEXT_PIECE(exposed_edges)
7:     available_pieces ← puzzle.available_pieces
8:     if len(available_pieces) == 0 then                          ▷ Abbruchbedingung
9:         return True
10:    end if
11:    exposed_edge_scores ← get_exposed_edge_scores()
12:    adjacent_pos_scores ← get_best_adjacent_pos(exposed_edge_scores)
13:    for pos ∈ adjacent_pos_scores do
14:        adjacent_edges ← puzzle.get_adjacent_edges(pos)
15:        matching_piece ← matcher.get_matching_pieces(adjacent_edges)[0]
16:        Rotiere matching_piece
17:        Platziere matching_piece an pos
18:        updated_exposed_edges ← Aktualisiere exposed_edges Liste
19:        if NEXT_PIECE(updated_exposed_edges) then
20:            return True
21:        else
22:            Entferne matching_piece an pos
23:        end if
24:    end for
25: end function
```

Zu Beginn erfolgt die Initialisierung der offen liegenden Kanten (exposed_edges). Dies umfasst alle nach innen gerichteten Kanten des bereits erstellten Rahmens. Im Vergleich zum Depth-Search-Algorithmus wird nun nicht mehr eine Position in die Rekursionsfunktion gegeben, sondern eine Liste an exposed_edges. Im Anschluss muss für alle offen liegenden Kanten jeweils der bestmögliche Matchscore ermittelt werden. Diese werden in *exposed_edge_scores* gespeichert.

Die Match Scores könnten bereits sortiert und mit dem höchsten Match begonnen werden. Es besteht jedoch die Möglichkeit, einen weiteren Schritt einzubauen, wenn davon ausgegangen wird, dass ein Match besser ist, sobald mehrere Edges an einer Position anliegen. Für alle Exposed Edges kann geschaut werden, an welcher Position sie anliegen, der Score der Position ergibt sich durch die anliegenden Edges. Gibt es mehrere Edges, wird der Score summiert. Genau dieser Schritt wird durch die Funktion *get_best_adjacent_pos*() ausgeführt.

Der Algorithmus beginnt mit der besten Position und berechnet für diese wie im Depth-Search-Algorithmus das beste *matching_piece*. Dieses wird rotiert und platziert. Im nächsten Schritt muss die Liste der Exposed Edges aktualisiert werden. Dazu werden die

zuvorigen Exposed Edges, die an der platzierten Position waren, entfernt und die durch das neue Puzzle Piece hinzugekommenen Exposed Edges hinzugefügt. Ansonsten ist die Funktionsweise des Algorithmus identisch mit der des Depth-First-Algorithmus.

Easy-First Ansatz Die Algorithmen sind abhängig von den Vergleichswerten, die der Matcher liefert. In Fällen, in denen die Werte zwischen den einzelnen Matches sehr nah aneinander sind, könnte dies für den Best-First-Algorithmus zu Schwierigkeiten führen. Eine Modifikation stellt der Easy-First-Algorithmus dar. Alternativ zur Festlegung des Scores lediglich am besten Match kann bei der Funktion $get_exposed_edge_scores()$ auch die Differenz zum nächsten Match als Score verwendet werden:

$$score_{eid} = matches[0] - matches[1]$$

Hierdurch wird theoretisch immer der Match zuerst gewählt bei dem man sich am sichersten ist.

Optimierter FrameBuilder Der Best-First- und der Easy-First-Algorithmus können natürlich den $FrameBuilder$ verwenden, der für den Depth-Search-Algorithmus verwendet wird. Dieser bietet jedoch nur die reine Tiefensuche an. Um die Auswahlmöglichkeit von jeweils nur einer möglichen Kante auf zwei zu erweitern, wurde der $FrameBuilder$ weiterentwickelt. Anstatt nur eine definierte Build Direction zu haben, bei der einfach eine Position an die Rekursionsfunktion übergeben wird, werden nun die Exposed Edges übergeben. Es gibt zwar immer nur zwei, aber dadurch kann der Frame von beiden Seiten aufgebaut werden und es kann immer die „Beste" bzw. „Einfachste" Kante gewählt werden.

5.4.4 Visualisierung [Jonas Leitner]

Während der Entwicklung der Algorithmen war es notwendig, eine gute Vorstellung davon zu haben, was der Algorithmus gerade zu tun versucht. Aus diesem Grund wurde ein $Visualizer$ entwickelt, der zu jedem Zeitpunkt die aktuelle Puzzlematrix in einem Bild visualisiert. Dies war sehr hilfreich bei der Identifizierung von Bugs. Da die Bilder der Puzzleteile nicht quadratisch waren, mussten diese zunächst verkleinert und mit Padding versehen werden, um sie zu einem Gesamtbild zusammenfügen zu können.

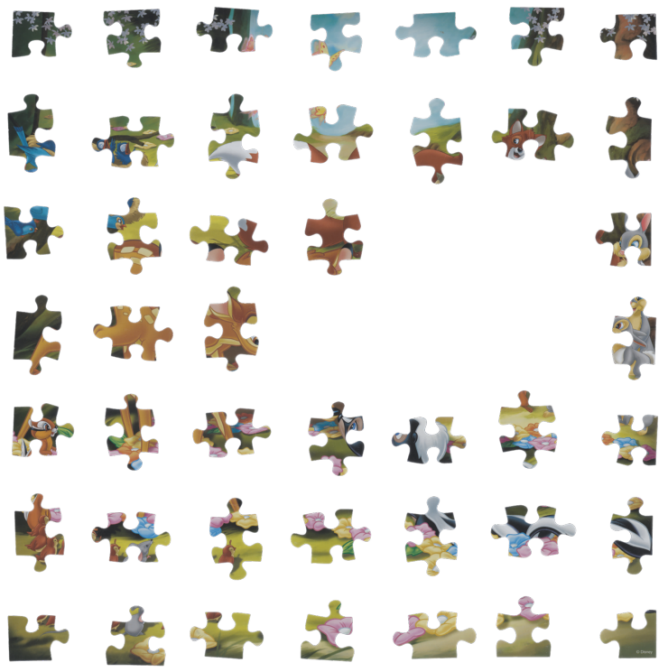


Abbildung 36: Visualisierung eines Puzzle Zustands

5.4.5 Simulated Annealing [Simon Hampp]

Motivation Es wurde ein weiterer Ansatz gesucht, um unser Puzzle zu lösen, der eine alternative Vorgehensweise hat. Bisher wurden klassische Verfahren entwickelt, wie den Depth Search Ansatz Unterunterabschnitt 5.4.2. Zusätzlich wurde in der Vorlesung aber auch Simulated Annealing angesprochen, um das Puzzle zu lösen.

Simulated Annealing Allgemein Simulated Annealing ist ein Optimierungsverfahren. Der Hauptzweck ist es, globale Optima in komplexen, auch nicht differenzierbaren Landschaften zu finden, in denen viele lokale Optima vorhanden sein können. Simulated Annealing beginnt mit einer hohen TTemperatur", die es ermöglicht, große Änderungen zu akzeptieren, auch wenn diese die Lösungsqualität kurzfristig verschlechtern. Mit sinkender Temperatur werden solche ßchlechten"Änderungen seltener angenommen, was der Suche hilft, sich auf eine bessere Lösungen zu konzentrieren.

Vorgehen Zu Beginn wurde Simulated Annealing unter Verwendung der Methoden existierender Bibliotheken von SciPy (scipy.optimize.basinhopping, scipy.optimize.minimize) ausprobiert, um das Annealing zu berechnen. Hierbei sind aber schnell Probleme aufgetreten, da das Puzzle Lösen Problem nicht differenzierbar ist und eine komplexe Optimierungsfunktion besitzt.

um das Puzzle zu lösen, wird eine Bibliothek benötigt, die nicht differenzierbare Funktionen erlaubt, und sich gut anpassen lässt. Während des annealing, müssen in unserem

Fall bspw. mehrere Listen verwaltet werden, um zu vorherigen States zurückzukehren. Diese Anpassung wäre nicht oder nur schwer möglich, wenn eine bestehende Methode von SciPy etc. benutzt wird.

Lösung Bei Simulated Annealing spielen zwei Funktionen eine zentrale Rolle. Die *move*() Funktion, die einen Zustand (*state*) nimmt und daraus einen neuen Zustand generiert. Außerdem die *energy*() Funktion, die die Güte eines Zustands berechnet. Beide Funktionen wurden für das Puzzle lösen implementiert. Als Basis für das tatsächliche annealing, wurde *Simanneal* (https://github.com/perrygeo/simanneal) benutzt.

Da *simanneal.anneal*() Listen als Funktionsparameter braucht und nicht zu viel bestehenden Code umgebaut werden sollte, aber das Puzzle ein $n \times n$ Array ist (mit $n = \#PuzzleTeile$), wird das Puzzle als $1 \times 2n^2$ dargestellt. Dabei stehen n^2 Positionen für das Puzzle und n^2 Positionen für die Rotationen zur Verfügung.

$$[piece_0, rotation_0, piece_1, rotation_1, ..., piece_{n-1}, rotation_{n-1}]$$

Um die Liste zurück auf das Array zu projektieren, wird die Position eines Puzzleteils in der Liste ganzzahlig halbiert und modulo die Seitenlänge genommen, so erhält man die Spalte des Teils im Puzzle Array. Um die Zeile zu erhalten, muss man die Position durch die Seitenlänge ganzzahlig teilen.
Bsp.: Puzzle 7×7 an Index 16

$$Spalte = (16/2 \ modulo \ 7) = 1$$
$$Zeile = (16/2) \ // \ 7 = 1$$

In der *move*() Funktion werden immer zwei zufällig Puzzleteile miteinander vertauscht und beide zufällig rotiert.

Die *energy*() Funktion berechnet anhand der Liste und der Match Matrix, wie gut die Nachbarn jedes Puzzleteils zueinander passen, und summiert alle Matcher Werte auf. Damit diese Funktion effizienter wird, berechnet die Funktion nur die Energie neu, der vom letzten move betroffenen Puzzleteile. So wird die *energy*() Funktion um mehrere Größenordnungen effizienter.

Mit dem Annealing konnte das 7×7 Puzzle nicht gelöst werden, die *energy*() Funktion ist immer noch nicht effizient genug um den riesigen Parameterraum der Puzzles zu lösen. Da dem Annealer auch keine weitere Logik gegeben wird, als die Liste an Puzzle Positionen und Rotationen, probiert der Annealer "blindäus.

Ausblick Damit das Annealing funktionieren könnte, müsste die *energy*() Funktion noch effizienter werden. Außerdem könnte man noch mehr Nachbarschaftsinformationen berücksichtigen. Zum Beispiel sollten korrekte Nachbarschaften nicht mehr von der *move*() Funktion aufgelöst werden.

5.4.6 Mock-Puzzle [Luis Schubert]

Zum Testen und Evaluieren der Solvingalgorithmen, unabhängig von anderen Komponenten des Praktikumsprojekts, simuliert das Mock-Puzzle ein Puzzle ohne dass dieses tatsächlich existieren und digitalisiert sein muss.

Motivation Da das Praktikumsprojekt in mehrere Teilprojekte unterteilt wurde gab es zu Beginn noch keine Matcher oder digitalisierten Puzzles um Solvingalgorithmen zu testen. Um dieses Problem zu lösen lag eine Simulierung (Mocking) von Puzzles nahe. Aufgrund des schnellen Fortschritts der Digitalisierung und des Matchings kam das Mock-Puzzle in dieser Hinsicht kaum zum Einsatz. Zum Ende des Projekts zahlte es sich jedoch zur Evaluierung der Solvingalgorithmen aus, siehe Kapitel 5.4.7.

Benötigte Funktionalität Um seinen Zweck der Simulation eines tatsächlichen Puzzles nachzukommen muss das Mock-Puzzle folgende Möglichkeiten bieten:

- Simulieren von Puzzles beliebiger Größe
- Simulieren von Matchingergebnissen beliebiger Güte
  → Verrauschen der exakten Matchingergebnisse
- Schematische Visualisierung des simulierten Puzzles
  → Erstellen von Bildern der Puzzleteile
- Serialisierung von Puzzles zur reproduzierbarkeit von Tests

Implementierung Diese Anforderungen lassen sich durch vier zentrale Funktionalitäten realisieren:

- Generierung eines gelösten Puzzles
  Hierzu wird über alle Positionen iteriert und je ein Puzzleteil mit zufälliger `id` und Orientierung erstellt. Die Ecken (`Male` / `Female`) werden so gewählt, dass sie zu den bereits existierenden Puzzleteilen passen. Für Kanten, welche an keine bereits existierenden Puzzleteile angrenzen wird zufällig `Male` oder `Female` gewählt. Das Ergebnis ist das gelöste Puzzle, welches zur späteren Verwendung gespeichert wird. Pro erstelltem Puzzleteil wird ein Bild generiert, welches in Textform die `id` des Puzzles, sowie dessen Kantentypen derstellt, siehe Abbildung 37.
- MMischen"des Puzzles
  Um einen realistischen, also gemischten Zustand zu simulieren werden die Puzzleteile anschließend um die zuvor zufällig bestimmte Orientierung zurückrotiert und nach aufsteigender `id` sortiert.
- Erstellen der Matchmatrix
  Die Matchmatrix wird mithilfe der zuvor gespeicherten Lösung erstellt. Hierbei wird pro Kantenpaar überprüft, ob die entsprechenden Puzzleteile in der Lösung

benachbart und so orientiert sind, dass siech das Kantenpaar berührt. Zum indirekten Zugriff auf die Inhalte der Matchmatrix implementiert die Klasse `MockMatcher` das Matcherinterface.

- Verrauschen der Matchmatrix
  Da bisher nur perfekte Matchingergebnisse berechnet werden existiert die Option, alle Einträge der Matchmatrix zu verrauschen.

  Zur Berechnung des Eintrags $m_{i,j}$ an Stelle i, j der Matchmatrix wird zunächst die Normalverteilung $\mathcal{N}$ berechnet und, falls nötig, an dessen Grenzen in den Bereich $[0, 1]$ gespiegelt:

$$m'_{i,j} = \begin{cases} \mathcal{N}(m_t, k_{noise}) & \texttt{if } 0 \leq \mathcal{N}(m_t, k_{noise}) \leq 1 \\ |\mathcal{N}(m_t, k_{noise})| & \texttt{if } \mathcal{N}(m_t, k_{noise}) < 0 \\ 1 - \mathcal{N}(m_t, k_{noise}) + 1 & \texttt{if } \mathcal{N}(m_t, k_{noise}) > 1 \end{cases},$$

  wobei $m_t \in \{0, 1\}$ das tatsächliche Matcherergebnis und k_{noise} der Verauschungsgrad ist. Eine Begrenzung vom $m'_{i,j}$ auf den Bereich $[0, 1]$ ergibt anschließend $m_{i,j}$:

$$m_{i,j} = \begin{cases} m'_{i,j} & \texttt{if } 0 \leq m'_{i,j} \leq 1 \\ 0 & \texttt{if } m'_{i,j} < 0 \\ 1 & \texttt{if } m'_{i,j} > 1 \end{cases}.$$

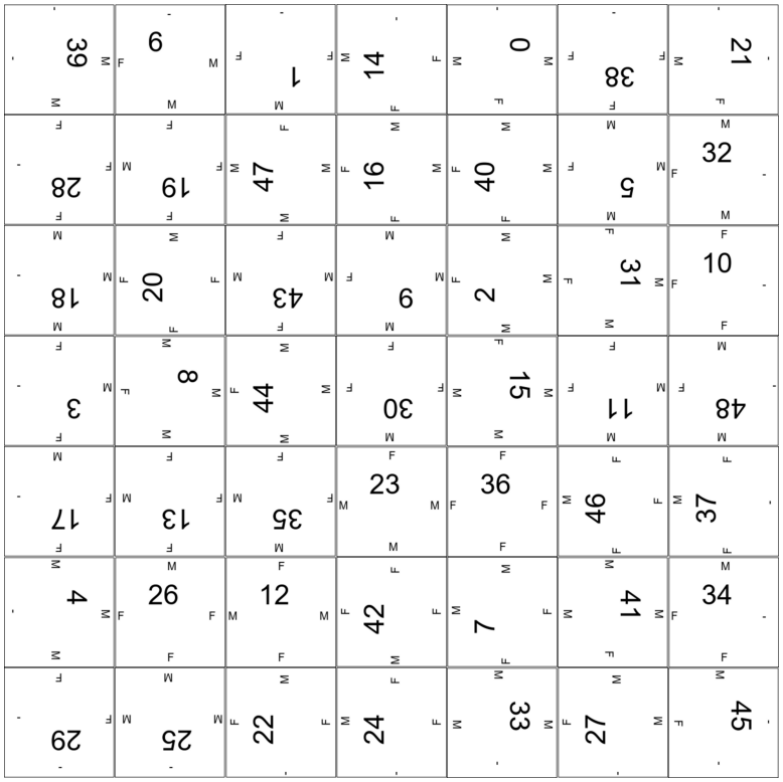


Abbildung 37: Beispiel eines 7×7 Mock-Puzzles.

5.4.7 Evaluierung **[Luis Schubert]**

Im Folgenden werden die Erfolgsrate und Laufzeit der Solvingalgorithmen untersucht.

Ziel der Evaluierung Während eine Evaluierung anhand von realen Puzzles die aussagekräftigste ist, so ist ihre Durchführung für viele verschiedene Puzzlegrößen sehr aufwändig. Um die Fähigkeiten der Solveralgorithmen angemessen zu evaluieren ist dies jedoch unabdinglich, da nur so die Grenzen der einzelnen Implementierungen erkundet werden können.

Durchführung Betrachtet sollen je Solveralgorithmus die Erfolgsquote beim Lösen verschiedener Puzzles und die dafür benötigte Zeit. Hierzu werden Mock-Puzzles mit verschiedenen Dimensionen und Verrauschungsgraden jedem Solver zum Lösen übergeben und hierbei die Ausführungszeit gestoppt. Um ein allgemeingültigeres Ergebnis zu erhalten wird dies, jeweils mit verschiedenen Puzzles, zehnmal durchgeführt. Die Mock-Puzzles sind immer quadratisch, um eine einheitliche Darstellung zu ermöglichen. Da alle Solvingalgorithmen einen Frame-Builder verwenden und bei einem quadratischen Puzzle die Anzahl der Puzzleteile im Rahmen minimal ist, sind quadratische Puzzles ohnehin eine sinnvolle Wahl zur Evaluierung. Da der Annealing Solver das Lösen abbricht, sobald

eine feste Zeit abgelaufen ist und nicht wie die anderen Solvingalgorithmen wenn seine Möglichkeiten ausgeschöpft sind, wird er in diesem Abschnitt nicht betrachtet.

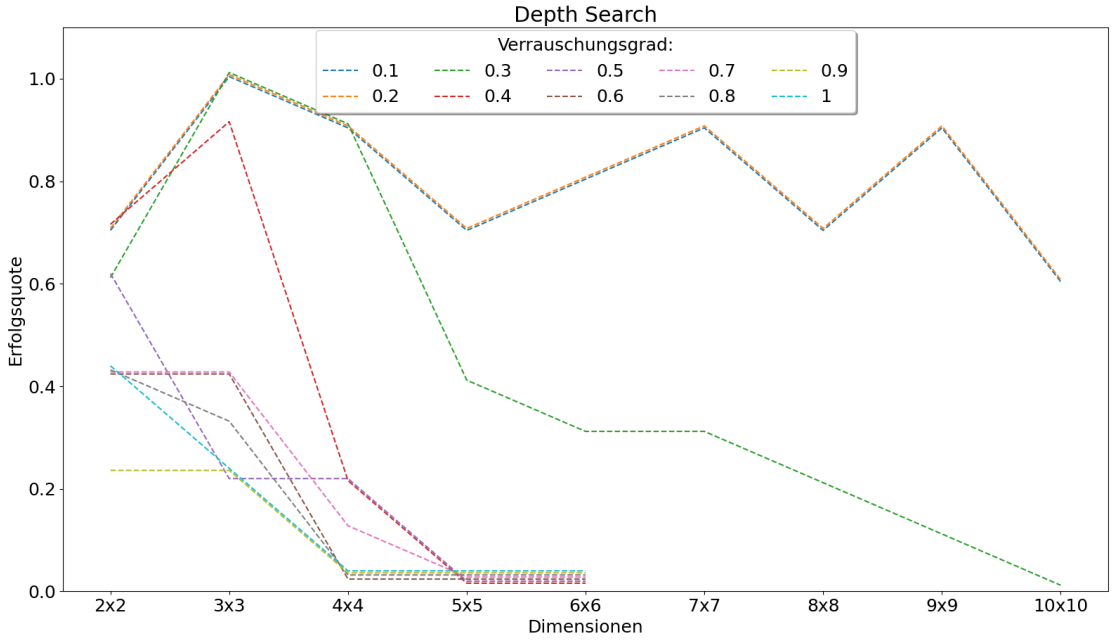


Abbildung 38: Evaluierungsergebnisse des Depth Search Solvingalgorithmus.

Erfolgsquoten Die Ergebnisse für Depth Seach, Easy First und Best First Solver verhalten sich ähnlich. In Abbildung 38 ist zu erkennen, dass bei einem Verrauschungsgrad $k_{noise} \geq 0.4$ Puzzles mit einer Größe von über 3×3 kaum mehr gelöst werden können. Bei $k_{noise} = 0.3$ können ab einer Größe über 7×7 nurnoch wenige Puzzles gelöst werden. Anders verhält es sich für $k_{noise} \leq 0.2$. Damit werden auch bei Größen oberhalb von 7×7 noch über die Hälfte aller Puzzles gelöst. Das ist insbesondere relevant, da die Accuracy eines Matchings für ein Mock-Puzzles mit $k_{noise} = 0.1$ immernoch unterhalb der von tatsächlichen Matchern (siehe Kapitel 5.3.5) liegt. Ein Puzzle mit $k_{noise} = 0.1$ lässt sich also schwerer lösen als ein reales Puzzle, auf welches die implementierten Matcher angewandt werden. Um nicht nur das best-case-Verhalten zu betrachten wird dieser Wert in den folgenden Evaluationen genutzt.

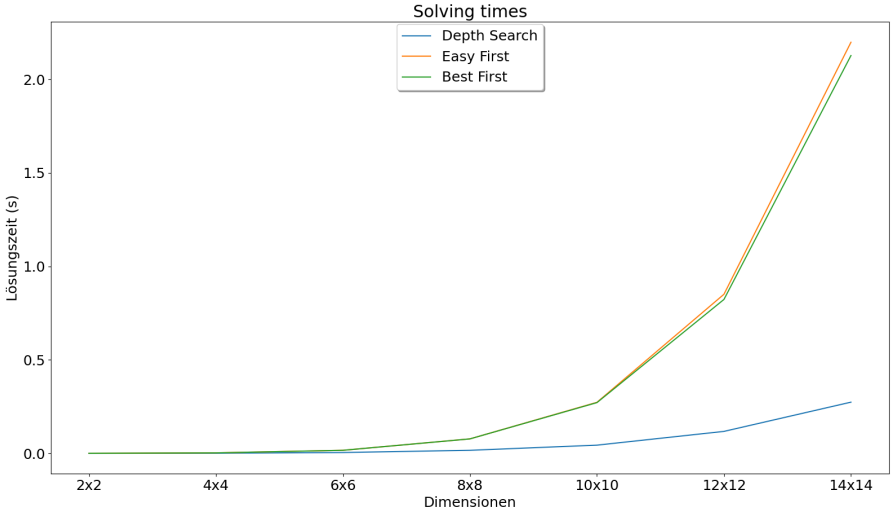


Abbildung 39: Laufzeiten pro Lösungsdurchlauf.

Laufzeiten In Abbildung 39 lässt sich, wie zu erwarten, ein exponenzieller Anstieg in der Laufzeit bei zunehmender Puzzlegröße ablesen. Da der Depth-first Solver hier am besten abschneidet während er in etwa die gleichen Erfolgsraten wie die anderen Solvingalgorithmen liefert wird nur zur Evaluierung größerer Puzzles betrachtet.

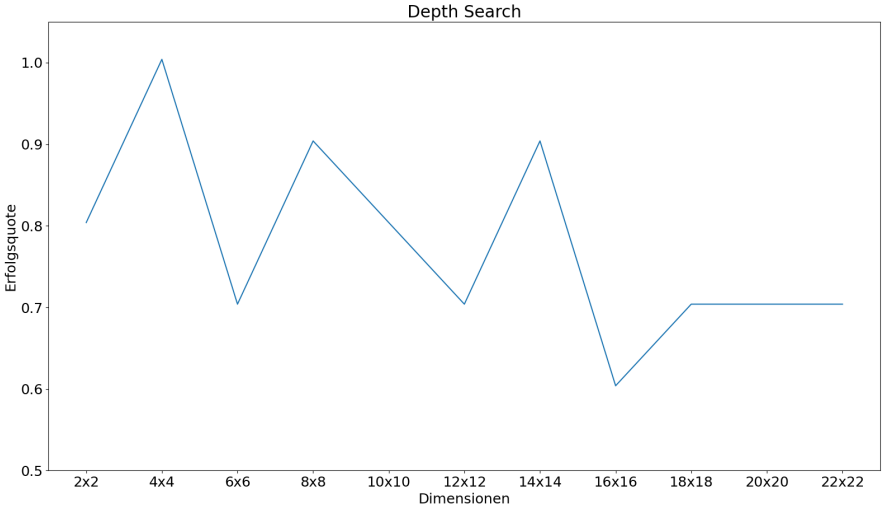


Abbildung 40: Erfolgsquoten bei verschiedenen Puzzlegrößen.

Größere Puzzles Wie in Abbildung 40 zu sehen ist, findet der Depth Search Solver auch bei Puzzlegrößen über 10×10 noch in den meisten Fällen die richtige Lösung. Eine zehnfache Evaluierung eines 22×22 Puzzles dauert etwa 37 Minuten.

5.5 Postprocessing

5.5.1 Linearer Layout Optimierer [Sami Hamza]

Motivation Ziel des Linearen Layout Optimierers ist es die Puzzleteile anhand ihrer detektierten Ecken möglichst gut anzuordnen. Im Gegensatz zu einem greedy Ansatz soll hier jedoch ein globales Optimum gefunden werden. Hierfür wird vorausgesetzt, dass die Korrespondenzen der jeweiligen Puzzleteilkanten bereits bekannt sind.

Herleitung des Least Square Problem Gegeben ist ein gelöstes Puzzle P mit N Zeilen und M Spalten, wobei jedes Puzzleteil P_i eine lokale Basis ${}^{(i)}B$ erhält. Die Basen der Puzzlteile ${}^{(i)}B$ befinden sich im jeweiligen Bildzentrum und weisen parallele Achsen zum Bildkoordinatensystem auf (siehe. Abbildung 41). Somit können die im Bildkoordinatensystem ${}^{(i)}B_{img}$ angegebenen Koordinaten der Exkpunkte durch einfaches subtrahieren den Offset-Vektors (von ${}^{(i)}B_{img}$ zu ${}^{(i)}B$) ermittelt werden. Im Folgenden werden alle Größen stets im zentrierten Koordinatensystem betrachtet. Als Skaleneinheit wird stets die Einheit Pixel verwendet.

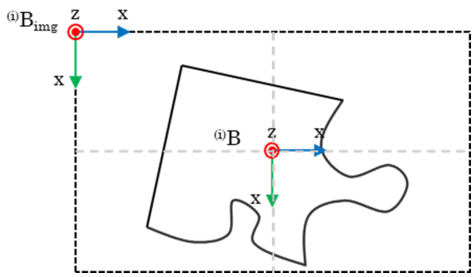


Abbildung 41: Platzierung des Puzzleteilbasen.

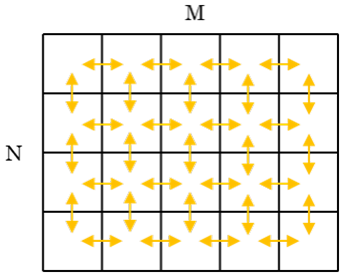


Abbildung 42: Puzzle-Gitter mit Korrespondierenden Kanten (durch gelbe Pfeile markiert).

Zu Beginn muss eine Fehlermetrik aufgestellt werden, über welche die Qualität der Optimierung ermittelt werden kann. Hier werden die Vektoren $\epsilon_{(i,n),(j,k)}$ zwischen korrespondierenden Ecken n und j zweier benachbarter Kanten der Teile i und j im Referenzsystem ${}^{(0)}B$ gewählt (vgl. Abbildung 43).

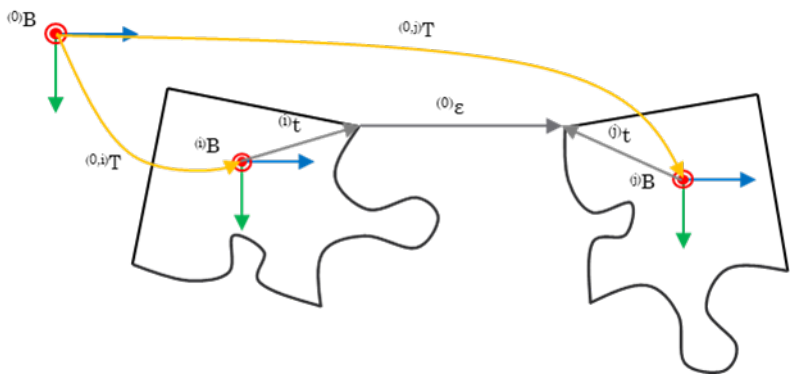


Abbildung 43: Veranschaulichung der Ermittelung der Fehlermetrik.

Daraus resultieren zwei Vektoren pro Kantenpaar. Idealerweise betragen alle Vektoren einen Wert von Null. Die Fehlervektoren $\epsilon_{(i,n),(j,k)}$ können anschließend wie folgt im globalen System beschrieben werden

$$\epsilon_{(i,n),(j,k)} = {}^{(0,i)}T^{(i)}t_n - {}^{(0,j)}T^{(j)}t_k \tag{1}$$

Dabei geben ${}^{(i)}t_n$ bzw. ${}^{(j)}t_k$ jeweils die Koordinaten der korrespondierenden Eckpunkte in ${}^{(i)}B$ bzw. ${}^{(j)}B$ an. Außerdem entspricht ${}^{(0,i)}T \in SE2$ der Pose der Basis ${}^{(i)}B$ im Referenzsystem, welche durch eine homogene Matrix im $\mathbb{R}^2$ ausgedrückt werden kann. Diese beinhaltet eine druch den Rotatinoswinkel $\phi_{r,i}$ parametrisierte Rotationsmatrix $R_z(\phi_{r,i}) \in SO2$, die eine Rotation um die z-Achse beschreibt und sich aus einem Vorrotationswinkel $\phi_{0,i}$ (aus der groben Lösung des Puzzles) und einem Feinjustierwinkel ϕ_i zusammensetzt, und eine durch x_i und y_i parametrisierte Translation $r(x_i, y_i) \in \mathbb{R}^2$ des Ursprungs.

$${}^{(0,i)}T = \begin{bmatrix} R_z(\phi_{r,i}) & r(x_i, y_i) \\ \mathbf{0} & 1 \end{bmatrix}, \text{ mit} \tag{2}$$

$$R = \begin{bmatrix} cos(\phi_{0,i} + \phi_i) & -sin(\phi_{0,i} + \phi_i) \\ sin(\phi_{0,i} + \phi_i) & cos(\phi_{0,i} + \phi_i) \end{bmatrix} \text{ und } r = \begin{bmatrix} x_i \\ y_i \end{bmatrix} \tag{3}$$

Das Ziel der Optimierung ist also die Minimierung des Least Squares Problems aus Summe der euklidischen Norm der Fehlervektoren der Menge aller Korrespondierender Ecken K (hier keine Betrachtung von doppelten Korrespondenzen (i,j) und (j,i) und keine Betrachtung diagonaler Teilkorrespondenzen).

$$\arg\min_x \frac{1}{2} \sum^{|K|} \|\epsilon(x)\|_2^2 \tag{4}$$

Der Parametervektor x setzt sich zusammen aus den Parametrisierungen für die Posen der Puzzleteile und beinhaltet somit pro Puzzleteil drei Parameter $x \in \mathbb{R}^{3|P|}$.

$$x = \begin{bmatrix} \phi_i & x_i & y_i & \dots & \phi_{|P|} & x_{|P|} & y_{|P|} \end{bmatrix}^T \tag{5}$$

Es wäre für eine effiziente Optimierung von Vorteil, wenn die ein linearer Zusammenhang $\epsilon = Ax + b$ zwischen den Fehlervektoren ϵ und dem Parametervektor x besteht. Dieser kann durch heranziehen der Kleinwinkelnäherung und Additionstheoremen für die in Gleichung 3 definierte Rotationsmatix hergestellt werden.

$$\begin{aligned} sin(\phi_{0,i} + \phi_i) &\approx sin(\phi_{0,i}) + cos(\phi_{0,i})\phi_i \\ &= s_{0,i} + c_{0,i}\phi_i \end{aligned} \tag{6}$$

$$\begin{aligned} cos(\phi_{0,i} + \phi_i) &\approx cos(\phi_{0,i}) - sin(\phi_{0,i})\phi_i \\ &= c_{0,i} - s_{0,i}\phi_i \end{aligned} \tag{7}$$

damit folgt für R

$$R = \begin{bmatrix} c_{0,i} - s_{0,i}\phi_i & -s_{0,i} - c_{0,i}\phi_i \\ s_{0,i} + c_{0,i}\phi_i & c_{0,i} - s_{0,i}\phi_i \end{bmatrix} \tag{8}$$

Folglich kann pro Eckenkorrespondenz ein lineares Gleichungssystem mit zwei Gleichungen aufgestellt werden.

$$\epsilon_{(i,n),(j,k)} = \overline{a1} \begin{bmatrix} \phi_i \\ x_i \\ y_i \end{bmatrix} + \overline{a2} \begin{bmatrix} \phi_i \\ x_j \\ y_j \end{bmatrix} + \overline{b} \text{ mit} \tag{9}$$

$$\overline{a1} = \begin{bmatrix} {}^{(i)}t_{x,n}s_{0,i} + {}^{(i)}t_{y,n}c_{0,i} & -1 & 0 \\ -{}^{(i)}t_{x,n}c_{0,i} + {}^{(i)}t_{y,n}s_{0,i} & 0 & -1 \end{bmatrix}$$

$$\overline{a2} \begin{bmatrix} -{}^{(j)}t_{x,k}s_{0,j} - {}^{(j)}t_{y,k}c_{0,j} & 1 & 0 \\ -{}^{(j)}t_{x,k}c_{0,j} - {}^{(j)}t_{y,k}s_{0,j} & 0 & 1 \end{bmatrix}$$

$$\overline{b} = \begin{bmatrix} -{}^{(i)}t_{x,n}c_{0,i} + {}^{(i)}t_{y,n}s_{0,i} + {}^{(j)}t_{x,k}c_{0,j} - {}^{(j)}t_{y,k}s_{0,j} \\ -{}^{(i)}t_{x,n}s_{0,i} - {}^{(i)}t_{y,n}c_{0,i} + {}^{(j)}t_{x,k}s_{0,j} + {}^{(j)}t_{y,k}c_{0,j} \end{bmatrix}$$

Für ein Puzzle P ergeben sich bei Vernachlässigung diagonaler Korrespondenzen $h = 2(2NM - N - M)$ Ecken-Korrespondenzen. Die für alle Korrespondenzen ermittelten resultierenden Gleichungssysteme können zur Matrix $A \in \mathbb{R}^{2h \times 3|P|}$ und zum Vektor $b \in \mathbb{R}^{2h}$ zusammengetragen werden.

Zusätzlich zur Minimierung der Fehlervektoren sollen einige Anforderungen an die Pose des linken, oberen Puzzleteils gestellt werden. Ziel ist es die Anforderungen so zu Formulieren, dass diese sinnvoll über eine Modifizierung des Gleichungssystems $\epsilon = Ax+b$ mit in den gesamt Fehlervektor eingespeist werden können. Der so modifizierte gesamt Fehlervektor ϵ, die Koeffizientenmatrix A und der Vektor b werden dann als ϵ^*, A^* und b^* bezeichnet.

Zu Beginn soll das Referenzsystem ${}^{(0)}B$ in die obere, linke Ecke des Puzzleteils platziert werden. Diese Information kann durch anhängen zweier weiter Zeilen an die Matrix A und den Vektor b in das Problem eingespeist werden. Dabei wird der Fehlerterm minimal für $x_1 =$ und $y_1 = 0$.

$$\begin{bmatrix} 0 & 1 & 0 & 0 & \dots & 0 \\ 0 & 0 & 1 & 0 & \dots & 0 \end{bmatrix} x \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{10}$$

Auch die Ausrichtung der linken oberen Ecke soll durch Adaption der Matrix A und des Vektors b beeinflusst werden. So sollen die Kanten des Puzzleteils entlang der x- und y-Achse des Referenzsystems ${}^{(0)}B$ ausgerichtet werden (siehe Abbildung 44). Hierfür

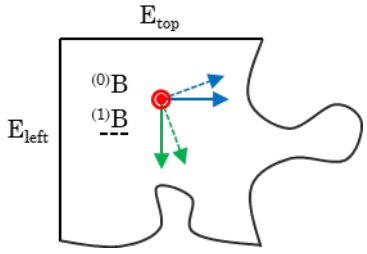


Abbildung 44: Gewünschte Orientierung des Teils (linke, obere Ecke) durch Optimierung der Orientierung der Basis ${}^{(1)}B$.

sollte die y-Koordinate des Kantenvektors ${}^{(0)}E_{top}$ der oberen Kante und die x-Koordinate des Kantenvektors ${}^{(0)}E_{left}$ nach Referenzsystem ${}^{(0)}B$ Null betragen. Hierfür werden die genannten Kannten von ihrer lokalen Basis ${}^{(1)}B$ in das Referenzsystem rotiert, wobei anstatt der Transformationsmatrix ${}^{(0,1)}T$ nur die angenäherte Variante der Rotationsmatrix R (siehe Gleichung 8) verwendet wird, da hier nur die Orientierung von Bedeutung ist.

$$ {}^{(0)}E_{top} = {}^{(0,1)}R^{(1)}E_{top} \tag{11} $$

$$ {}^{(0)}E_{left} = {}^{(0,1)}R^{(1)}E_{left} \tag{12} $$

Daraus ergibt sich für die y-Koordinate der oberen Kante ${}^{(0)}E_{top,y}$ bzw. für die x-Koordinate der linken Kante ${}^{(0)}E_{left,x}$

$$ \begin{bmatrix} {}^{(0)}E_{top,y} \\ {}^{(0)}E_{left,x} \end{bmatrix} = \begin{bmatrix} c_{0,1}{}^{(1)}E_{top,x} - s_{0,1}{}^{(1)}E_{top,y} & 0 & 0 \\ -s_{0,1}{}^{(1)}E_{left,x} - c_{0,1}{}^{(1)}E_{left,y} & 0 & 0 \end{bmatrix} \begin{bmatrix} \phi_1 \\ x_1 \\ y_1 \end{bmatrix} + \begin{bmatrix} s_{0,1}{}^{(1)}E_{top,x} + c_{0,1}{}^{(1)}E_{top,y} \\ c_{0,1}{}^{(1)}E_{left,x} - s_{0,1}{}^{(1)}E_{left,y} \end{bmatrix} \tag{13} $$

Diese können nun analog in die Matrix A und den Vektor b mit aufgenommen werden.

Lösung des Least Squares Problems Mit dem linearen Zusammenhang $\epsilon^* = A^*x + b^*$ kann das Least Square Problem aus Gleichung 4 neu formuliert werden.

Die Inverse der symmetrischen und quadratischen Matrix lässt sich effizient über die Singulärwertzerlegung berechnen (SVD).

Aufgrund der approximation der Rotationsmatrix mit der Kleinwinkelnäherung kann es sein, dass die ermittelte Lösung nicht dem tatsächlichen Optimum entspricht. Allerdings kann durch mehrmaliges Iterieren bei vorheriger Übertragung des Lösungs-Feinjustierwinkels auf den neuen Vorrotationswinkel für Konvergenz zum Optimimum gesorgt werden.

Ergebnisse In Abbildung 45 und Abbildung 46 sind die Ergebnis der Implementierung des beschriebenen Optimierungsverfahrens dargestellt. Abbildung 45 zeigt hierbei eine Darstellung der Puzzleteile anhand ihrer ermittelten Eckpunkte. Abbildung 46 hingegen, zeigt das tatsächliche, anhand der Optimierungsparameter Zusammengesetzte Puzzle. Es ist anzumerken, dass es sich hierbei, um eine Debug Visualisierung handelt, die beim zusammensetzen der Bilder teilweise zum Abschneiden einzelner Teile führt. Allerdings wurde hier keine weitere Zeit investiert, da diese Artefakte in der tatsächlichen Coin-Visualisierung nicht mehr auftreten. Das hier implementierte Verfahren (/home/vscode/workspace/src/solver/algorithm/src/solver/layout_optimizer.py) erweist sich als sehr Zeit effizient und benötigt für die Lösung (unter Verwendung des gegeben Puzzles) nur 30 Millisekunden (Siehe Evaluierung des Solvers in /home/vscode/workspace/src/solver/algorithm/p

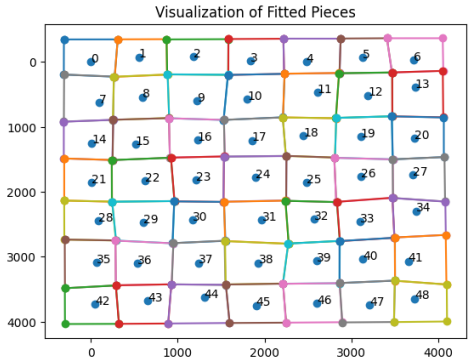


Abbildung 45: Eckpunkt-Visualisierung der gelösten Puzzels zur Besseren Darstellung des Optimierungsergebnisses, da hier die angenommenen Eckpunkte dargestellt werden.

Abbildung 46: Aus den ermittelten Posen zusammengefügte Puzzle. Teilweise abgeschnittene Puzzleteile sind Artefakte der (Debug-) Visualisierung.

5.5.2 Non Linearer Layout Optimierer [Mohammadjavad Zamani Pozveh]

Motivation Das Ziel des nichtlinearen Optimierers ist es, die Gesamtabstände zwischen benachbarten Ecken von Puzzleteilen zu minimieren, was zu einer optimalen Zusammenstellung führt. Diese Methode berücksichtigt die exakte Drehung jedes Puzzleteils, ohne dabei Rotationsschätzungen zu verwenden. Dies führt zu nichtlinearen Operationen, wie die Drehung von Vektoren oder euklidische Abstände zwischen Ecken, die mit linearen Methoden allein und ohne Schätzungen nicht gelöst werden können. Durch die Anwendung einer Fehlerminimierung stellt der Optimierer schließlich sicher, dass der Gesamtabstand zwischen den entsprechenden Ecken des Puzzles minimiert wird. Mit diesem Ansatz wird ein globales Optimum erreicht.

Verfahren Wie bereits erwähnt, ist ein gelöstes Puzzle mit N Zeilen und M Spalten gegeben. Für jedes Puzzleteil sind die Positionen seiner Ecken relativ zum Bildkoordinatensystem angegeben, welches in der oberen linken Ecke des Bildes beginnt (siehe Abbildung 47).

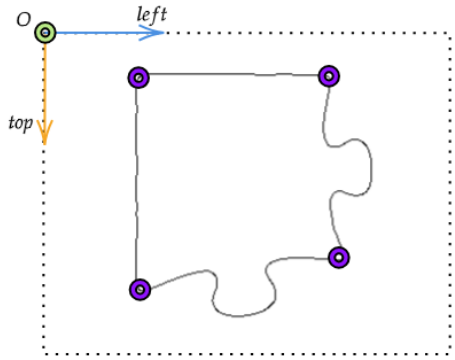


Abbildung 47: Positionen der Ecken im Bildkoordinatensystem (top, left).

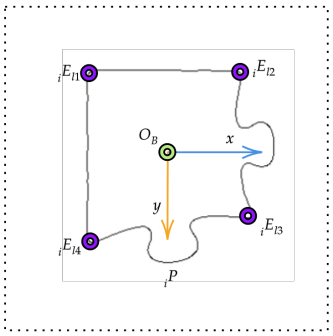


Abbildung 48: Positionen der Ecken relativ zum Mittelpunkt des Bildes (padded image).

Um die Fehlerfunktion zu definieren, müssen zuerst alle Teile in ein globales Koordinatensystem eingefügt werden und die Position jeder Ecke relativ zu diesem Koordinatensystem berechnet werden. Um die Berechnungen für die Fehlerfunktion zu reduzieren, wird die Position der Ecken jedes Puzzleteils in bekannte und unbekannte Variablen zerlegt, wobei die bekannten Variablen nur einmal berechnet und für weitere Berechnungen verwendet werden.

Zunächst wird jedes Bild so aufgefüllt, dass es ein Quadrat bildet, und der Mittelpunkt des quadratisch aufgefüllten Bildes wird als Ursprung des neuen Koordinatensystems gewählt, im Gegensatz zur oberen linken Ecke. Alle Eckpositionen werden dann in dieses neue Koordinatensystem umgewandelt und gespeichert (siehe Abbildung 48):

$$_iE_{l1} \;=\; _iE_{B1} \;-\; O_B \tag{14}$$

wobei $\mathbf{_iE_{l1}}$ die Position der ersten Ecke des Puzzleteils $\mathbf{i}$ relativ zum Mittelpunkt $\mathbf{O_B}$ des Bildes ist und $\mathbf{_iE_{B1}}$ die Position derselben Ecke im Bild-Koordinatensystem ist.

Diese Veränderung des Ursprungs vereinfacht die Berechnungen der neuen Eckpositionen nach der Drehung des Bildes. Anschließend werden die Eckkoordinaten basierend auf ihrer Position (z.B. obere linke oder untere rechte Ecke) sortiert und in einer Matrix der Größe $2N \times 2M$ gespeichert.

Als nächstes werden die Teile im globalen Koordinatensystem positioniert, und die Position des Mittelpunkts des Bildes relativ zum Ursprung des globalen Koordinatensystems wird als globale Teileposition in einer Matrix der Größe $N \times M$ gespeichert (siehe

Abbildung 49). Nun kann die Position jeder Ecke im globalen Koordinatensystem wie folgt berechnet werden:

$$ {}_iE_{g1} \;=\; {}_iE_{l1} \;+\; {}_iP \tag{15} $$

wobei $\mathbf{{}_iE_{g1}}$ die Position der ersten Ecke des Puzzleteils $\mathbf{i}$ und $\mathbf{{}_iP}$ die Position des ersten Puzzleteils im globalen Koordinatensystem ist.

Es wird angenommen, dass jedes Teil durch einen Vektor $\mathbf{T}$ translatiert und um einen Winkel θ gedreht wird, um seine Endposition zu erreichen. Daher können die endgültigen Koordinaten der Ecken jedes Puzzleteils basierend auf den bekannten Variablen der Teil-Position und der lokalen Eck-Position sowie den unbekannten Variablen $\mathbf{T}$ und θ ausgedrückt werden. Nach der Verschiebung und Drehung werden die Eckpositionen im globalen Koordinatensystem wie folgt berechnet:

$$ {}_iE\prime_{g1} \;=\; R_{\theta_i} \cdot_i E_{g1} \;+\; {}_iP + \; {}_iT \tag{16} $$

$$ R_{\Theta_i} = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i) \end{bmatrix} \tag{17} $$

wobei $\mathbf{{}_iE\prime_{g1}}$ die Position der ersten Ecke vom Puzzleteil $\mathbf{i}$ nach der Verschiebung und Drehung ist, $\mathbf{{}_iT}$ der Translationsvektor des Puzzleteils und $\mathbf{R_{\theta_i}}$ die Rotationsmatrix für den Winkel $\theta_\mathbf{i}$ ist.

Die $\mathbf{{}_iP}$ und $\mathbf{{}_iE_{g1}}$ müssen nur einmal berechnet werden und bleiben bei jeder Auswertung der Fehlerfunktion und folglich der Eckposition unverändert.

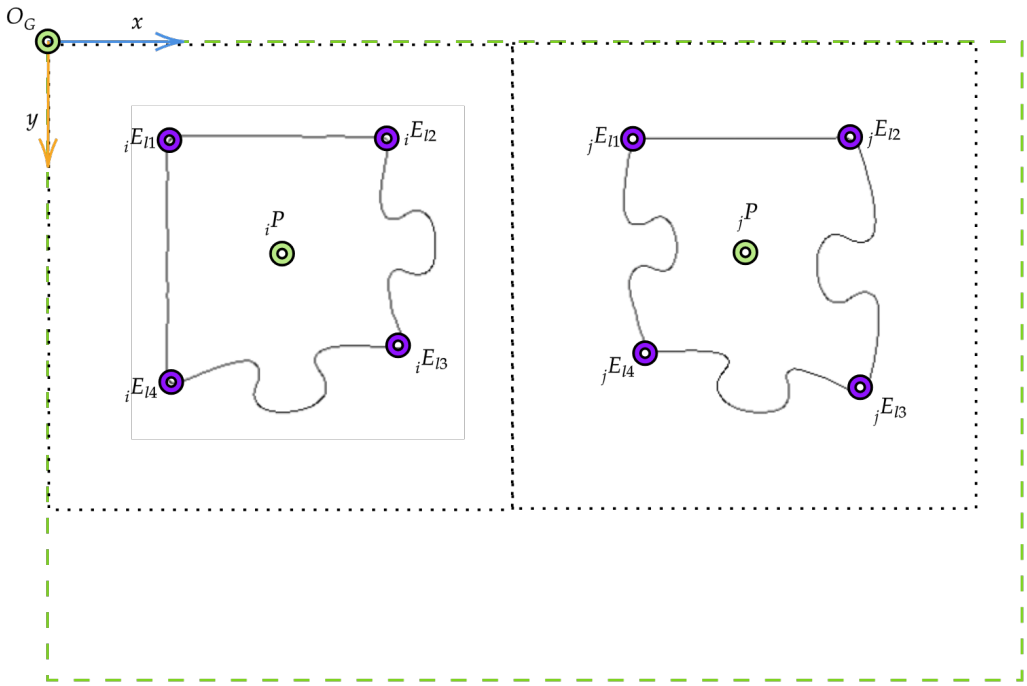


Abbildung 49: Positionierung der Puzzleteile im globalen Koordinatensystem. Wobei $_{\mathbf{i}}\mathbf{P}$ und $_{\mathbf{j}}\mathbf{P}$ die Positionen von Puzzleteilen **i** und **j** im globalen Koordinatensystem sind.

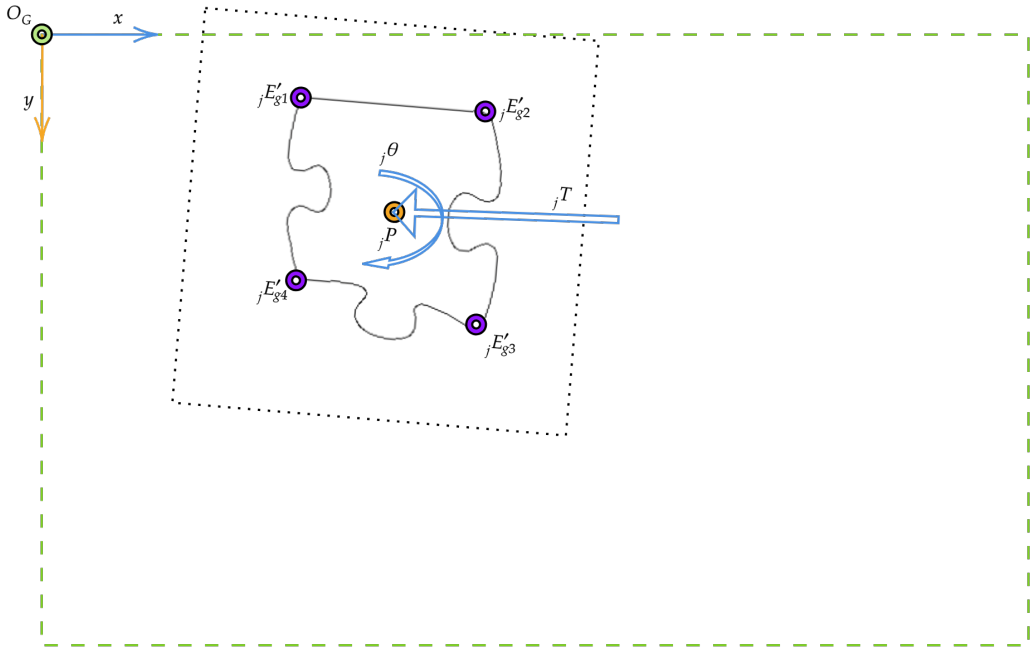


Abbildung 50: Position der Ecken nach Durchführung von Translation $\mathbf{T}$ und Drehung θ.

Mit den Eckpositionen im globalen Koordinatensystem kann die Fehlerfunktion als Summe aller euklidischen Distanzen zwischen benachbarten Ecken definiert werden (siehe Abbildung 51). Dies wird mathematisch dargestellt als:

$$F_{\text{err}}(\overline{E_g}) = \sum_{\text{Nachbarn Ecken } \mathbf{i},\mathbf{j}} \|\mathbf{E}_{gi} - \mathbf{E}_{gj}\|^2 \tag{18}$$

wobei $\mathbf{E}_{gi}$ und $\mathbf{E}_{gj}$ die Positionen benachbarter Ecken sind und $\overline{\mathbf{E_g}}$ die Positionsmatrix aller Ecken ist.

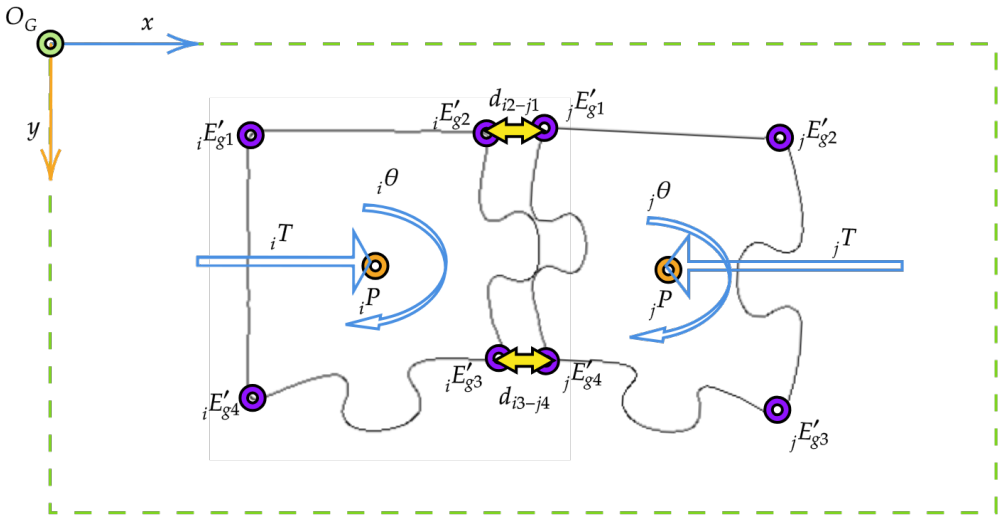


Abbildung 51: Distanzen zwischen benachbarten Ecken ($\mathbf{d_{i2-j1}}$ und $\mathbf{d_{i3-j4}}$) nach Durchführung von Translation $\mathbf{T}$ und Drehung θ.

Minimierung der Fehlerfunktion Mit der definierten Fehlerfunktion ist der nächste Schritt, diese mithilfe von `scipy.minimize` zu minimieren. Es ist erforderlich, einen Vektor der Unbekannten zu erstellen und eine Anfangsschätzung für jede von ihnen bereitzustellen. Der Vektor der Unbekannten ist wie folgt definiert:

$$X = \begin{bmatrix} {}_iT_x & {}_iT_y & {}_i\theta \end{bmatrix}_{1\times 3MN} \tag{19}$$

wobei $\mathbf{X}$ der Vektor der Unbekannten ist. Da für jedes Puzzleteil 3 Unbekannte existieren, hat der Vektor eine Größe von $\mathbf{1 \times 3MN}$. ${}_\mathbf{i}\mathbf{T_x}$ und ${}_\mathbf{i}\mathbf{T_y}$ sind Translationen in Richtung der $\mathbf{x}$- und $\mathbf{y}$-Achsen. ${}_\mathbf{i}\theta$ ist der Drehwinkel. Alle Werte beziehen sich auf Puzzleteil $\mathbf{i}$.

Die Anfangsschätzung ist besonders wichtig, insbesondere wenn die zu optimierende Funktion nicht kontinuierlich ist. Wie im folgenden Abschnitt besprochen (5.5.3), kann die Anfangsschätzung das Ergebnis eines vorherigen linearen Optimierers sein, auf dem dann weitere nichtlineare Optimierungen durchgeführt werden können.

Für jede unbekannte Variable sollten obere und untere Grenzen definiert werden. Diese Begrenzung stellt sicher, dass der Optimierer effizient arbeitet, indem er unnötige Bewegungen einschränkt und sich auf die Bereiche konzentriert, die für die Minimierung des Fehlers am relevantesten sind. Eine gut definierte Begrenzung kann die Anzahl der erforderlichen Iterationen reduzieren und somit die Gesamtzeit für die Optimierung verkürzen.

Die Berechnung der Grenzen hängt wesentlich vom Abstand der Ecke jedes Puzzleteils zu seinem Rand ab. In unserem Programm sind die Grenzen jedes Puzzleteils auf die Hälfte der Gesamtgröße des Puzzles festgelegt. Diese Konfiguration der Grenzen ist

so gestaltet, dass alle Puzzleteile sich in Richtung der oberen linken Seite des globalen Koordinatensystems bewegen können, indem den negativen Werten der unbekannten Variablen mehr Spielraum gegeben wird als den positiven.

In der Praxis bedeutet dies, dass für jede unbekannte Variable T_x, T_y (Translationen) und θ (Rotation) folgende Grenzen definiert werden:

- $T_x \in [-\frac{1}{2}\text{Puzzlegröße-x}, \frac{1}{10}\text{Puzzlegröße-x}]$
- $T_y \in [-\frac{1}{2}\text{Puzzlegröße-y}, \frac{1}{10}\text{Puzzlegröße-y}]$
- $\theta \in [-\frac{\pi}{4}, \frac{\pi}{4}]$

Diese asymmetrischen Grenzen ermöglichen eine größere Flexibilität in den negativen Richtungen, was die Bewegung der Puzzleteile in Richtung des Ursprungs fördert.

Es ist wichtig zu beachten, dass der Bereich der unbekannten Drehung θ sich von den Translationen unterscheidet, daher muss θ zuerst auf die gleiche Dimension wie die Translationen normiert werden. Diese Normierung stellt sicher, dass der Optimierer alle Unbekannten konsistent behandelt.

Als Nächstes wird die Fehlerfunktion zusammen mit dem Vektor der Grenzen und dem Vektor der ersten Anfangsschätzung an den Optimierer übergeben, um das Minimum der Fehlerfunktion zu erhalten.

Optimierungsmethode und Konfiguration Die gewählte Optimierungsmethode für dieses Problem ist L-BFGS-B (Limited-memory Broyden-Fletcher-Goldfarb-Shanno with Box constraints). Diese Methode ist besonders geeignet für Probleme mit großen Datenmengen und nichtlinearen Funktionen, wie es bei der Anordnung von Puzzleteilen der Fall ist. L-BFGS-B ist eine Erweiterung des BFGS-Algorithmus, die speziell für große Optimierungsprobleme entwickelt wurde. Der Algorithmus verwendet eine begrenzte Menge an Speicher (daher llimited-memory"), was ihn besonders effizient für Probleme mit vielen Variablen macht. Weitere Vorteile sind:

- **Speichereffizienz:** Durch die Verwendung von begrenztem Speicher ist L-BFGS-B effizienter als andere Optimierungsalgorithmen, die den gesamten Hessian (die zweite Ableitungsmatrix) speichern und manipulieren müssen.
- **Handhabung von Box-Beschränkungen:** L-BFGS-B kann direkt mit Box-Beschränkungen umgehen, d.h. es können einfache obere und untere Grenzen für die Variablen festgelegt werden, was für unser Problem der Puzzleanordnung ideal ist.
- **Skalierbarkeit:** Die Methode skaliert gut mit der Anzahl der Variablen und ist daher für große Optimierungsprobleme geeignet (für ein Puzzle mit $\mathbf{x}$ Teilen haben wir $\mathbf{3 \cdot x}$ unbekannte Variablen).

Für eine optimale Leistung der L-BFGS-B Methode müssen einige Konfigurationsparameter geeignet gewählt werden:

- **Startwerte (Anfangsschätzung):** Die Wahl guter Startwerte ist entscheidend. In unserem Fall können die Startwerte die Positionen und Drehungen der Puzzleteile aus einer vorherigen linearen Optimierung sein oder ganz zufällig ausgewählt werden.
- **Schrittweite:** Die Schrittweite, oder `stepsize`, ist ein wichtiger Parameter, der die Größe der Schritte beeinflusst, die der Optimierer bei der Suche nach dem Minimum unternimmt. Eine zu große Schrittweite kann dazu führen, dass der Optimierer über das Minimum hinaus schießt, während eine zu kleine Schrittweite die Konvergenz verlangsamen kann.
- **Grenzen:** Die oben definierten Grenzen für die Variablen T_x, T_y und θ helfen, die Suchregion des Optimierers einzugrenzen und die Konvergenz zu beschleunigen.
- **Toleranzen:** Die Parameter für Toleranzen, wie `gtol` (Gradienten-Toleranz) und `ftol` (Funktions-Toleranz), beeinflussen, wann der Optimierer die Suche stoppt. Diese sollten so gewählt werden, dass sie eine gute Balance zwischen Präzision und Rechenzeit bieten (Wenn die Puzzleteile voneinander entfernt sind, können wir mit einer großen Zahl beginnen, z. B. 10, und wenn sie sich einander nähern, können wir sie auf 0,1 reduzieren).
- **Maximale Iterationen:** Die maximale Anzahl an Iterationen, `maxiter`, sollte ausreichend groß gewählt werden, um die Konvergenz zu ermöglichen, aber nicht so groß, dass unnötig Rechenzeit verbraucht wird. Ein typischer Wert könnte bei 100-1000 Iterationen liegen.

Trotz ihrer vielen Vorteile hat die L-BFGS-B Methode auch einige Einschränkungen:

- **Lokales Optimum:** L-BFGS-B kann in lokale Optima geraten, insbesondere wenn die Startwerte weit von einem globalen Optimum entfernt sind. Daher ist die Wahl guter Startwerte entscheidend.
- **Nicht geeignet für diskrete Variablen:** L-BFGS-B ist für kontinuierliche Variablen konzipiert und nicht für Probleme mit diskreten Variablen geeignet. Zum Beispiel, wenn die Eckpositionen nicht korrekt erkannt werden oder andere Faktoren dazu beitragen, dass die Fehlerfunktion diskret ist, würde diese Methode keine optimale Ausgabe liefern.
- **Sensitivität gegenüber Parametereinstellungen:** Die Methode kann empfindlich auf die Einstellungen der Konfigurationsparameter reagieren. Falsche Einstellungen können die Konvergenz verlangsamen oder sogar verhindern.

Es kann vorkommen, dass die optimierten Werte in einem lokalen Minimum enden (siehe Abbildung 52). In einem solchen Fall gibt es zwei mögliche Ansätze, um dieses Problem zu lösen:

Zufälliges Rauschen hinzufügen: Der erste Ansatz besteht darin, dem Ergebnis des Optimierers ein kleines zufälliges Rauschen hinzuzufügen und die optimierten Werte

erneut in den Optimierer einzugeben. Dieses zufällige Rauschen kann dem Optimierer helfen, aus dem lokalen Minimum herauszukommen, indem es ihn dazu bringt, neue Bereiche des Lösungsraums zu erkunden. Der Nachteil dieser Methode ist jedoch, dass sie iterativ durchgeführt werden muss, um sicherzustellen, dass der Optimierer tatsächlich das globale Optimum gefunden hat. Diese Iterationen können zeitaufwendig sein.

Kombination von linearer und nichtlinearer Optimierung: Der zweite Ansatz ist im Unterunterabschnitt 5.5.3 diskutiert.

Abbildung 52: Optimuzer ist in einem lokalen Minima gelandet. ($\mathbf{F}_{\text{err}} = \mathbf{3.75e+3}$)

Ergebnisse In Abbildung 53 ist das Puzzle zu sehen, mit dem der Optimierer gestartet ist, und in Abbildung Abbildung 54 das Ergebnis. Der Wert der Fehlerfunktion liegt zu Beginn der Optimierung bei $\mathbf{F}_{\text{err}} = \mathbf{1.23e+5}$ und wird nach der Optimierung auf den Wert von $\mathbf{F}_{\text{err}} = \mathbf{6.62e+2}$ reduziert. Die Laufzeit des Optimierers hängt von den Startwerten, Konfiguration und auch von der Hardware ab, auf der er läuft. Auf meinem System dauerte jeder Durchlauf durchschnittlich über 90 Sekungden (mit dem gegebenen Puzzle), was relativ lang ist, aber es kann durch die Wahl besserer Startwerten verbessert werden (Siehe Ablauf des Optimierers in src/solver/algorithm/postprocessing/non-linear-optimal-layout.ipynb).

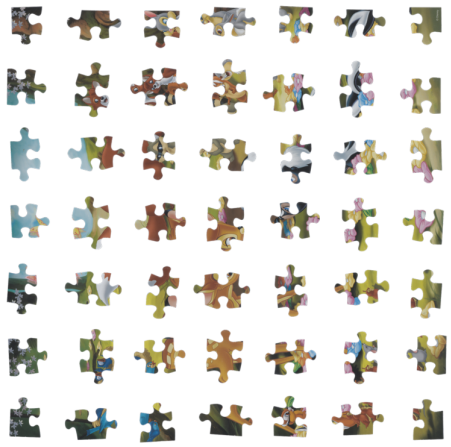


Abbildung 53: Layout des Puzzles vor der Optimierung ($\mathbf{F}_{\text{err}} = \mathbf{1.23e+5}$)

Abbildung 54: Layout des Puzzles nach der Optimierung ($\mathbf{F}_{\text{err}} = \mathbf{6.62e+2}$)

Fazit Die Fehlerfunktion wird durch die euklidischen Distanzen zwischen den benachbarten Ecken definiert. Der Optimier minimiert die Gesamtabstände zwischen benachbarten Ecken der Puzzleteile, indem er sowohl Translationen als auch Rotationen berücksichtigt. Die Auswahl geeigneter Startwerte, die Definition von Grenzen für die Variablen und die korrekte Konfiguration der Optimierungsparameter spielen eine entscheidende Rolle für die Effizienz und Genauigkeit der Methode. Wenn der Optimier in einem lokalen Minima bleibt, können Methoden wie das Hinzufügen von zufälligem Rauschen oder die Kombination von linearen und nichtlinearen Optimierungen angewendet werden.

5.5.3 Kombination von lineareren und nichtlineareren Layout Optimierer [Zamani Pozveh]

Motivation Lineare Optimierer sind in der Regel schneller und weniger rechenintensiv als nichtlineare Optimierer. Sie können schnell eine gute erste Annäherung an die optimale Lösung finden, indem sie die Variablen innerhalb eines linearen Modells optimieren.

Nichtlineare Optimierer können komplexe und nichtlineare Beziehungen besser modellieren. Allerdings kann es passieren, dass sie in lokalen Minima stecken bleiben, insbesondere wenn die Ausgangswerte weit vom globalen Optimum entfernt sind. Durch die Kombination beider Methoden werden die Geschwindigkeit und Effizienz der linearen Optimierung mit der Genauigkeit der nichtlinearen Optimierung vereint.

Im Kontext der Anordnung von Puzzleteilen kann der lineare Optimierer schnell eine grobe Anordnung basierend auf den Ecken der Puzzleteile finden. Der nichtlineare Optimierer verfeinert dann die Positionen und Rotationen, um die minimale Gesamtdistanz zwischen den benachbarten Ecken zu erreichen. Diese zweistufige Optimierungsmethode stellt sicher, dass die Lösung sowohl effizient als auch präzise ist.

Verfahren Die Ergebnisse der linearen Optimierung werden als Startwerte für den nichtlinearen Optimierer verwendet. Die Grenzen und Startwerte werden so konfiguriert, dass der nichtlineare Optimierer innerhalb der durch den linearen Optimierer definierten Suchregion arbeitet.

Ergebnisse In Abbildung 55 ist die Puzzle-Ausgabe des linearen Optimierers zu sehen, die als Startwerte für den nichtlinearen Optimierer gegeben sind. In Abbildung Abbildung 56 ist das verfeinerte Puzzle zu sehen. Der Wert der Fehlerfunktion wurde von $\mathbf{F}_{\text{err}} = \mathbf{6.81 + e2}$ auf $\mathbf{F}_{\text{err}} = \mathbf{6.61e + 2}$ reduziert. Der nichtlineare Optimierer hat die Drehung und Position einiger Puzzleteile verbessert(Siehe Abbildung 57).

Die Laufzeit des Optimierers hängt von den Startwerten, Konfiguration und auch von der Hardware ab, auf der er läuft. Auf meinem System dauerte jeder Durchlauf durchschnittlich unter 15 Sekungden (mit dem gegebenen Puzzle und Ausgabe des linearen Optimierers als Startwerte für den nichtlinearen Optimierer), was akzeptabel ist. Der kann auch durch eine Verbesserung der Konfiguration (nämlich `stepsize`) reduziert werden (Siehe Ablauf des Verfeinerung in src/solver/algorithm/postprocessing/fine-tunning.ipynb).

Abbildung 55: Layout des Puzzles nach linearer Optimierung ($\mathbf{F}_{\text{err}} = \mathbf{6.81 + e2}$)

Abbildung 56: Layout des linear optimierten Puzzles nach der Verfeinerung mit dem nichtlinearen Optimierer ($\mathbf{F}_{\text{err}} = \mathbf{6.62 + e2}$)

Abbildung 57: Bildunterschied zwischen dem Layout des Puzzles vor und nach der Verfeinerung. Die Puzzleteile, die während des Prozesses verändert wurden, sind rosa gefärbt.

Fazit Durch Nutzung eines linearen Optimierers, um eine initiale Annäherung an die optimale Lösung zu finden, wird die Wahrscheinlichkeit reduziert, dass der nichtlineare Optimierer in einem lokalen Minimum stecken bleibt. Diese anfängliche Schätzung liegt näher am globalen Optimum, was dem nichtlinearen Optimierer ermöglicht, schneller und effizienter zu konvergieren. Insgesamt führt diese zweistufige Methode zu einer schneller(als nichtlinearen Optimierer alleine) und präziseren Lösung, da die Vorteile beider Optimierungsansätze kombiniert werden.

5.6 ROS2 Integration [Sami Hamza]

5.6.1 Aufgaben

Die wesentliche Aufgabe der ROS2 Implementierung ist das Empfangen der von Team Digitalize segmentierten Bilder der Puzzleteile. Anschließend muss das Puzzle anhand der empfangen Bilder gelöst werden. Danach soll das Puzzle im Visualizer aufgebaut werden. Hierfür muss für je ein Puzzleteil die Übergabeposition für den Roboter und die Zielposition im Robotersystem berechnet werden. Die Start und Endpositionen werden dann über das Topic "solver/nextpiece" an die Control-Node versendet. Zusätzlich muss die Visualizer-Node über das Topic "solver/startpos" die Startposition des aktuellen Teils erhalten. Wird das Puzzleteil abgelegt, so erhält die Solver-Node diese Information über das Topic "control/puzzlebool" und soll die Informationen für das nächste Teil versenden.

5.6.2 Schnittstellen

Abbildung 61 gibt eine Übersicht über die verwendeten Schnittstellen aus Sicht der Solver-Node.

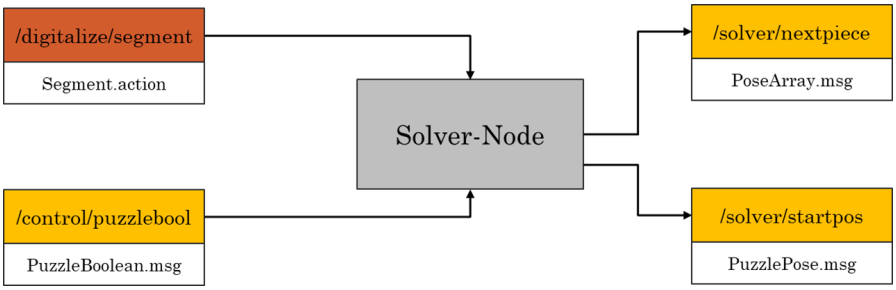


Abbildung 58: ROS2-Schnittstellen Übersicht.

Solver - Digitalize Hier wird als Art der Kommunikation die von ROS2 bereitgestellte Action-Server Schnittstelle verwendet. Der Server kann über das Topic "/digitalize/segment" erreicht werden. Diese erlaubt es Clients (hier die Solver-Node) über eine Goal-Anfrage einen den Mechanismus in der Server-Node (hier die Digitalize-Node) zu aktivieren. Rückmeldung zum Status der Abfrage kann Client-seitig über die *goal_ response_ callback* verarbeitet werden. Wird die Anfrage angenommen, führt der Server den Benutzer definierten Ablauf durch und sendet währenddessen Feedback-Nachrichten, welche hier von der *feedback_ callback* abgefangen werden können und für den Bildtransport verwendet werden. Abschließend wird eine Result-Nachricht versendet, welche von der *get_ result_ callback* abgefangen wird und dem Client das Ende des Ablaufs indiziert. Diese Struktur ist ideal für den Transport der segmentierten Bilder, da diese sicherstellt, dass beide Parteien für die Kommunikation bereit sind. Außerdem muss die Solver-Node keine Kenntnis darüber haben, wie viele Bilder gesendet werden, da das Ende der Übertragung eindeutig gekennzeichnet wird.

Solver - Visualize Über diese Schnittstelle teilt die Solver-Nodel der Visualizer-Node die Startposition (Spawn-Punkt) der Puzzleteile mit. Es handelt sich hierbei Solver-seitig um einen einfachen Publisher, der Nachrichten vom eigens definiertem Typen *PuzzlePose* auf das Topic "solver/startpos" published. Listing 1 zeigt die Definition der versendeten Nachricht. Diese beinhaltet die jeweilige Piece-ID des Puzzleteils und die Posen Information, welche durch eine Position als *Point* und eine Orientierung als *Quaternion* im Robotersystem (Einheit Meter) definiert ist.

Listing 1: Nachrichten Definition für PuzzlePose.msg

```
# id of the puzzle piece
int32 id
# pose of the puzzle piece (with x- and y-axis aligned
# with image x- and y-axis) in robot base frame
geometry_msgs/Pose pose
```

Solver - Control Die Solver-Node übermittelt der Control-Node mit dieser Schnittstelle die Startposition (Spawn-Punkt) und die Zielposition der Puzzleteile. Es handelt sich hierbei Solver-seitig wieder um einen einfachen Publisher, der Nachrichten des in geometry_msgs von ROS vordefinierten Nachrichtentyps *PoseArray* auf das Topic "solver/startpos" published. Diese Nachricht enthält ein Array aus ROS-Poses, welche durch eine Position und eine Orientierung als Quaternion im Robotersystem (Einheit Meter) definiert sind.

5.6.3 Node - Ablauf

Um die gesamten Aufgaben zu erfüllen wird im Rahmen des Projektes ein monolithischer Ansatz gewählt. Dieser bietet sich an, da alle Aufgaben sukzessiv gelöst werden können und damit sehr schön durch Callback-Funktionen realisiert werden kann. In Abbildung 62 ist der sequenzielle Ablauf der Solver Node-Callbacks dargestellt.

1. Der 'main-Thread" main-Node initiert den Ablauf über die *send_segment_goal*-Funktion und dient als warte-Thread falls grade keine Callback aktiv ist. Sie sendet zu Beginn eine Anfrage an den Digitalize-Action-Server (Event E_1).

2. Die Antwort der Action-Server-Goal-Anfrage wird empfangen (Event E_2) und anschließend in der *goal_response_callback*-Funktion verarbeitet.

3. Dannach wird auf Feedback-Nachrichten, die die Bilder enthalten, gewartet. Wird eine dieser Nachrichten empfangen (Event E_3), so wird das Bild ausgelesen und unter dem Namen "image_<pid>_.png" abgespeichert. Dabei ist pid, die mit dem Image versandte Piece-ID.

4. Wurden alle Bilder gesendet, sendet der Server die Result-Nachricht und die *get_result_callback*-Funktion wird aufgerufen. In dieser wird das Puzzle gelöst. In Folge dessen wird das erste Teil aus der Solver Historie gezogen und dafür die Start- und Endposition

berechnet. Hierbei muss die Konversion zwischen Posen in Pixelkoordinaten, wie sie im gelösten Puzzle verwendet werden, und Metern ((Faktor 0,00004 mm/pixel)) für das Control-Team bzw. Milimetern (Faktor 0,04 mm/pixel) für das Visualizer-Team beachtet werden. Die korrespondierenden Posen werden in die Topics "solver/nextpiece" und "solver/startpos" gepublished.

5. Sobald der Solver eine PuzzleBool-Message auf dem Topic "control/puzzlebool" erhält (Event E_4) wird in der *puzzle_ bool_ callback*-Funktion überprüft ob ein Wechsel von True auf False stattgefunden hat, was dem Ablegen des Teils an der Zielposition entspricht. Ist dies der Fall, so wird das nächste Teil aus der Solver Historie gezogen, die Posen berechnet und die Nachrichten gepublished (Analog zur *get_ result_ callback*-Funktion). Wurden bereits alle Teile aus der Solver-Historie gezogen, so wird die Node nach erneuten Aufrufen der *puzzle_ bool_ callback*-Funktion beendet.

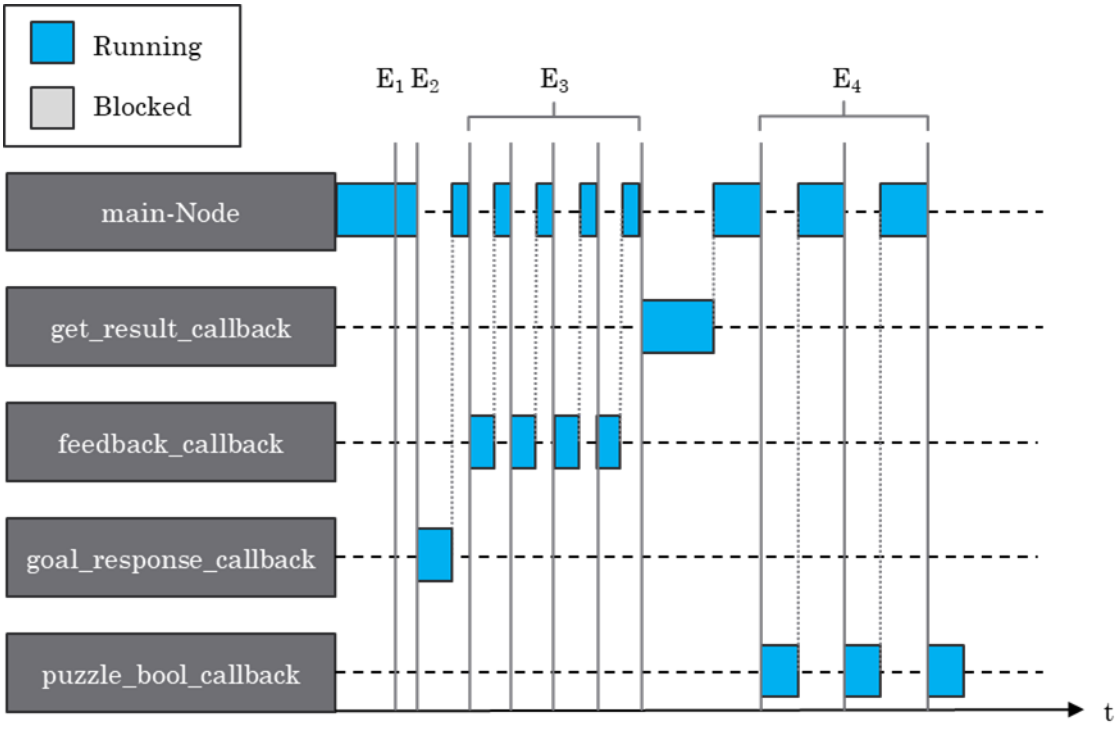


Abbildung 59: Sequenzbeispiel zur Veranschaulichung der Callback-Interaktion.

5.6.4 Schnittstellentests

Im Rahmen des Projekts wurden jeweils in Zusammenarbeit mit den anderen Teams Integrationstests durchgeführt. Hierfür wurden Teilfunktionalitäten manuell durch Starten der betroffenen Nodes über ROS-Launchfiles auf Richtigkeit überprüft.

5.6.5 Fazit

Im Allgemeinen lässt sich sagen, dass die Funktionalität gut gewährleistet ist. Allerdings könnte der entwickelte Code durch Refaktoring noch besser strukturiert werden. Auch die gewählten Schnittstellen weisen noch viel Verbesserungspotential auf. So könnte man anstatt der für den Action-Server verwendeten *sensor_ msgs/Image* Feedback-Nachricht eine *sensor_ msgs/CompressedImage* Feedback-Nachricht verwenden, um effizienter zu Transportieren. Ein weiterer Problematischer Aspekt ist, dass die Feedbacknachrichten nicht zu schnell aufeinander folgen dürfen, da sonst Bilder Solver-seitig verloren gehen. Das Problem kann Digitalize-seitig durch eine ausreichend lange Verzögerung gelöst werden. Der Begriff ausreichend ist aber start Abhäng von den zu Verfügung stehenden System-Resourcen. Auch Redundanzen bezüglich der Startposition könnten reduziert werden, indem sowohl der Visualizer, als auch die Control Node auf das "solver/nextpiece" Topic subscriben würden. Denn in dieser Nachricht ist die Startposition ja ebenfalls enthalten. Auch unnötige Konvertierungen der Einheiten könnten durch einheitlich Konventionen gespart werden.

6 Control

6.1 Roboterübersicht

Zu Beginn des Projektes arbeitete sich das gesamte Team in die Thematik der Robotik ein. Hierzu gehörten das Verstehen der zu verwendeten Toolbox und das eigenhändige aufstellen von DH Parametern nach modifizierten Standart.

Implementiert wurde ein KUKA-KR6, dargestellt durch die folgenden modifizierten DH-Parameter.

Tabelle 5: modifizierte DH-Parameter

Link	**a**	α	**d**	θ
1	0	π	-0,675	0
2	0,3	$\frac{\pi}{2}$	0	0
3	0,65	0	0	$-\frac{\pi}{2}$
4	0,155	$\frac{\pi}{2}$	-0,6	0
5	0	$-\frac{\pi}{2}$	0	0
6	0	$-\frac{\pi}{2}$	0,1	0

6.2 Organisation **[Pascal Severin]**

Zur besseren Organisation des Projekt wurde in Meilensteinen gearbeitet. Im zweiwöchigen Turnus wurden neue Aufgaben in form von Issues erstellt. Jedes Issue wurde nach T-shirt Größen (XS/1, S/2, M/3, L/4, XL/5) gewichtet und einem Meilenstein zugeordnet.

Jede:r Entwickler:in arbeitete in einen eigenem Branch, die über einen Merge-Request in main gemergt wurde. Hierfür wurden jeweils Code-Reviews gemacht.

6.3 Architektur der Control-Node

Zur Implementierung der Roboterfunktionen wurde die folgende Klasse und Funktionen umgesetzt:

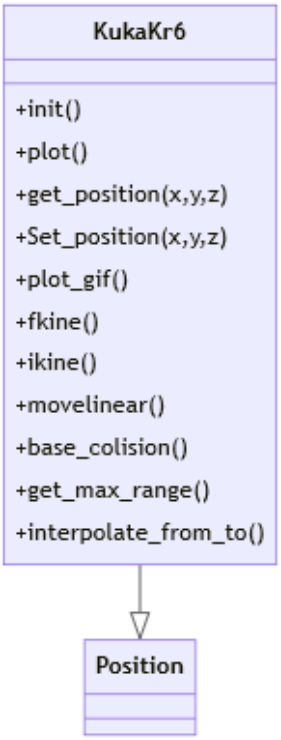


Abbildung 60: Klassendiagramm der Control Node.

- **class KukaKr6**
  - `init`: class initialization
  - `plot`: plots the robot using matplotlib as a static image
  - `get_position`: returns the current position (x, y, z) of the end-effector
  - `set_position`: sets the current position (x, y, z) of the end-effector
  - `plot_gif`: plots the robot's movement using matplotlib as a gif
  - `fkine`: forward kinematics for the robot (using the toolbox implementation)
  - `ikine`: inverse kinematics (using Newton-Raphson)
  - `move_linear`: moves the robot in a straight line from start to end pose
  - `base_collision`: Kollision mit Basis

  - `check_reachability_at_base`: checks whether the base of the robot is reachable
  - `get_max_range`:Range-Calculation
  - `interpolate_from_to`: a wrapper function for multiple movement algorithms that can be chosen via the function's parameters
    * `int_pos_linear`: linear interpolation of the position
    * `int_pos_casteliau`: interpolation using De Casteljau
    * `int_rot_slerpQuad`: interpolate rotation using SLERP
  - `calculate_joints`: calculates joint configurations in radians

- **class Position**: describes x, y, z positioning

6.4 ROS2 Integration [Sami Hamza, Pascal Severin, Chiara Piccolroaz]

6.4.1 Aufgaben

Die wesentliche Aufgabe der ROS2-Implementierung ist das Empfangen der Nextpiece-Message auf dem Topic "solver/nextpiece", welches die anzusteuernde Pickup- und Endpositionen beinhaltet. Aus diesen Positionen soll dann eine Trajektorie berechnet werden und die ermittelten Gelenkkonfigurationen an die Visualizer-Node über das Topic "control/angles" geschickt werden. Zudem muss die Control-Node die Solver-Node über das Platzieren eines Puzzleteils informieren, damit diese die nächsten Start- und Endpositionen bereitstellen kann. Dafür soll das Topic "solver/puzzlebool" verwendet werden.

Schnittstellen Abbildung 61 gibt eine über die verwendeten Schnittstellen aus Sicht der Control-Node.

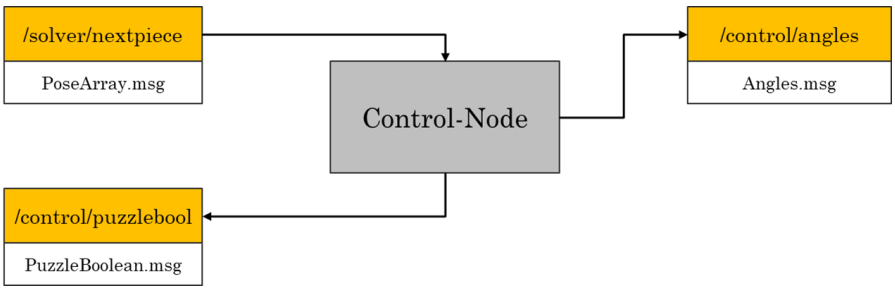


Abbildung 61: ROS2-Schnittstellen Übersicht.

Control - Solver Die Solver-Node übermittelt der Control-Node mit dieser Schnittstelle die Pickup- (Spawn-Punkt) und die Zielposition der Puzzleteile. Es handelt sich hierbei Control-seitig um einen einfachen Subscriber, der Nachrichten des in geometry_msgs von ROS vordefinierten Nachrichtentyps *PoseArray* vom Topic "solver/startpos" subscribed. Diese Nachricht enthält ein Array aus ROS-Poses, welche durch eine Position und eine Orientierung als Quaternion im Robotersystem (Einheit Meter) definiert sind.

Control - Visualize Hierbei handelt es sich um einen Publisher auf das Topic "control/angles", bei dem die Control-Node die Gelenkkonfigurationen, sowie die Information, ob ein Teil gegriffen wird published. Dafür wird die eigens definierte *Angles*-Nachricht verwendet (Nachrichtendefinition siehe Listing 2). Die Gelenkkonfiguration wird als *std_msgs/Float64MultiArra* übertragen. Die Information, ob ein Puzzleteil gegriffen wird, wird über den Boolean *picked* übermittelt. Die Piece-ID des Teils steckt in dem 32-Bit Integer *id*, jedoch wird von diesem Anteil kein Gebrauch gemacht.

Listing 2: Nachrichten Definition für die Nachrit Angles

```
# angle data
std_msgs/Float64MultiArray angles
# boolean to indicate if puzzle piece is picked up or not
bool picked
# id of the puzzle piece
int32 id
```

Um dem Solver zu signalisieren, dass das Puzzleteil platziert wurden und das nächste Teil gesendet werde kann, wird ein Publisher auf das Topic "control/puzzlebool" registriert. Dieser published eine *PuzzleBoolean*-Nachricht, sobald sich der Greifstatus ändert (Nachrichtendefinition siehe Listing 3). Der Greifstatus wird über den Boolean *data* transportiert. Der string *id*, der die Piece-ID des aktuell transportierten Teils beinhaltet, wird hier ebenfalls nicht verwendet.

Listing 3: Nachrichten Definition für die Nachricht PuzzleBoolean

```
# toggle-data for a puzzle piece
bool data
# id of the specific piece
string id
```

6.4.2 Node - Ablauf

Um die gesamten Aufgaben zu erfüllen, wird im Rahmen des Projektes ein monolithischer Ansatz gewählt. Dieser bietet sich an, da die Aufgaben sehr gut auf Callback-Funktionen einer ROS-Node aufgeteilt werden können. In Abbildung 62 ist der sequenzielle Ablauf der Control Node-Callbacks dargestellt.

1. Der 'main-Thread" main-Node dient als warte-Thread falls grade keine Callback aktiv ist.

2. Falls eine Nextpiece-Message empfangen wird (Event E_1), wird in die *nextpiece_callback*-Funktion gesprungen. In dieser werden einzelne Trajektorienabschnitte berechnet. Diese folgen stets dem Schema: Start- zu Pickup-Position; Pickup- zu Startposition; Start- zu Zielposition; Ziel- zu Startposition. Nach jeder Berechnung der Teiltrajektorien wird der ein Timer aktiviert (Event E_2).

3. Wurde der Timer aktiviert, wird bei jedem Timerevent in die *visualize_ timer_ callback*-Funktion gesprungen und die Gelenkkonfigurationen auf das Topic "control/angles" gepublished. Die *nextpiece_ callback*-Funktion ist so lange blockiert, bis die Trajektorie versendet wurde (Event E_3).

4. Ist die Trajektorie versendet, wird die sie als kritische Ressource wieder freigegeben und die *nextpiece_ callback*-Funktion fortgesetzt. Wurde die Pickup- bzw. Zielposition erreicht wird zusätzlich noch in das Topic "control/puzzlebool" gepublished.

Anmerkung: In ROS müssen die Eigenschaften und Rechte von Callbacks über sogenannte Callbackgroups geregelt werden. Da in diesem Fall die *visualize_ timer_ callback*-Funktion die *nextpiece_ callback*-Funktion unterbricht, müssen sich diese in unterschiedlichen Callback-Gruppen befinden. Hier werden die Callbacks zwei Mutually Exclusive Callback Groups zugeornet. Damit haben diese die Berechtigung sich gegenseitig zu unterbrechen.

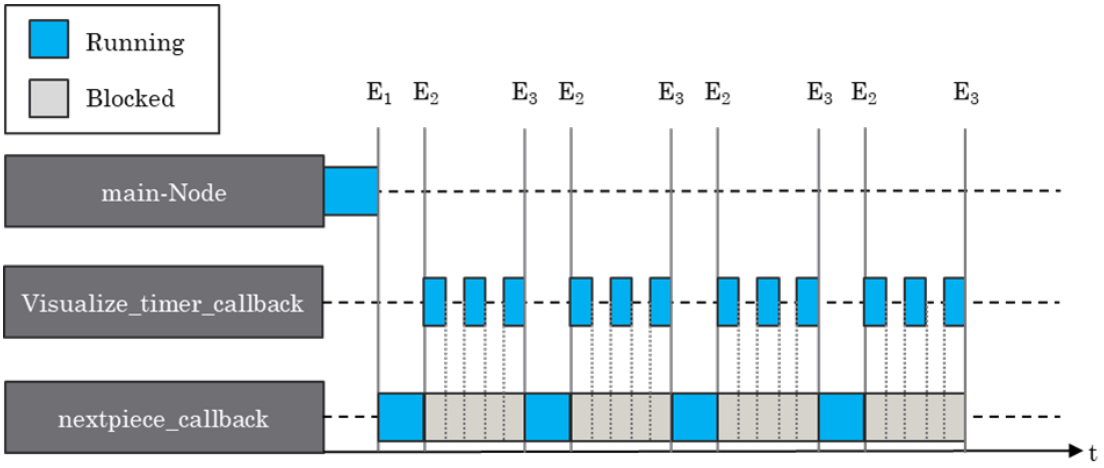


Abbildung 62: Sequenzbeispiel zur Veranschaulichung der Callback-Interaktion.

6.4.3 Schnittstellentests

Im Rahmen des Projekts wurden jeweils in Zusammenarbeit mit den anderen Teams Integrationstests durchgeführt. Hierfür wurden Teilfunktionalitäten manuell durch Starten der betroffenen Nodes bzw. der eingeführten Tester-Nodes über ROS-Launchfiles auf Richtigkeit überprüft.

6.4.4 Fazit

Die Funktionalität im ROS-System ist gewährleistet. Allerdings könnten hier die Nachrichtendefinitionen überarbeitet werden, um beispielsweise unverwendete Nachrichtenfelder zu verwerfen.

6.5 Testing [Simon Mederer]

Um unabhängig von den anderen Teams das Verhalten der Control-Node testen zu können, wurde ein Testing Framework entwickelt, in dem sowohl für die Solver-, als auch für die Visualisierungs Node ein sogenanntes Mockup erstellt wurde. Diese sollen das Verhalten der jeweiligen Node simulieren um somit das Verhalten der Control Node im finalen System besser einschätzen und testen zu können. Abbildung 63 visualisiert hier wie die Mockups in die Architetkur eingebunden werden. Diese subscriben bzw. publishen auf die Topics ihrer jeweiligen Originale um so ihre Interaktion mit der Control Node simulieren zu können.

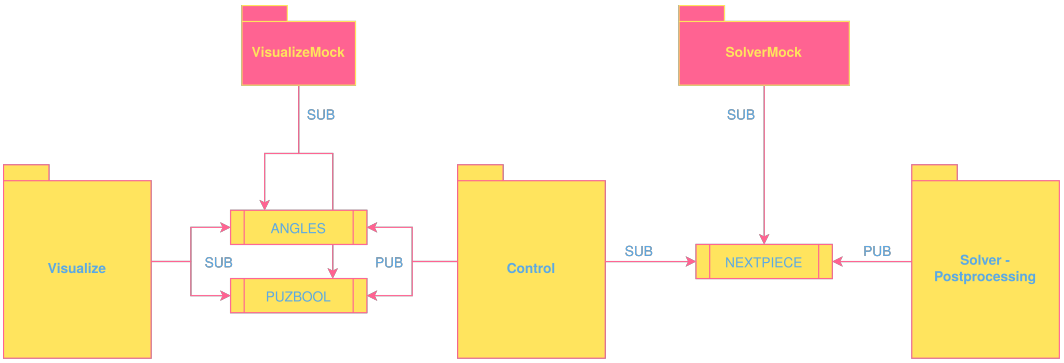


Abbildung 63: Test Node Architektur

Zusätzlich wurde eine GitLab CI/CD erstellt um automatisch Unit-Tests in der Control Node ausführen zu können.

6.5.1 Test Package

Die Test Node besteht aus zwei Mockups. Dem *SolverMock* und dem *VisualizeMock*, welche jeweils in einer eigenen ROS-Node realisiert wurden.

SolverMock Die Hauptaufgabe, des SolverMocks ist es der Control Node Strecken zu publishen, welche durch Start- und Endpunkt definiert sind. Diese werden in einer eigenen Funktion *generate_pose_array_message* erstellt. Dabei wird in erster Iteration folgende Strecke erstellt.

$$startPos: \begin{cases} position: \begin{cases} x = 0.5 \\ y = 0.5 \\ z = 0 \end{cases} \\ orientation: \begin{cases} x = cos(\pi) \\ y = 0 \\ z = 0 \\ w = sin(\pi) \end{cases} \end{cases} \Rightarrow endPos: \begin{cases} position: \begin{cases} x = -0.5 \\ y = -0.5 \\ z = 0 \end{cases} \\ orientation: \begin{cases} x = cos(\pi) \\ y = 0 \\ z = 0 \\ w = sin(\pi) \end{cases} \end{cases}$$

Diese beiden Punkte werden in die PoseArray Nachricht gepackt und an das *NEXT-PIECE* Topic gepublished. Auf dieses subscribed wie im normalen Betrieb die Control Node und kann aus den jeweiligen Positionen die Trajektorie berechnen.

VisualizeMock Um die berechneten Daten zu visualisieren bzw. zu validieren ist eine zweite Node notwendig, welche die Daten, die von der Contol Node versendet werden evaluiert. Wie in Abbildung 63 dargestellt subscribed der *VisualizeMock* auf die beiden Topics *ANGLES* und *PUZBOOL*. Über diese beiden Topics kann alles, was die Control Node ausgibt durch die *VisualizeMock* Node empfangen und ausgewertet werden. Dies soll durch UnitTests realisiert werden.

6.5.2 Unit Tests

Unit Tests sind Tests, mit denen normalerweise einzelne Programmkomponenten getestet werden. Die beiden implementierten Komponenten *SolverMock* und *VisualizeMock* erlauben es jedoch das gesamte Verhalten der Contol Node zu testen. Hierfür kann das von ROS zur Verfügung gestellte Unit Testing Framework, genauer GoogleTest (gtest) genutzt werden. Gtest ist ein assertion basiertes, plattformunabhägiges C++ Testing Framework, mithilfe dessen einzelne Komponenten einer Software getestet werden können.

In erster Iteration war geplant, dass die beiden Mockups auf den Watchdog der Control Node subscriben um eine simultane Ausführung beider Knoten zu gewährleisten. Dies wurde dadurch realisiert, dass beide einem Executor übergeben wurden, welcher in einem separaten Thread gestarted wird. Wenn nun die Watchdog Nachricht der Contol Node empfangen wird, kann dies nun über das Testing Framework ermittelt werden. Der erste Test wird als erfolgreich angesehen, wenn beide Knoten die Watchdog Nachricht der Control Node empfangen haben.

Ein weiterer Test, der jedoch aus Zeitgründen nicht mehr implementiert wurde, sollte die konkreten Nachrichten der Control Node validieren. Dabei sollte eine Nachricht an die Control Node geschickt werden, und daraufhin geprüft werden, ob die Control Node ihre Nachrichten in der richtigen Reihenfolge ausgibt, sprich:

1. Angles bis die Startposition erricht ist
2. Puzzle-Boolean auf **true** mit korrekter ID
3. Angles bis die Endposition erreicht ist
4. Puzzle-Boolean auf **false** mit korrekter ID

Dieser Test wurde jedoch aus Zeitgründen nicht implementiert.

6.5.3 CI/CD

Um die oben beschriebenen Unit-Tests mit dem GitLab Repository verbinden zu können sollte eine CI/CD-Pipeline erstellt werden, die automatisch die geschriebenen Unit-Tests ausführt und dem jeweiligen Entwickler Feedback darüber gibt, ob die grundsätzliche

Funktionalität der Control Node nach dem Pushen seiner Änderungen noch gegeben ist. Dafür bietet GitLab eine komfortable Umgebung an innerhalb derer die CI/CD erstellt werden kann.

Als Runner wird der bereits für das Docker-Image erstellte Runner verwendet.

Folgende Schritte sollten in der Pipeline ausgeführt werden.

1. Clonen des Repositorys
2. Installieren der notwendigen Dependencies (Roboricstoolbox, Numpy)
3. Initialisieren der Subrepositories
4. Bauen des Repositories
5. Ausführen der Tests

Im Laufe der Enwticklung stellten sich der 5. Schritt als größte Herausforderung dar, da trotz korrekter Initialisierung und doppelter Validierung der korrekten Versionen das Importieren von Modulen in einer Python-Library fehlschlägt, weshalb dieser aus Zeitgründen weggelassen wurde. Folglich wird durch die Pipeline lediglich abgedeckt, ob das Repository nach dem Pushen von Änderungen noch gebaut werden kann oder nicht.

6.5.4 Schnittstellen [Sami Hamza, Chiara Piccolroaz, Pascal Severin]

Zum Testen der Schnittstellen der Control-Node wurden ROS-launch files angelegt. Diese ermöglichen ein händisches Testen einzelner Komponenten.

6.6 Inverse Kinematik [Sami Hamza, Pascal Severin]

Das Ziel der Inversen Kinematik ist es die Emittlung der Abbildung IK einer gewünschten End-Effektor Pose $T_e \in SE3$ auf die Gelenkkonfiguration q eines Roboters mit n-DoF.

$$\begin{aligned} IK &: SE3 \to \mathbb{R}^n \\ IK &: T_e \mapsto q \end{aligned} \tag{20}$$

Dies entspricht der inversen Abbildung der Vorwärtskinematik FK, welche die Gelenkkonfiguration auf die Gelenkkonfiguration q auf die Endeffektorpose T_e abbildet. Die Pose ergibt sich hier durch Einsetzen der Konfiguration in die aus den DH-Parametern hergeleitete, parametrisierte Transformation.

$$\begin{aligned} FK &: \mathbb{R}^n \to SE3 \\ FK &: q \mapsto T_e \end{aligned} \tag{21}$$

Es wurden diese zwei Arten der inversen Kinematik implementiert:

Numerische Inverse Kinematik Die numerische Kinematik versucht das inverse Kinematik Problem durch numerische Optimierungsverfahren zu lösen, anstatt analytische Lösungen zu berechnen. Numerische Optimierung erweist sich zwar als weniger performant, erlaubt jedoch die Berücksichtigung von Gelenkwinkellimitierungen, maximierung der Manipulativität und weiterer Randbedingungnen. Zudem wird der Lösungsraum implizit eingegrenzt, da i.d.R. das Optimum ausgehend von einer Startkonfiguration q_0 bzw. Start-End-Effektor Pose T_ep gefunden wird.

Im Allgemeinen versucht die numerische inverse Kinematik eine Fehlerfunktion $E \in \mathbb{R}_+$ zu minimieren. Diese wird berechnet durch:

$$E = \frac{1}{2} e^T W e \tag{22}$$

wobei $e \in \mathbb{R}^6$ ein Maß für die Abweichung der Pose T_e von der Zielpose T_ep darstellt und W ein diagonal Matirx zur Gewichtung dieses Fehlers darstellt $W = diag(w), w \in \mathbb{R}^6$.

$$e = \begin{bmatrix} \tau(T_{ep}) - \tau(T_e) \\ \alpha(\rho(T_{ep})\rho(T_e^T)) \end{bmatrix} \tag{23}$$

Hierbei extrahiert τ den Positionsvektor $t \in \mathbb{R}^3$ und ρ die Orientierung als Rotationsmatirx $R \in SO3$ aus homogenen Matrizen. Zusätzlich wird die resultierende Fehlerrotation $\rho(T_{ep})\rho(T_e^T)$ durch die Abbildung $\alpha : R \in SO3 \mapsto v\mathbb{R}^3$ als Eulervector parametrisiert. Die Definition von α kann [13] entnommen werden.

Zur Optimierung der Fehlerfunktion stehen verschiedene Verfahren zu Verfügung. Im Projekt wurde die Newton-Raphson-Methode implementiert, da es sich hierbei um eine einfache und anschauliche Methode handelt. Die Newton-Raphson-Methode versucht eine Gelenkkonfiguration so lange zu aktualisieren bis die Fehlerfunktion E kleiner als ein geggebener Schwellenwert ϵ_E ist. Die Update-Gleichung lässt sich wie folgt formulieren

$$q_{k+1} = q_k + J(q_k)^{-1} e_k \tag{24}$$

$J \in \mathbb{R}^{6xn}$ beschreibt hierbei die Jakobimatrix bei gegebener Konfiguration q_k. Es ist üblich die Pseudo-Inverse J^+ der Jakobimatrix zu verwenden, da diese effizient über die Singular Value Decomposition (SVD) berechnet werden kann. Außerdem ermöglicht diese die Betrachtung von redundanten Robotern (DoF > 6) und gibt den Update-Schritt zurück, der mit am wenigsten Gelenkaufwand realisiert werden kann. Auch im Falle von Singularitäten wird der bestmögliche Update-Schritt berechnet.

$$q_{k+1} = q_k + J(q_k)^{+} e_k \tag{25}$$

Diese Methode ist einfach zu implementieren, aber nicht die robusteste Lösung. Sie neigt dazu für ungünstige Startkonfigurationen nicht zu konvergieren. Robustheit kann durch einen Wechsel zu komplexeren, numerischen Methoden wie der Gaus-Newton- oder der Levenberg-Marquard-Methode, die dieses Problem berücksichtigen, hinzugefügt werden [13][14][15]. Allerdings erweist sich die genannte Methode im Rahmen dieses Projek-

tes als ausreichend für die Lösungsfindung. Es sei jedoch angemerkt, dass keine Gelenkwinkel und Kollisionschecks durchgeführt werden, weswegen die implementierte Funktion nur in einer Simulation verwendet werden kann.

Analytische Inverse Kinematik Die grafische (oder analytische) Inverse Kinematik ist in zwei Unteraufgaben aufgeteilt:

- Inveres Positionierungsproblem
- Inverses Orientierungsproblem

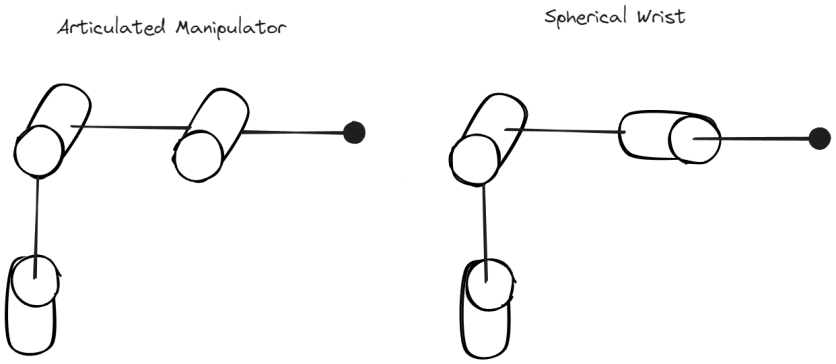


Abbildung 64: Manipulatoren des KUKA-KR6 Roboters

Um die inverse Positionierung eines 6DoF-Manipulators[1] zu berechnen, muss folgendes vorrausgesetzt werden. Der Roboter lässt sich in jeweils 3-Achsen Manipulatoren teilen, wobei die vorderen drei Gelenke alleinig die Position des Endeffektors bestimmen und die restlichen drei seine Orientierung. Diese Manipulatoren sind in 64 aufgezeigt. Die Längen der Glieder des *Sphärichen Gelenkes (spherical wrist)* sollten für diese Berechnung möglichst klein sein.

Der erste Schritt besteht in der Berechnung der Position unseres *Spärischen Gelenkes*, positioniert in dem vierten Gelenk des Roboters. Da wir, wie in den DH-Parametern in 5 beschrieben. Da die Distanz zwischen den vierten und fünften Glied unseres Roboters nicht null ist müssen wir diese als einen Offset in Z-Richtung berücksichtigen:

$$P_{wrist} = P_{end} - offset \cdot \vec{direction} = P_{end} - 0.155 \cdot \vec{Z_{end}} \tag{26}$$

Wir beginnen damit die ersten drei Winkel zu berechnen. Hierfür wird der Roboter schmatisch von der Seite und von oben aufgezeichnet, wie in 65 ersichtlich. Unter Verwendung von verschiedenen geometrischen Rechensätzen (wie Sinus- und Kosinussätze, Satz des Pythagoras, etc.) erhalten wir diese Ergebnisse.

[1] *6 degree of motion* bezieht sich auf die Freiheitsgrade eines Roboters. Besitzt dieser (wie hier vorhanden) 6 Gelenkachsen, so hat er insegsamt 6 Freiheitsgrade

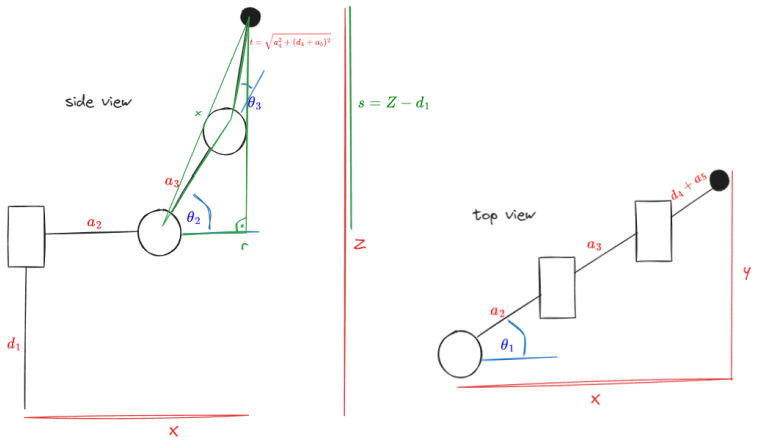


Abbildung 65: schematische Darstellungen

$$\theta_1 = \tan^{-1}\left(\frac{Y}{X}\right) \theta_2 = \cos^{-1}\left(\frac{s^2 - r^2x^2}{2rx}\right) - \cos^{-1}\left(\frac{t^2 - a_3^2x^2}{2a_3x}\right) \theta_3 = 180 - \cos^{-1}\left(\frac{x^2 - t^2a_3^2}{2ta_3}\right) \tag{27}$$

Zur Berechnung der Inversen Orientierung betrachten wir die Orientierungsmatrizen der letzten drei Gelenke:

$$R_6^3 = \begin{bmatrix} \cos(q4)\cdot\cos(q5)\cdot\cos(q6) - \sin(q4)\cdot\sin(q6) & -\cos(q6)\cdot\sin(q4) - \cos(q4)\cdot\cos(q5)\cdot\sin(q6) & -\cos(q4)\cdot\sin(q5) \\ \cos(q4)\cdot\sin(q6) + \cos(q5)\cdot\cos(q6)\cdot\sin(q4) & \cos(q4)\cdot\cos(q6) - \cos(q5)\cdot\sin(q4)\cdot\sin(q6) & -\sin(q4)\cdot\sin(q5) \\ \cos(q6)\cdot\sin(q5) & -\sin(q5)\cdot\sin(q6) & \cos(q5) \end{bmatrix} \tag{28}$$

Mit dieser Matrix und der gewünschten End-Rotierung können die letzten drei Winkel berechnet werden:

$$\theta_1 = \tan^{-1}\left(\frac{Y}{X}\right) \theta_2 = \cos^{-1}\left(\frac{s^2 - r^2x^2}{2rx}\right) - \cos^{-1}\left(\frac{t^2 - a_3^2x^2}{2a_3x}\right) \theta_3 = 180 - \cos^{-1}\left(\frac{x^2 - t^2a_3^2}{2ta_3}\right) \tag{29}$$

[16][17]

6.7 Berechnung des Greifbereichs [Chiara Piccolroaz]

Das Ziel der Berechnung des Greifbereichs des Roboters ist es, das Solver-Team über die maximalen und minimalen x/y-Koordinaten für die Abhol- und Pickup-Positionen der einzelnen Puzzelteile zu informieren.

Nämlich berechnet das Steuerungsteam anhand dieser Koordinaten die Bewegung des Roboters, die dann weiter an das Visualisierungsteam weitergeleitet wird. Sind diese Abhol- und Pickup-Punkte nicht erreichbar, so kann keine inverse Kinematik berechnet

werden und keine Informationen an das Visualisierungsteam weitergegeben werden.

Zum ersten wurden mittels der Methoden *checkreachabilityatBase*() und *getmaxrange*() in *robot.py* die minimalen und maximalen Streckungen des Roboterarms berechnet, welche 0 cm und 1433 cm betragen (Figure 66).

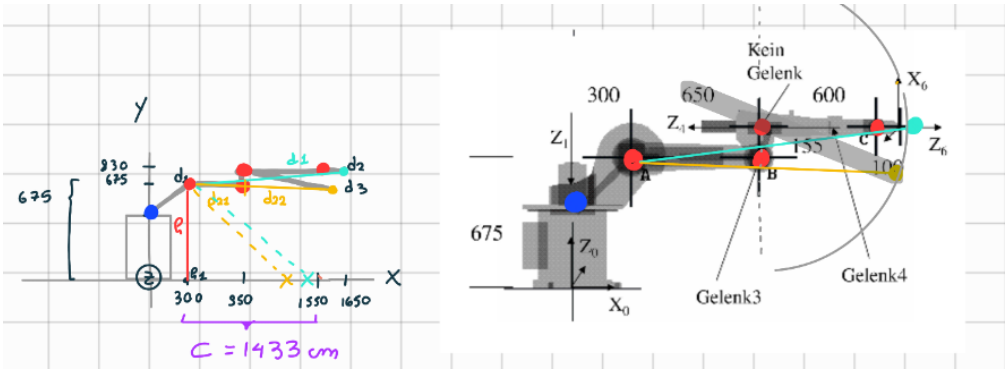


Abbildung 66: Maximale Streckung des Roboterarms

Mithilfe des Satzes des Pythagoras und trigonometrischen Funktionen wurde dann eine potenzielle maximale Fläche berechnet, innerhalb derer das fertige Puzzle optimal plaziert werden könnte (Figure 67).

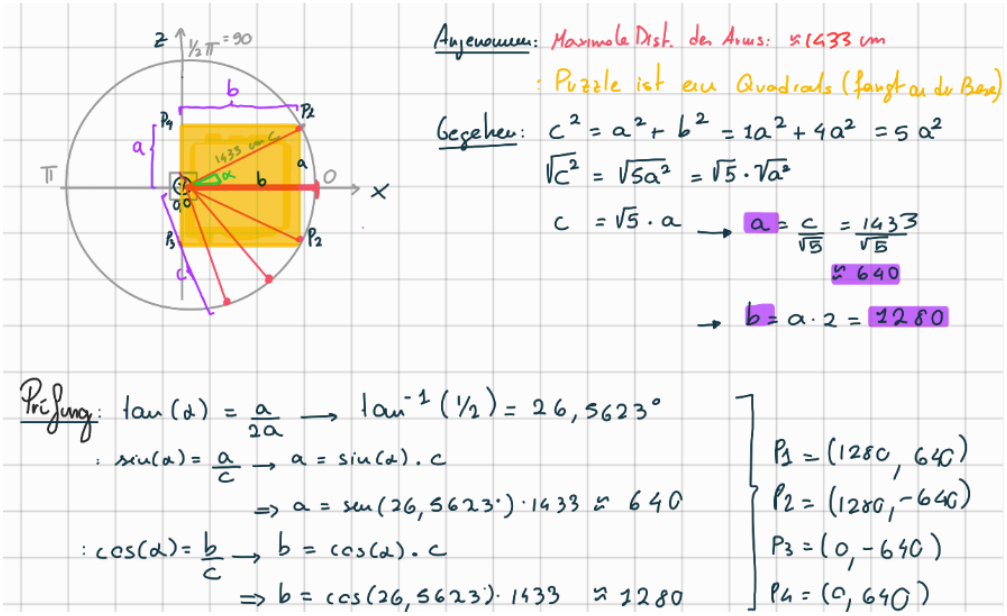


Abbildung 67: Potenzielle maximale Fläche des fertigen Puzzles

6.8 Kollisionsprüfung gegenüber der Basis [Johannes Decker]

Im Rahmen des zweiten Meilensteins MMS2 Basic functionality and Inverse kinematics"des Control-Teams wurde das Thema Kollision des Roboters diskutiert. Zuerst wurde es für sinnvoll erachtet eine Kollisionsprüfung gegenüber der Basisebene ($z = 0$) durchzuführen, da der Roboter für die Manipulation der Puzzleteile sich stehts nah an dieser bewegt. Es sollte dabei verhindert werden das der Roboter die Basisebene durchdringt und im Raum für $z \leq 0$ agiert, da Nutzung dieses Bewegungsraums die Grenzen eines realen Roboters überschreitet und keine sinnvolle Abstraktion eines realen Roboters darstellt. Da dieser Fall als potenziell mögliche Lösung der inversen Kinematik nicht zur Anwendung kommen darf, sollte eine Logik zur Kollisionsprüfung gegenüber der Basis entwickelt werden. Der gegebene Roboter ist ähnlich zu einem menschlichen Arm aufgebaut. Das heißt alle Segmente des Roboters folgen sequenziell aufeinander, sodass jedes Segment des Roboters mit maximal zwei Gelenken verbunden ist. Aus dem Aufbau des Roboters lässt sich also schließen das bei einer Kollision gegenüber der Basis, bzw. ein Eindringen in den $z \leq 0$ Raum, sich mindestens ein Gelenk des Roboters im $z \leq 0$ Raum befindet. Es muss also für jeden Bewegungsschritt des Roboters geprüft werden ob sich alle Gelenke im $z \geq 0$ Raum befinden. Dafür wird die Vorwärtskinematik für den Endpunkt des Bewegungsschritts berechnet. Anhand dieser kann für alle Gelenke des Roboters die Position in z-Richtung bestimmt und geprüft werden. Wenn der Wert der z-Koordinate eines jeden Gelenks des Roboters ≥ 0 ist, liegt keine Kollision gegenüber der Basis vor. Eine erste Version der Kollisionsprüfung gegenüber der Basis wurde durch die Funktion `base_collision` in `src/control/pkg_control/pkg_control/robot.py` implementiert.

Im weiteren Verlauf des Meilensteins MMS2 Basic functionality and Inverse kinematics"viel jedoch auf das eine Kollisionsprüfung gegenüber der Basis aufgrund des Aufbaus des Roboters komplett ausgeschlossen werden kann. Aus Figure 68 kann abgelesen werden das die Länge des 2. Segmentes (Öberarm") 25cm kürzer ist als die Länge des 1. Segments. Dadurch ist eine Kollision gegenüber der Basis für das Gelenk 3 und alle vorherigen Segmente und Gelenke des Roboters unmöglich. Ebenfalls können durch die Inverse Kinematik auch keine Konfigurationen der Gelenkwinkel mehr gefunden werden, in welchen der $z \leq 0$ Raum durchfahren wird. Aus diesem Grund wurde die Entwicklung und Implementierung der Funktion `base_collision` nicht weiterverfolgt. Die Funktion findet auch in der finalen Implementierung keine Verwendung.

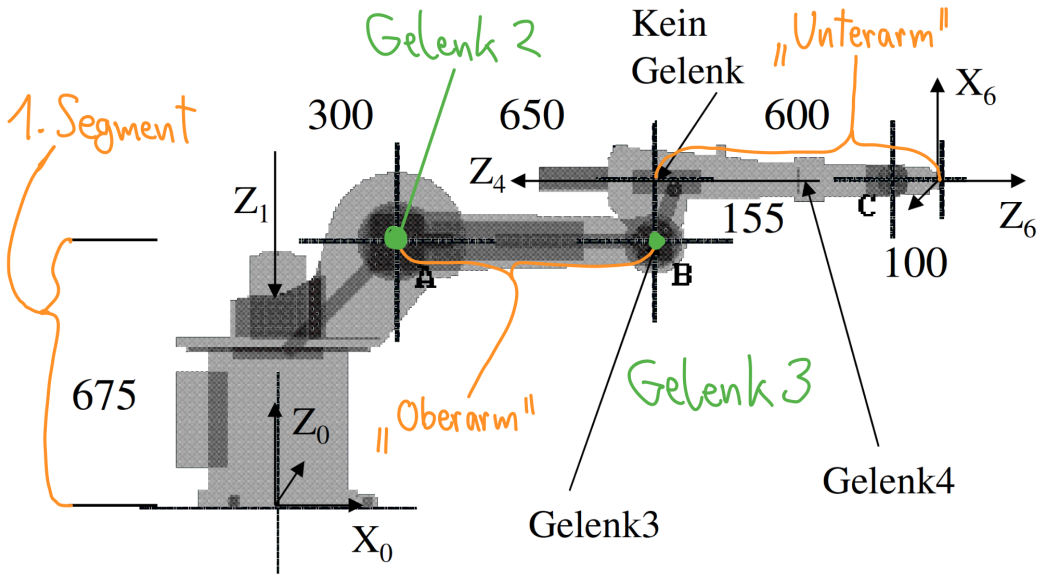


Abbildung 68: Aufbau des Roboters KUKA-KR6

6.9 Robotsbewegung-Wrapper [Chiara Piccolroaz]

Wie in Abbildung 69 gezeigt, sind die Algorithmen für die endgültige Roboterbewegung in einer Wrapper-Funktion zusammengefasst. Diese Funktion nimmt als Eingabe die Start- und Endtransformationsmatrix vom Solver-Team und leitet die interpolierten Gelenkkonfigurationen an das Visualisierungsteam weiter. Die Interpolation wird durch den oben dargestellten De-Casteljau-Algorithmus, den SLERP-Algorithmus und die inverse Kinematik umgesetzt.

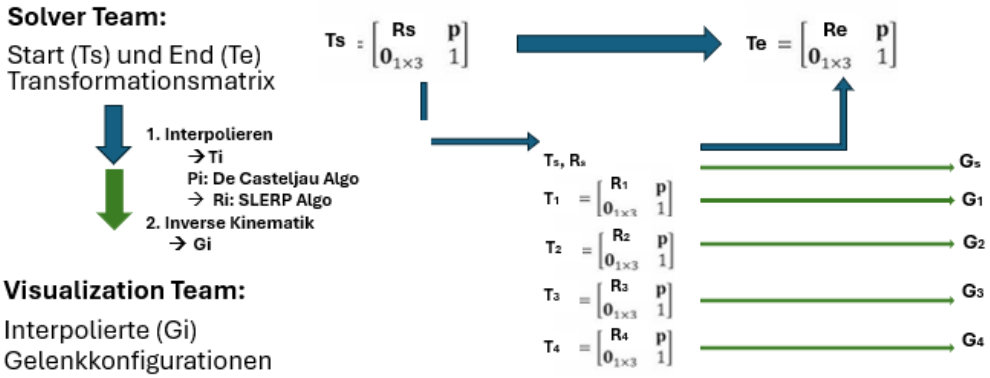


Abbildung 69: Zweck und Inhalt von dem Robotsbewegung-Wrapper

6.10 SLERP - Spherical Linear Interpolation [Chiara Piccolroaz]

Ziel des SLERP-Algorithmus ist es, eine sanfte Rotation des Roboterkopfs von einer Startorientierung zur Zielorientierung zu ermöglichen. Dies ist wichtig, um unerwünschte (visuell uninteressante oder unrealistische) Rotationen zu vermeiden.

Im Fokus stehen die sogenannten Frame-Rotationen. Bei der Rahmenrotation ist der betrachtete Punkt statisch, während sich das Koordinatensystem bewegt.

Aus Performance- und Robustheitsgründen verwendet der Code Hilfsfunktionen aus dem Python-Package `scipy.spatial.transform`, das auf Quaternionen basiert. Diese stellen eine Möglichkeit zur Darstellung von Rotationen dar, neben anderen wie der Rotationsmatrix, den Euler-Winkeln und der Axis-Angle-Darstellung.

Im Vergleich zu diesen bieten Quaternionen einige Vorteile. Erstens ermöglichen sie eine kompakte Darstellung von Rotationen mit nur vier Werten, was den Rechenaufwand reduziert und schnellere Berechnungen ermöglicht. Zudem tritt kein Gimbal Lock auf, eine unerwünschte Dimensionreduzierung bei der Verwendung von Euler-Winkeln, die die Bewegungsfreiheit einschränkt. Schließlich erlauben Quaternionen eine einfache und kontinuierliche Interpolation zwischen zwei Rotationen, wie sie der SLERP-Algorithmus ermöglicht.

SLERP steht für Spherical Linear Interpolation und wird zur Interpolation zwischen zwei Punkten auf einer Kugel, $p0$ und $p1$, verwendet.

Der geometrische SLERP setzt voraus, dass jeder Punkt auf einer Kurve eine lineare Kombination seines Start- und Endpunkts ist, wie in der unten gezeigten Formel und Abbildung dargestellt (Figure 70).

$$\operatorname{slerp}(p_0, p_1; t) = \frac{\sin\left[(1-t)\Omega\right]}{\sin\Omega} p_0 + \frac{\sin[t\Omega]}{\sin\Omega} p_1 .$$

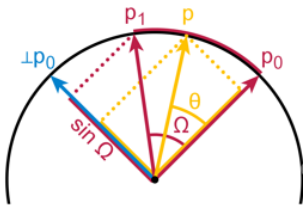


Abbildung 70: Formel und Skizze für den geometrischen SLERP

Der Quaternionen-SLERP basiert auf demselben Prinzip. Gegeben sind die Anfangsorientierung Os, die Zielorientierung Oe und ein Zeitparameter t. Es wird ein Weg zwischen zwei Orientierungen interpoliert, der die kürzeste Bewegung auf einem Großkreisbogen darstellt (Figure 71).

Da die Darstellung und Implementierung von Quaternionen eher komplex ist, haben wir uns in diesem Projekt entschieden, alternative Methoden für die Rotationsinterpolation zu testen. So wurde die Axis-Winkel-Darstellung genutzt, deren Schritte hier zusammengefasst werden (Figure 72):

1. Rotationsmatrizen R_s und R_e aus den Transformationsmatrizen T_s und T_e extrahie-

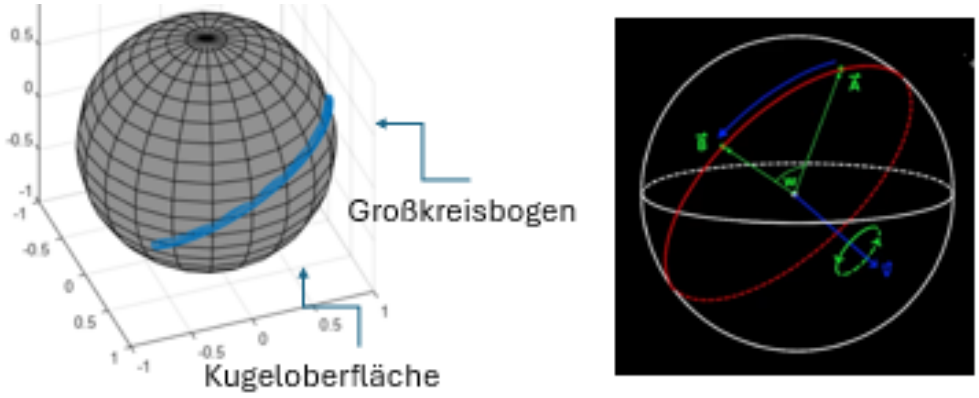


Abbildung 71: Skizze für den geometrischen SLERP

ren.
2. Rotationsmatrix R_{se} von R_s finden: R_{se} = inv R_s * R_e
3. Axis-Winkel-Darstellung aus Rse rekonstruieren
4. Winkel von 0 bis θ linear interpolieren.
5. Rotationsmatrizen aus der Axis-Winkel-Darstellung rekonstruieren.
6. Transformationsmatrizen aus den Rotationsmatrizen und Positionen rekonstruieren.

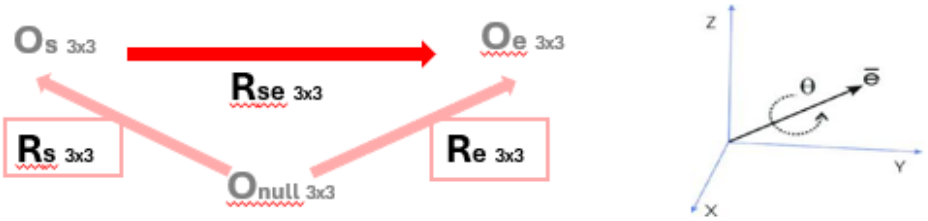


Abbildung 72: Skizze der wichtigsten Implementierungsschritte (2. und 3.)

6.11 Interpolation der Bewegungskurve durch den Algorithmus von De Casteljau [Johannes Decker]

Im Allgemeinen dient der Algorithmus von De Casteljau dazu die Funktion für eine Kurve im Raum aufzustellen, welche sich an mindestens drei Punkten orientiert. Die Funktion interpoliert dabei zwischen allen gegebenen Punkten. Dabei stellen der erste und der letzte gegebene Punkt den Anfangs- und Endpunkt der interpolierten Kurve da. Alle weiteren gegebenen Punkte dienen lediglich zur Orientierung für die Kurve und liegen daher meist nicht auf der interpolierten Kurve. Im Rahmen dieses Projektes soll der Algorithmus von De Casteljau dazu dienen die translatorische Bewegung des Endeffektors geschmeidiger zu machen. Statt die Bewegung des Endeffektors aus mehreren linearen Bewegungen zwischen dem Anfangs- und End- sowie den Hilfspunkten zusammenzusetzen, siehe Figure 73, kann durch die Interpolation mit dem Algorithmus von De Casteljau

eine kurvenförmige Bewegung erreicht werden.

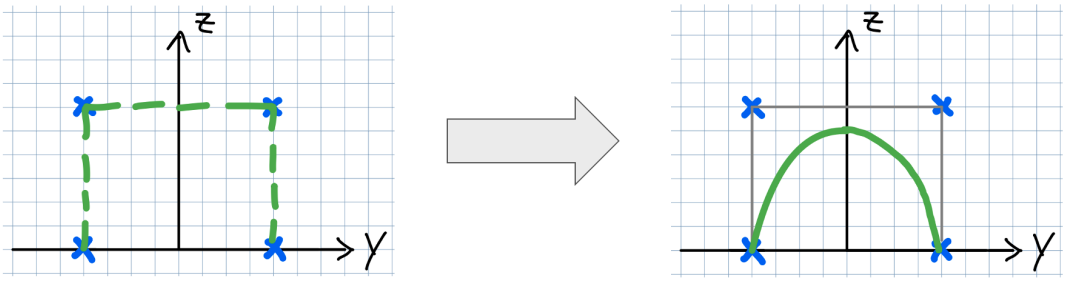


Abbildung 73: Beispielhafte Interpolation einer Kurve aus vier Punkten

Die translatorische Bahn des Endeffektors setzt sich dann aus beliebig vielen Teilabschnitten zusammen, welche durch die Funktion für die Interpolation aus dem Algorithmus von De Casteljau berechnet werden. In Figure 74 ist die Interpolation der Transformationsmatrix T vom Startpunkt einer Bewegung bis zu deren Endpunkt dargestellt. Dabei werden sowohl der Translationsvektor p Als auch die Rotationsmatrix R für jeden Teilschritt i interpoliert.

$$T_s = \begin{bmatrix} R_s & \rho_s \\ 0_{1\times3} & 1 \end{bmatrix} \quad T_i = \begin{bmatrix} R_i & \rho_i \\ 0_{1\times3} & 1 \end{bmatrix} \quad T_{i+1} = \begin{bmatrix} R_{i+1} & \rho_{i+1} \\ 0_{1\times3} & 1 \end{bmatrix} \quad \dots \quad T_e = \begin{bmatrix} R_e & \rho_e \\ 0_{1\times3} & 1 \end{bmatrix}$$

Abbildung 74: Interpolation des Translationsvektors p von Startposition zur Endposition über beliebig viele Teilschritte i

Die Interpolation zwischen den gegebenen Punkten durch den Algorithmus von De Casteljau erfolgt dann in mehreren Stufen. Dabei wird in jeder Stufe erneut zwischen allen Punkten der vorherigen Stufe interpoliert, siehe Figure 75.

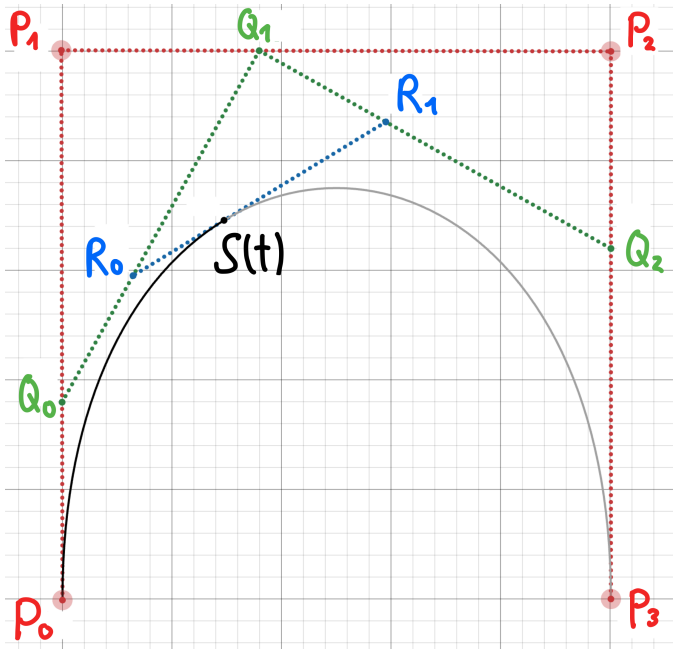


Abbildung 75: Interpolation zwischen den gegebenen Punkten P_i. Die Punkte Q_i werden in der ersten Interpolationsstufe erzeugt, die Punkte R_i in der zweiten, und der Punkt $S(t)$ in der dritten. Dabei stellt $S(t)$ einen beliegiben Punkt der interpolierten Kurve da. Dieses Beispiel ist angelehnt an https://www.desmos.com/calculator/s78usaowv9?lang=de.

Die Funktion für die Kurve der interpolierten Bewegung $S(t)$ ergibt sich dann aus den einzelnen Funktionen für die Interpolation zwischen zwei Punkten. Einzelne Punkte für die Funktion $S(t)$ können dann für beliebe Werte von t berechnet werden, wobei $0 \leq t \leq 1$ sein muss. Dabei ist $S(0) = P_{Start}$, wobei in Figure 75 $P_0 = P_{Start}$ und $S(1) = P_{Ende}$, wobei in Figure 75 $P_3 = P_{Ende}$.

$$
\begin{aligned}
&Q_0(t) = (1-t)P_0 + tP_1; \quad Q_1(t) = (1-t)P_1 + tP_2; \quad Q_2(t) = (1-t)P_2 + t; P_3 \\
&R_1(t) = (1-t)Q_1(t) + tQ_2(t) = (1-t)^2 P_1 + 2t(1-t)P_2 + t^2 P_3 \\
&R_0(t) = (1-t)Q_0(t) + tQ_1(t) = (1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2 \\
&S(t) = (1-t)R_0 + tR_1 = (1-t)^3 P_0 + \left(3t - 6t^2 + 3t^3\right) P_1 + 3t^2(1-t)P_2 + t^3 P_3 \\
&\textit{wobei} \quad 0 \leq t \leq 1
\end{aligned}
$$

Die Interpolation der Bewegungskurve durch den Algorithmus von De Casteljau wurde in der Funktion `int_pos_casteliau` in `src/control/pkg_control/pkg_control/robot.py` implementiert. Die Funktion benötigt für die Interpolation Start- und Endpunkt der Bewegung und die Anzahl der Teilschritte, für welche die Punkte $S(t)$ auf der interpolierten Bewegungskurve berechnet werden sollen. Das Intervall von t wird dabei auf die Anzahl der Teilschritte aufgeteilt, wodurch entsprechend viele Punkte $S_i(t_i)$ berechnet werden.

Optional können beim Aufruf der Funktion `int_pos_casteliau` auch zwei Hilfspunkte angegeben werden, um die Form der interpolierten Bewegungskurve zu beeinflussen. Wenn auf die Angabe von Hilfspunkten verzichtet wird, berechnet die Funktion diese anhand von Start- und Endpunkt. Hierfür werden aus den X- und Y-Komponenten des Start- und Endpunktes Polarkoordinaten berechnet. Die Hilfspunkte besitzen dann jeweils denselben Winkel des korrespondierenden Punktes und liegen in einem fest definierten Radius vom Koordinatenursprung von 75cm in 35cm Höhe (also Z = 0,35). Für die Interpolation werden die Hilfspunkte dann wieder in kartesische Koordinaten umgerechnet. Anschließend erfolgt die Interpolation zwischen allen Punkten P_i, Q_i und R_i für alle i um alle Punkte $S_i(t_i)$ zu erhalten, welche dann von der Funktion zurückgegeben werden.

7 Master Node **[Simon Mederer]**

Um einen zentralen Start und Stop aller Nodes mittels einer eigenen GUI zu realisieren wurde eine sogenannte Master Node entwickelt.

7.1 Grundidee

Die Grundidee ist, dass der Start des gesamten Systems über eine Zentrale Node synchronisiert wird, bzw. der aktuelle Zustand der einzelnen Komponenten dem Nutzer über eine grafische Benutzeroberfläche zugänglich gemacht wird. Dazu wurde für alle relevanten Nodes zum einen eine Startuproutine entwickelt und ein Watchdog, welcher den Ausfall der jeweiligen komponente signalisiert, integriert. Zudem soll, beim Ausfall einer Node der Nutzer die Möglichkeit haben alle Nodes zu stoppen. Die dabei resultierende Architektur ist unter Abbildung 76 zu sehen.

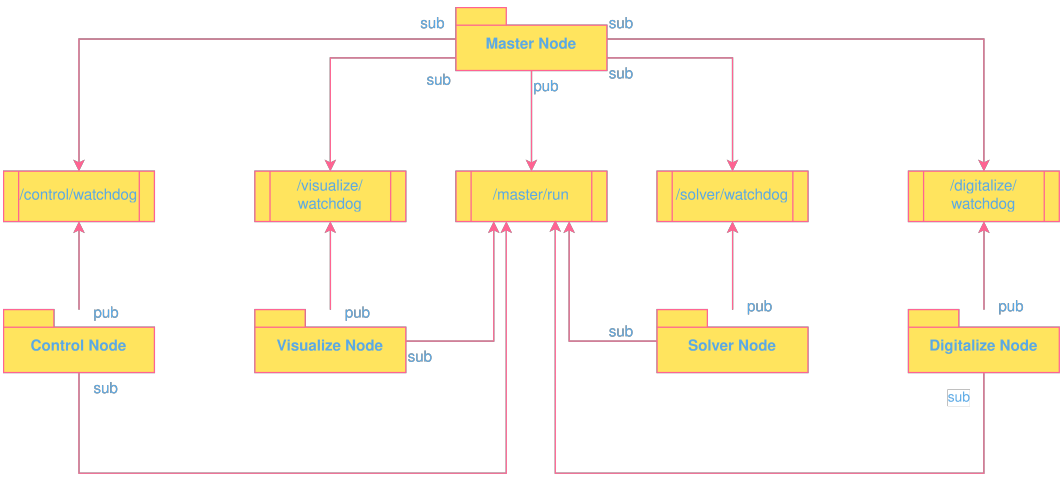


Abbildung 76: Master Node Architektur

Der geplante Ablauf ist folgender. Zu Beginn werden alle Nodes über ein Launch-File

gestartet, darunter auch die Master Node, woraufhin alle Nodes auf das Startsignal des Masters warten. Ist dieses geschickt beginnen alle Nodes gleichzeitig mit ihren jeweiligen Aufgaben.

Wenn der Master das Stop Signal sendet, werden alle Nodes abgebrochen.

7.2 User Interface

Um den Status aller Nodes zu visualisieren wurde eine graphische Benutzeroberfläche mithilfe des QT5 Frameworks erstellt. Diese ist in Abbildung 77 abgebildet.

Abbildung 77: Master GUI

Dieses besteht aus 4 Labels und 4 Anzeigen, die den aktuellen Status jeder Node visualisieren, wie sie durch die *Watchdog Node*, beschrieben in 7.3, ermittelt werden.

Zudem sind zwei Buttons in die Oberfläche integriert, ein **Start** und ein**Stop** Button, die jeweils den gesamten Vorgang starten oder stoppen. Der Start Button kann zudem erst gedrückt werden, wenn alle Nodes stabil laufen.

7.3 Watchdog

Die Logik des Masters ist in der Watchdog Node realisert, welche eine ROS2 Node ist. Diese besteht aus vier Subscribern, eine für jede Komponente und einem Publisher, der für die Start und Stop Befehle verantwortlich ist. Zusätzlich wurde ein Timer integriert, der zyklisch den Status aller Nodes ermittelt und die Zeit speichert wann das letzte Lebenszeichen der Node gekommen ist. Ist dieser Zeitpunkt mehr als 5 Sekunden vergangen, wird die Node als ausgefallen betrachtet und dies der GUI weitergegeben.

Damit der Master sinnvolle Daten erhält muss in jede Node ein Watchdogmodul integriert, welches jede Sekunde *True* published um dem Master zu signalisieren, dass diese noch planmäßig läuft.

8 Workspace **[Lucas Schlager / Christoph Schwarz]**

Die Entwicklungsumgebung wurde mithilfe von Docker und Devcontainer realisiert. Diese Herangehensweise bietet mehrere Vorteile, darunter das Einsparen von Ressourcen, da keine virtuelle Maschine aufgesetzt werden muss.

Ein bedeutender Vorteil der Nutzung von Docker für diesen Use Case ist die Konsistenz der Entwicklungsumgebungen. Unabhängig davon, auf welchem Rechner das Projekt bearbeitet wird, kann durch das standardisierte Docker-Image eine identische Umgebung geschaffen werden, die überall gleich läuft. Dies minimiert Kompatibilitätsprobleme und erleichtert die Zusammenarbeit im Team erheblich. Zudem ermöglicht Docker eine einfache Skalierung und Verwaltung von Abhängigkeiten, was besonders in größeren Projekten von Vorteil ist.

Die aktuelle Umgebung kann immer mit einem einfachen ‘docker pull‘ aktualisiert werden. Das Docker-Image wird bei einer Änderung mit Hilfe der GitLab CI gebaut und auf die GitLab Registry hochgeladen. Über eine ‘devcontainer.json‘ konnte zusätzlich die IDE, in diesem Fall VSCode, mit Plugins konfiguriert werden.

Eine Herausforderung bestand darin, GUI-Anwendungen aus dem Docker-Image auf das Host-System zu pipen. Dies war unter Linux weniger aufwendig, jedoch hatten wir unter Windows einige Probleme. Diese konnten jedoch mit verschiedenen Skripten und Programmen gelöst werden, sodass unsere Entwicklungsumgebung sowohl unter Windows als auch unter Linux einwandfrei funktioniert.

9 Visualizer

9.1 Was war das zu lösende Problem?

Die Anforderungen an das Teilmodul umfassten das Modellieren und Texturieren des Puzzles, die Beweglichkeit des Roboters zu gewährleisten sowie die korrekte Anordnung

seiner Einzelteile. Zusätzlich war die Entwicklung der Logik erforderlich, die das Aufnehmen und Ablegen der Puzzleteile steuert.

Zusätzlich stellten wir den Workspace für die Entwicklung dieser wurde unter verwendung von Docker und Devcontainer in VSCode realisiert.

9.2 Was haben wir gemacht? / Wie haben wir die Lösungen umgesetzt / Implementierung

Zu Beginn unseres Projekts bestand unser erstes Ziel darin, einen einheitlichen Workspace für das gesamte Team zu schaffen, um zu verhindern, dass bei der Integration der Arbeit Unterschiede in den Standardkonfigurationen Probleme verursachen. Ein einheitlicher Workspace, der automatisch alle notwendigen Abhängigkeiten installiert, vereinfacht zudem den Entwicklungsprozess. Die Docker-Container wurden mit Hilfe von GitLab CI/CD erstellt und in die GitLab Docker-Registry hochgeladen, von wo aus sie dann von allen Projektteilnehmern über die `devcontainer.json` heruntergeladen werden konnten. Eine Herausforderung bestand darin, UI-Programme aus Docker heraus zu pipen, sodass das Fenster auf dem Host-Rechner angezeigt wird. Auf Linux-Systemen war dies weniger problematisch, jedoch traten anfangs Schwierigkeiten unter Windows auf, die jedoch ebenfalls gelöst wurden.

Nachdem der Workspace eingerichtet war, konnten wir uns den ersten Aufgaben der 3D-Visualisierung widmen. Zunächst integrierten wir den Roboter in Coin3D. Dazu wurden mehrere sogenannte IV-Dateien, die jeweils einen Arm des Roboters enthalten, durch Imports in eine große IV-Datei zusammengefügt. An jedes Import-Statement fügten wir die Transformationen der DH-Parameter des Control-Teams ein. Als der Roboter anschließend nicht mehr korrekt zusammenpasste, positionierten wir die einzelnen Arme neu, sodass der Roboter als ein zusammenhängendes Objekt im 3D-Viewer existiert (Figure 78a). Zusätzlich implementierten wir Transformationen, die es dem Roboter ermöglichen, sich realitätsgetreu über seine Gelenke zu bewegen. Durch die Trennung der einzelnen Arme mittels eines `SoSeparator` und das Einfügen der Transformationen kann nun über eine `SoSearchOperation` der Knoten im Programm gefunden werden, was die Animation des Roboters ermöglicht.

Nachdem der Roboter fertiggestellt und beweglich gemacht wurde, konnten wir uns unserem nächsten Projektziel zuwenden, die erstellung des Puzzelteils. Zuerst führten wir einen Test durch, um die Leistungsfähigkeit von Coin3D in einem Container mit zahlreichen Texturen zu testen. Für diesen Test platzierten wir 1000 unterschiedliche Bilder auf Objekten und führten ein Rendering durch. Das Ergebnis war positiv, da der 3D-Viewer weiterhin flüssig und ohne Verzögerungen bedient werden konnte.

Vom Solver-Team erhalten wir eine PNG-Datei (Textur des Puzzles) und eine SVG-Datei (Kontur des Puzzles), die in ein dreidimensionales Objekt umgewandelt werden müssen.

Dazu laden wir zunächst die Punkte der Kontur in Coin3D mittels eines `SoCoordinate3`-Objekts. Um das dreidimensionale Puzzle zu erstellen, werden dieselben Punkte erneut, jedoch mit einem Höhenoffset, ins Programm geladen. Da das SVG lediglich die Kontur abbildet und die Mitte des Puzzles keine Punkte enthält, ist es notwendig, die Kontur zu triangulieren. Dies erfolgt mit Hilfe der Bibliothek Poly2Tri.
Anschließend wurde die Textur mit Hilfe eines `SoTexture`-Objekts auf das Puzzle-Teil aufgetragen. Nachdem die Koordinaten in der SVG-Datei mit den Pixeln in der PNG-Datei abgeglichen wurden, können wir nun die Textur präzise über das dreidimensionale Objekt legen (Figure 78). Danach wird noch der Mittelpunkt des Puzzelteils auf den Bildmittelpunkt des PNG gesetzt, da dies notwendig für die anderen Teams war.

Schnittstellen implementierung:

1. */control/angles* Über diese node werden die Bewegungen so wie das Ablegen und Aufnehmen eines Puzzelteils an den Roboter gesendet.

2. */digitalize/imgs* Über diese node werden die Puzzelteile als 3D-Objekt gebaut und in eine Map geladen.

3. */solver/startpos* Über diese node werden die Puzzelteile initial an den mitgegebenen Koordinaten im 3D-Raum eingefügt.

Nachdem es möglich war den Roboter anzuzeigen und zu bewegen, ein Puzzelteile aus Punkten und einem PNG zu erstellen, haben wir uns an die Schnittstellen implementierung gesetzt. Zuerst haben wir hierfür die Schnittstelle ***/control/angles"*** implementiert. Diese ist dafür zuständig den Roboter zu bewegen. Dafür mussten wir eine callback Methode implementieren, welche die neuen Winkel in Radiant in ein Array speichert. Dieses musste von einem Mutex geschützt werden, da es sonst passieren kann, dass der SoIdelSensor im selben Moment auf das Array zugreift. Der SoIdleSensor wurde für alle Schnittstellen implementiert. Dieser stellt sicher, dass Änderungen am Szenegraphen nur dann geschehen, wenn sich dieser im Idle-Zustand befindet. Wenn dieser Aufgerufen wird, wird die Methode ***set_rotation_angle_robot()*** ausgeführt, welche dann die Winkel aller Arme neu setzt. Dies geschieht wie zuvor schon erwähnt über eine SoSearch und der Transformationsanwendung auf diese.

Die nächste Schnittstelle ist die ***/digitalize/imgs"***, aus welcher die Punkte und PNG´s der einzelnen Puzzelteile geholt werden. Aus dieser message werden die Puzzelteile dann verarbeitet und an einen SoSeperator gehangen. Diese werden dann in einer map mit key: id und value: SoSeperator* gespeichert.
Die Schnittstelle ***/solver/startpos"*** wird dann subscribed um die Startpositionen der einzelnen Puzzelteile zu bekommen. Diese werden so übergeben, dass aus diesen die Position und die Translation im Raum auf das Puzzelteil angewendet werden kann.

Die letzte zu implementierende Schnittstelle war die ***/control/puzzlebool"***. Diese Schnittstelle ist dafür da mittzuteilen, ob ein Puzzelteil aufgehoben oder abgelegt werden soll.

Dies geschieht über einen mitgegebenen boolean (true=pick, false=drop) und der ID des Puzzelteils. Hierfür gibt es zwei Fälle:

1. Fall (true=pick): Hier wird das zu nehmende Puzzelteil aus der map geladen und seine aktuelle Transformation wird gelöscht. Danach wird es an den SoSeperator des Roboter gehangen und bewegt sich dadurch mit.

2. Fall (false=drop): Hier wird die aktuelle Position des Puzzelteils über eine SoSearchAction und eine darauffolgende SoGetMatrixAction ermittelt. Sobald diese ermittelt ist, wird das Puzzelteil aus dem SoSeperator des Roboter gelöscht und dem root SoSeperator wieder hinzugefügt. Davor wird allerdings noch die vorher ermittelte Transformation auf den SoSeperator des Puzzelteils angewendet.

Um den Zeitpunkt, wann das Puzzelteil abgelegt werden soll, besser mit der Schnittstelle ***/control/angles*** zu verbinden, wurden die beiden ros nodes zu einer Zusamengefasst. Der Grund liegt hierbei daran, dass wenn die node ***/control/puzzlebool*** gesendet wurde, es sein kann, dass der Roboter sich durch die node ***/control/angles*** schon wieder weiterbewegt hat und dadurch das Puzzelteil nicht genau dort liegt wo es soll. Da nun der boolean Wert mit der ***/control/angles*** node gesendet wird, wird dieser Fehler behohben. Auch mögliche verluste von Nachrichten sind dadurch behoben, da boolean=true nun bedeutet, dass ein Puzzle attached ist und dieser Wert erst wieder auf false wechselt, wenn das Puzzle abgelgt werden soll.

Durch das Zusammenspiel der einzelnen Schnittstellen und der Implementierung der Visualisierung ist es nun also möglich:

1. Den Roboter zu laden.

2. Die Puzzelteile, nachdem diese per ros verschickt wurden als 3D-Objekte zu laden.

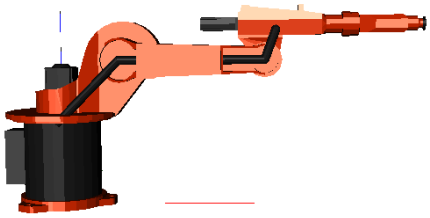
3. Die Puzzelteile an ihren Startpositionen zu platzieren.

4. Den Roboter an die gewünschte Stelle zu bewegen.

5. Ein Puzzelteil mit dem Roboter aufzuheben und mit dem Roboter mitzubewegen.

6. Ein Puzzelteil and er Stelle ab zu setzen, an der der Roboter sich gerade befindet.

Dadurch ist es möglich die komplette Lösung eines beliebigen Puzzles über die Schnittstellen der ros2 nodes darzustellen.



(a) Visualisierte Roboter in Coin3D über .iv-Files

(b) 3D-Objekt eines Puzzelteils

Abbildung 78: 3D-Objekte des Roboters und eines Puzzelteils

10 Ausführen des Programms [Marvin Wolf]

Wie die Entwicklungsumgebung aufgesetzt und das Programm gestartet wird, ist in der Datei README.md im Wurzelverzeichnis des Projekts beschrieben.

Nach dem Start zeigt die Master-Node GUI den Zustand aller ROS Nodes. Sobald alle bereit sind, wird der Ablauf über ”Run” gestartet.

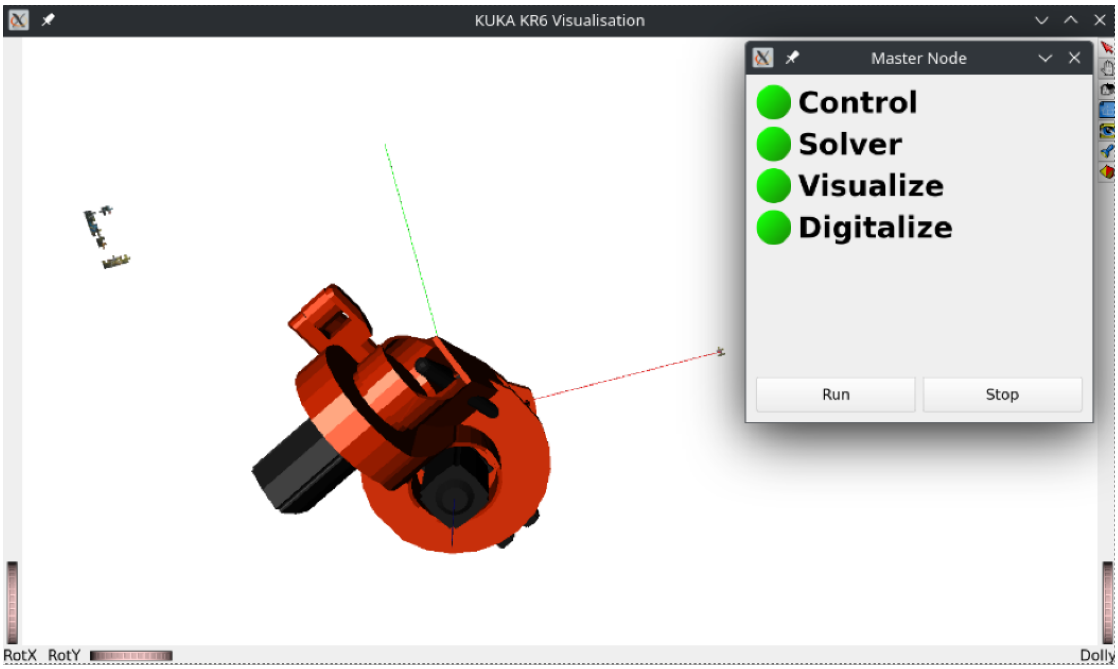


Abbildung 79: Master-Node sowie Roboter

Nach Ablauf des Programms ist das Puzzle gelöst und korrekt angeordnet auf dem Boden ausgebreitet.

Abbildung 80: Endergebnis

Literaturverzeichnis

[1] S. Kauffeld, *Arbeits-, Organisations- und Personalpsychologie für Bachelor*. Springer-Verlag: Berlin Heidelberg, 2014.

[2] W. Burger und M. J. Burge, *Digitale Bildverarbeitung: Eine Algorithmische Einführung Mit Java*. Springer-Verlag, 2009.

[3] D. Kaur und Y. Kaur, "Various image segmentation techniques: a review", *International Journal of Computer Science and Mobile Computing*, Jg. 3, Nr. 5, S. 809–814, 2014.

[4] G. Bradski, "The OpenCV Library", *Dr. Dobb's Journal of Software Tools*, 2000.

[5] A. Clark, *Pillow (PIL Fork) Documentation*, 2015. Adresse: https://buildmedia.readthedocs.org/media/pdf/pillow/latest/pillow.pdf

[6] S. Macenski, T. Foote, B. Gerkey, C. Lalancette und W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild", *Science Robotics*, Jg. 7, Nr. 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074 Adresse: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074

[7] A. Mordvintsev und K. Abid, "Opencv-python tutorials documentation", *Obtenido de https://media. readthedocs. org/pdf/opencv-python-tutroals/latest/opencv-python-tutroals. pdf*, 2014.

[8] A. Kirillov u. a., *Segment Anything*, 2023. arXiv: 2304.02643 [cs.CV]. Adresse: https://arxiv.org/abs/2304.02643

[9] D. Bergmann, *What is zero-shot learning?*, https://www.ibm.com/topics/zero-shot-learning, [Online; Zugriff am 27-Juni-2024], 2024.

[10] S. van der Walt u. a., "scikit-image: image processing in Python", *PeerJ*, Jg. 2, e453, Juni 2014, ISSN: 2167-8359. DOI: 10.7717/peerj.453 Adresse: https://doi.org/10.7717/peerj.453

[11] D. H. Douglas und T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature", *Cartographica: the international journal for geographic information and geovisualization*, Jg. 10, Nr. 2, S. 112–122, 1973.

[12] L. Zhi, Q. Ge und Z. Ji, "Image Matching Algorithm Based on Edge Color Used in Automatic Computer Jigsaw Puzzle", in *2009 Second International Symposium on Knowledge Acquisition and Modeling*, Bd. 3, 2009, S. 269–272. DOI: 10.1109/KAM.2009.149

[13] J. Haviland und P. Corke, "Manipulator Differential Kinematics: Part I: Kinematics, Velocity, and Applications", *IEEE Robotics and Automation Magazine, pp. 2-11*, 2024.

[14] M. A. Arteaga, A. Gutiérrez-Giles und J. Pliego-Jiménez, "A General Overview of Robot Manipulators", in *Local Stability and Ultimate Boundedness in the Control of Robot Manipulators*. Cham: Springer International Publishing, 2022, S. 3–14, ISBN: 978-3-030-85980-0. DOI: 10.1007/978-3-030-85980-0_1 Adresse: https://doi.org/10.1007/978-3-030-85980-0_1

[15] J. Haviland und P. Corke, "A Systematic Approach to Computing the Manipulator Jacobian and Hessian using the Elementary Transform Sequence", *arXiv [cs.RO]*, 2020.

[16] A. Sodemann, *RoboGrok University-Level Robotics Course and Parts Kit — robogrok.com*, `https://robogrok.com`, [Accessed 16-06-2024].

[17] *SYSC 4206 Lecture 7: Inverse kinematics 2, 6DOF robot arm with spherical wrist — youtube.com*, `https://www.youtube.com/watch?v=wDus2EKLg3s&t=1783s`, [Accessed 16-06-2024].

Abbildungsverzeichnis

1 Architekturübersicht grob . . . 5
2 Interfaces - In späteren Kapiteln genauer beschrieben . . . 6
3 Setup zur Fotografie der Puzzleteile mit Stativ, Ringleuchte und Hintergrundgrafik . . . 12
4 Hintergrundgrafik für Aufnahme der Puzzleteile . . . 13
5 Beispiel eines Bilder nach dem Entwicklungsschritt . . . 13
6 Erkannte Abbildungsfläche im unkorrigierten Bild (links) und Resultat der Perspektivkorrektur (rechts) . . . 15
7 Linear entzerrtes und zugeschnittenes Puzzleteil . . . 16
8 Graustufen des Puzzelteils . . . 18
9 Verbesserte Kanten mit Canny Edge Detector . . . 19
10 Nicht geglätette Kontur . . . 19
11 Segmentierung - Canny (links), Threshold (rechts), Watershed (unten) . . . 20
12 Bounding Box Region . . . 21
13 Ergebnisse der Multimask-Output-Bewertung in der Reihenfolge von Maske 1 (links), Maske 2 (rechts) und Maske 3 (unten) . . . 22
14 Vor (links) und nach (rechts) dem Zuschneiden mit der Segmentierungsmaske 23
15 Vergleich von vor (links) und nach (rechts) der Approximierung . . . 24
16 Sequenzdiagramm Kommunikation . . . 26
17 Digitalize ROS Node Architektur . . . 27
18 Übersicht des Solver Prozesses . . . 27
19 Vereinfachtes Klassendiagram . . . 28
20 Input für Hough Transformation . . . 31
21 Output Hough Transformation . . . 31
22 Zoom an Ecke . . . 31
23 Kantentyp: $female$ . . . 32
24 Kantentyp: *male* . . . 32
25 GeoMatcher Kanten-Vorverarbeitung: Ursprüngliche Kanten im lokalen Bildkoordinatensystem (links) und transformierte Punktwolken (rechts, N=30), die paarweise verglichen werden können. . . . 33
26 Beide vorverarbeiteten Punktwolken (interpoliert, N=30) einer einzelnen Kante im GeoMatcher . . . 34
27 Beispiel Puzzleteil . . . 36
28 Farbextrahierung mit Normalenvektor . . . 36
29 Farbliche Kennzeichnung der Patches entlang einer Kante . . . 37

30 Histogramme zweier übereinstimmender Patches 38
31 Vergleich der jeweils korrespondierenden Patches 38
32 Grundidee des XOR Matchers 40
33 Matcher-Scores und Verteilung der Trennschärfe für Kombination aus GeoMatcher und GeoMatcherRGB 46
34 Matcher-Scores und Verteilung der Trennschärfe für Kombination aus GeoMatcher, GeoMatcherRGB und ColorMatcher 47
35 Frame Builder Algorithmus 49
36 Visualisierung eines Puzzle Zustands 54
37 Beispiel eines 7×7 Mock-Puzzles. 58
38 Evaluierungsergebnisse des Depth Search Solvingalgorithmus. 59
39 Laufzeiten pro Lösungsdurchlauf. 60
40 Erfolgsquoten bei verschiedenen Puzzlegrößen. 60
41 Platzierung des Puzzleteilbasen. 61
42 Puzzle-Gitter mit Korrespondierenden Kanten (durch gelbe Pfeile markiert). 61
43 Veranschaulichung der Ermittelung der Fehlermetrik. 62
44 Gewünschte Orientierung des Teils (linke, obere Ecke) durch Optimierung der Orientierung der Basis ${}^{(1)}B$. 64
45 Eckpunkt-Visualisierung der gelösten Puzzels zur Besseren Darstellung des Optimierungsergebnisses, da hier die angenommenen Eckpunkte dargestellt werden. 65
46 Aus den ermittelten Posen zusammengefügte Puzzle. Teilweise abgeschnittene Puzzleteile sind Artefakte der (Debug-) Visualisierung. 65
47 Positionen der Ecken im Bildkoordinatensystem (top, left). 66
48 Positionen der Ecken relativ zum Mittelpunkt des Bildes (padded image). 66
49 Positionierung der Puzzleteile im globalen Koordinatensystem. Wobei ${}_{\mathbf{i}}\mathbf{P}$ und ${}_{\mathbf{j}}\mathbf{P}$ die Positionen von Puzzleteilen **i** und **j** im globalen Koordinatensystem sind. 68
50 Position der Ecken nach Durchführung von Translation $\mathbf{T}$ und Drehung θ. 69
51 Distanzen zwischen benachbarten Ecken ($\mathbf{d_{i2-j1}}$ 70
52 Optimuzer ist in einem lokalen Minima gelandet. ($\mathbf{F}_{\text{err}} = \mathbf{3.75e+3}$) . . . 73
53 Layout des Puzzles vor der Optimierung ($\mathbf{F}_{\text{err}} = \mathbf{1.23e+5}$) 74
54 Layout des Puzzles nach der Optimierung ($\mathbf{F}_{\text{err}} = \mathbf{6.62e+2}$) 74
55 Layout des Puzzles nach linearer Optimierung ($\mathbf{F}_{\text{err}} = \mathbf{6.81+e2}$) 75
56 Layout des linear optimierten Puzzles nach der Verfeinerung mit dem nichtlinearen Optimierer ($\mathbf{F}_{\text{err}} = \mathbf{6.62+e2}$) 75
57 Bildunterschied zwischen dem Layout des Puzzles vor und nach der Verfeinerung. Die Puzzleteile, die während des Prozesses verändert wurden, sind rosa gefärbt. 76
58 ROS2-Schnittstellen Übersicht. 77
59 Sequenzbeispiel zur Veranschaulichung der Callback-Interaktion. 79
60 Klassendiagramm der Control Node. 81
61 ROS2-Schnittstellen Übersicht. 82
62 Sequenzbeispiel zur Veranschaulichung der Callback-Interaktion. 84

63 Test Node Architektur 85
64 Manipulatoren des KUKA-KR6 Roboters 89
65 schematische Darstellungen 90
66 Maximale Streckung des Roboterarms 91
67 Potenzielle maximale Fläche des fertigen Puzzles 91
68 Aufbau des Roboters KUKA-KR6 93
69 Zweck und Inhalt von dem Robotsbewegung-Wrapper 93
70 Formel und Skizze für den geometrischen SLERP 94
71 Skizze für den geometrischen SLERP 95
72 Skizze der wichtigsten Implementierungsschritte (2. und 3.) 95
73 Beispielhafte Interpolation einer Kurve aus vier Punkten 96
74 Interpolation des Translationsvektors p von Startposition zur Endposition über beliebig viele Teilschritte i 96
75 Interpolation zwischen den gegebenen Punkten P_i. Die Punkte Q_i werden in der ersten Interpolationsstufe erzeugt, die Punkte R_i in der zweiten, und der Punkt $S(t)$ in der dritten. Dabei stellt $S(t)$ einen beliegiben Punkt der interpolierten Kurve da. Dieses Beispiel ist angelehnt an https://www.desmos.com/calculator/s78usaowv9?lang=de. 97
76 Master Node Architektur 98
77 Master GUI 99
78 3D-Objekte des Roboters und eines Puzzelteils 104
79 Master-Node sowie Roboter 104
80 Endergebnis 105